ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO MÔN HỌC

HỆ THỐNG NHÚNG VÀ THIẾT KẾ GIAO TIẾP NHÚNG

ĐỀ TÀI

THIẾT KẾ VÀ TRIỂN KHAI MÁY PHÂN TÍCH LOGIC 16 KÊNH 175,5 MS/s TRÊN NỀN TẢNG FPGA TANG PRIMER 20K

Nhóm: 12

STT	Họ và tên	MSSV
1	Nguyễn Đức Phúc	20234032
2	Bùi Đức Công Vinh	20210969
3	Nguyễn Minh Đức	20234001
4	Trần Quang Khải	20234014

Giảng viên hướng dẫn: TS. Đào Việt Hùng

Hà Nội, 2026

PHIẾU GIAO ĐỀ TÀI MÔN HỌC

HỆ THỐNG NHÚNG VÀ THIẾT KẾ GIAO TIẾP NHÚNG

Học kỳ 2025.2

Nhóm thực hiện: Nhóm 12 **Giảng viên hướng dẫn:** TS. Đào Việt Hùng

STT	Họ và tên	MSSV
1	Nguyễn Đức Phúc	20234032
2	Bùi Đức Công Vinh	20210969
3	Nguyễn Minh Đức	20234001
4	Trần Quang Khải	20234014

1. Tên đề tài

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MÁY PHÂN TÍCH LOGIC HIỆU NĂNG CAO TRÊN NỀN TẢNG FPGA

2. Mục tiêu và nội dung thực hiện

Đề tài xây dựng máy phân tích logic LA16 trên FPGA Tang Primer 20K, đáp ứng tối thiểu 8 kênh tại 25 MHz và hướng tới 16 kênh tại 175,5 MS/s. FPGA thực hiện lấy mẫu song song, tạo timestamp, nén RLE, chuyển miền xung nhịp, điều khiển MCP4728/TCA9535 và truyền UDP qua Ethernet. Bo mạch analog bốn lớp đảm nhiệm bảo vệ đầu vào và nguồn, suy giảm, chuẩn hóa mức logic và tạo ngưỡng so sánh. Phần mềm PRISM thu nhận, hiển thị, đo tín hiệu, điều khiển ngưỡng/PWM và giải mã UART, I2C, SPI, CAN. Hệ thống được đánh giá bằng mô hình SIL và chín kịch bản HIL sử dụng nguồn I2S0–DMA về timing, tài nguyên, độ chính xác, thông lượng và giới hạn vận hành.

3. Sản phẩm dự kiến

Nguyên mẫu LA16 và bo mạch analog bốn lớp; mã nguồn FPGA, I2C Master, PRISM và bộ phát ESP32 I2S0–DMA; bộ kiểm thử SIL/HIL, dữ liệu thực nghiệm, báo cáo PnR/STA và tài liệu kỹ thuật.

4. Thời gian thực hiện: Học kỳ 2025.2, theo kế hoạch của học phần.

Hà Nội, năm 2026

GIẢNG VIÊN HƯỚNG DẪN

TS. ĐÀO VIỆT HÙNG

BẢNG PHÂN CHIA CÔNG VIỆC THÀNH VIÊN

Họ và tên & MSSV	Nhiệm vụ chính trong dự án	Mã nguồn & Khối phần cứng chịu trách nhiệm
Nguyễn Đức Phúc MSSV: 20234032 *Trưởng nhóm*	(i) Chủ trì kiến trúc số RTL trên FPGA Gowin; (ii) thiết kế pipeline RLE ba tầng gồm đồng bộ 2FF, so sánh byte song song và bộ đếm delta tách 12+20 bit để đóng timing ở 175,5 MHz; (iii) hiện thực FIFO bất đồng bộ Gray-pointer cho CDC 175,5/50 MHz; (iv) thiết kế packetizer và tích hợp UDP/RMII; (v) xây dựng I2C Master phần cứng để cấu hình MCP4728, TCA9535 và trình tự điều khiển TPS7A8300.	`sampler_rle.sv`, `async_fifo.sv`, `packetizer.sv`, `la16_top.sv`, `i2c_master_core.sv`; datapath FPGA, giao tiếp RMII và bus điều khiển MCP4728/TCA9535.
Bùi Đức Công Vinh MSSV: 20210969	(i) Biên soạn báo cáo kỹ thuật và thiết kế slide thuyết trình; (ii) nghiên cứu, hiện thực heuristic nhận dạng giao thức chạy nền; (iii) xây dựng decoder UART auto-baud, I2C, SPI Mode 0 và CAN Classic 2.0A với bit-destuffing, CRC-15/CAN; (iv) thiết kế TC0–TC8 và mô hình Software-in-the-Loop; (v) quản lý Ring Buffer, phân tích gói UDP và lưu thời gian bằng chu kỳ nguyên để loại bỏ sai số tích lũy do cộng dồn số thực.	`detect.py`, `store.py`, `packet.py`, `la16_paint.cpp`, `run_testcases.py`; Host backend, bộ giải mã, dữ liệu golden và thang kiểm thử SIL.
Trần Quang Khải MSSV: 20234014	(i) Chủ trì thiết kế schematic và layout PCB bốn lớp cho bo mạch mở rộng bảo vệ, suy giảm và dịch mức; (ii) tính toán bộ phân áp bù tần số với $K = 0{,}6$ và hiệu chỉnh $C_2 = 3{,}32$ pF sau khi xét điện dung ký sinh; (iii) triển khai TPD1E05U06, BAT15-04W, TLV3801 và TLV3601; (iv) thiết kế bảo vệ nguồn USB-C bằng TPS259250, LT3045 và TLVH431; (v) lập trình driver I2S0 LCD-parallel DMA mức thanh ghi để phát đồng thời 16 kênh, loại bỏ jitter do bit-banging.	Schematic, layout PCB bốn lớp và mô phỏng SPICE của front-end/power; TPD1E05U06, BAT15-04W, TLV3801/TLV3601, TPS259250, LT3045, TLVH431; `i2s16.cpp` trên ESP32.
Nguyễn Minh Đức MSSV: 20234001	(i) Chủ trì giao diện PRISM bằng CustomTkinter và Matplotlib; (ii) tối ưu render bằng decimation và cơ chế PAUSE snapshot không chặn luồng nhận; (iii) xây dựng Cursor A/B để tính Δt và tần số; (iv) tích hợp widget điều khiển ngưỡng MCP4728 và điện áp PWM 0,9–3,3 V của TPS7A8300; (v) thực hiện đo kiểm chín kịch bản HIL, trích xuất và phân tích báo cáo PnR/STA với setup slack xấu nhất dương 0,208 ns.	`la_gui.py`; giao diện PRISM, Matplotlib Canvas, bộ điều khiển ngưỡng/PWM, dữ liệu đo HIL, báo cáo `rpt`, timing paths và log PnR của FPGA.

LỜI CẢM ƠN

Nhóm 12 trân trọng cảm ơn TS. Đào Việt Hùng, giảng viên hướng dẫn học phần Hệ thống nhúng và thiết kế giao tiếp nhúng (ET4361). Trong quá trình thực hiện đề tài, Thầy đã định hướng phương pháp thiết kế giao tiếp nhúng, kiến trúc hệ thống thời gian thực và tổ chức kiểm thử kỹ thuật. Các góp ý về FPGA, chuyển miền xung nhịp và Ethernet đã giúp nhóm nhận diện giới hạn, hoàn thiện máy phân tích logic LA16 và hình thành phương pháp giải quyết vấn đề có hệ thống.

Nhóm đồng thời cảm ơn Trường Điện – Điện tử, Đại học Bách khoa Hà Nội đã tạo điều kiện học tập và nghiên cứu. Báo cáo khó tránh khỏi thiếu sót; nhóm kính mong nhận được các ý kiến đóng góp để tiếp tục hoàn thiện sản phẩm.

MỤC LỤC

BẢNG PHÂN CHIA CÔNG VIỆC THÀNH VIÊN **4**

LỜI CẢM ƠN **5**

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ **i**

DANH SÁCH BIỂU ĐỒ **v**

DANH SÁCH BẢNG **vi**

TÓM TẮT **vii**

ABSTRACT **viii**

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI **1**

1.1 Đặt vấn đề 1

1.2 Cơ sở thực tế và các hạn chế của nghiên cứu hiện tại 2

1.3 Mục tiêu nghiên cứu 3

1.4 Các đóng góp chính của đề tài 3

1.5 Bố cục báo cáo 4

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP TÍNH TOÁN **5**

Tổng quan chương 5

2.1 Cơ sở lý thuyết lấy mẫu và đồng bộ tín hiệu 5

2.2 Lý thuyết nén dữ liệu RLE và tái dựng thời gian 6

2.3 Cơ sở lý thuyết các giao thức truyền thông số cần giải mã 7

2.3.1 Giao tiếp UART 7

2.3.2 Giao tiếp SPI 8

2.3.3 Giao tiếp I2C 8

2.3.4 Giao tiếp CAN Classic 2.0A 9

2.3.5 Tín hiệu CLK và PWM 9

2.4 Giao thức truyền dữ liệu Ethernet thời gian thực: UDP và TCP 10

2.5 Khảo sát đặc tả phần cứng nền tảng 11

2.5.1 Nền tảng thu nhận Sipeed Tang Primer 20K 11
2.5.2 Nền tảng phát tín hiệu NodeMCU ESP32-WROOM-32 13
2.5.3 Khả năng tương thích điện và phân chia chức năng 14
2.6 Thiết lập mô hình kiểm thử Hardware-in-the-Loop 14
2.6.1 Kiến trúc và luồng dữ liệu của vòng kiểm thử 15
2.6.2 Ánh xạ kênh và đấu nối vật lý 15
2.6.3 Cơ sở điện học của kết nối GND chung 17
2.6.4 Nguyên lý phát song song I2S0 LCD-Parallel và DMA 17
2.6.5 Liên hệ với lấy mẫu bất đồng bộ và bản chất HIL 19
2.7 Cơ sở lý thuyết của mạch phân áp bù tần số và lựa chọn bộ so sánh tốc độ cao 19
2.7.1 Ảnh hưởng của điện dung ký sinh và điều kiện bù tần số 19
2.7.2 Tỷ số phân áp và miền điện áp đồng pha 20
2.7.3 Ngân sách timing cho 12 kênh tốc độ cao 21
2.7.4 Tiêu chí cho bốn kênh tốc độ thấp 22
2.7.5 Giới hạn chịu đựng quá áp một chiều liên tục 22
2.8 Cơ sở lý thuyết của khối tạo điện áp tham chiếu tĩnh và bộ đệm cách ly . 23
2.8.1 Ngân sách sai số và độ phân giải DAC 24
2.8.2 Bộ đệm cách ly nhiễu kick-back 25
2.9 Cơ sở lý thuyết của khối dịch mức xung PWM 27
2.9.1 Yêu cầu dịch mức và lựa chọn kiến trúc 27
2.9.2 Mạng định thiên và giới hạn dòng 28
2.9.3 Băng thông theo điện dung tải 28
2.10 Cơ sở lý thuyết của khối nguồn và bảo vệ nguồn 29
2.10.1 Kiến trúc cây nguồn 5 V và 3,3 V 29
2.10.2 Bảo vệ đầu vào USB-C bằng TVS và eFuse 30
2.10.3 Ngân sách dòng và ổn áp 3,3 V nhiễu thấp 31
2.10.4 Kẹp dòng dội ngược trên rail 3,3 V 31
Kết chương 32

CHƯƠNG 3. TRIỂN KHAI DỰ ÁN **33**

Tổng quan chương 33
3.1 Sơ đồ khối và luồng dữ liệu hệ thống 33
3.1.1 Kiến trúc tổng thể 33

3.1.2 Sơ đồ quy trình xử lý dữ liệu (Flowchart) 34
3.1.3 Sơ đồ tuần tự của hệ thống (Sequence Diagram) 36
3.2 Thiết kế phần cứng FPGA 38
3.2.1 Clock, Synthesis, PnR và Timing Constraints 38
3.2.2 Khối lấy mẫu pipeline và nén RLE 38
3.2.3 FIFO bất đồng bộ Gray-pointer 39
3.2.4 Packetizer và UDP stack 39
3.3 Thiết kế phần mềm Host PRISM 40
3.3.1 Kiến trúc multi-threading và Ring Buffer thread-safe 40
3.3.2 Heuristic algorithm tự động nhận dạng 41
3.3.3 Các bộ giải mã giao thức 41
3.4 Thiết kế và Hiện thực hóa Giao diện Đồ họa PRISM 43
3.4.1 Kiến trúc đa luồng và đồng bộ dữ liệu thời gian thực 43
3.4.2 Bộ dựng dạng sóng Matplotlib và Decimation 44
3.4.3 Cơ chế đóng băng Snapshot không chặn khi PAUSE 44
3.4.4 Cursor A/B đo thời gian và tần số 44
3.4.5 Phân vùng điều khiển và bảng đọc DECODE 45
3.5 Thiết kế và hiện thực hóa mạch bảo vệ và khối thu nhận đầu vào 45
3.5.1 Cấu trúc đường tín hiệu và các lớp bảo vệ 45
3.5.2 Phân tích dịch mức DC tại dải đầu vào 0,9 V 47
3.5.3 Kết cuối song song đường truyền LVDS tại FPGA 48
3.5.4 Phân bố 12 kênh tốc độ cao và bốn kênh 50 MHz 48
3.6 Triển khai khối tạo điện áp tham chiếu và nguồn kéo lên điều khiển được 49
3.6.1 Điện áp tham chiếu comparator 49
3.6.2 Dịch mức và bảo vệ ngõ ra PWM 50
3.6.3 Điều khiển ANY-OUT của TPS7A8300 bằng TCA9535 51
3.6.4 Mạch xả nhanh bằng transistor số DTC114E 52
3.7 Triển khai khối bảo vệ nguồn đầu vào USB-C 54
3.7.1 Cây nguồn và bảo vệ ESD 54
3.7.2 Giới hạn dòng bằng eFuse TPS259250 54
3.7.3 Nguồn 3,3 V nhiễu thấp bằng LT3045 55
3.7.4 Kẹp dòng dội ngược bằng TLVH431 56
Kết chương 56

CHƯƠNG 4. KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ **57**
Tổng quan chương 57
4.1 Chuẩn hóa kiểm thử theo V-Model cho hệ thống nhúng 57
4.1.1 Mạch phần cứng thực tế được đưa vào kiểm thử HIL 57
4.1.2 Ánh xạ thiết kế và kiểm thử theo V-Model 59
4.2 Giải pháp tạo tín hiệu kiểm thử không jitter trên ESP32 60
4.3 Kết quả tài nguyên tổng hợp phần cứng trên FPGA 60
4.4 Thiết kế và kết quả các kịch bản kiểm thử 61
4.4.1 TC0 – CALIBRATION 63
4.4.2 TC1 – UART_SWEEP 64
4.4.3 TC2 – I2C_SWEEP 65
4.4.4 TC3 – SPI_SWEEP 66
4.4.5 TC4 – CAN_SWEEP 68
4.4.6 TC5 – CLK_SWEEP 69
4.4.7 TC6 – MIX_ALL 71
4.4.8 TC7 – EDGE_CASES 72
4.4.9 TC8 – BW_STRESS 73
4.5 Giải pháp mô phỏng Software-in-the-Loop 75
Kết chương 75

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **77**
Tổng quan chương 77
5.1 Kết luận 77
5.2 Hạn chế của hệ thống 78
5.3 Hướng phát triển và nâng cấp 78
Kết chương 79

TÀI LIỆU THAM KHẢO **80**

PHỤ LỤC **84**
Appendix 1: Cấu trúc thư mục nguồn dự án LA16 84

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt/ thuật ngữ	Tên tiếng Anh	Diễn giải tiếng Việt
ACK	Acknowledge	Bit xác nhận một byte hoặc khung dữ liệu đã được tiếp nhận hợp lệ.
Auto-baud	Automatic baud-rate detection	Cơ chế tự động ước lượng tốc độ baud từ khoảng thời gian giữa các cạnh tín hiệu.
Bit-destuffing	Bit destuffing	Quá trình loại bỏ các bit chèn theo quy tắc bit stuffing để khôi phục khung CAN ban đầu.
BSRAM	Block Static Random-Access Memory	Bộ nhớ SRAM dạng khối tích hợp trong FPGA, được sử dụng để triển khai FIFO.
CAN	Controller Area Network	Giao thức truyền thông nối tiếp vi sai thường dùng trong ô tô và hệ thống điều khiển.
CDC	Clock Domain Crossing	Quá trình truyền dữ liệu hoặc tín hiệu điều khiển giữa hai miền xung nhịp khác nhau.
CLK	Clock	Tín hiệu xung nhịp dùng làm mốc thời gian cho mạch số hoặc giao thức.
CRC-15/CAN	Cyclic Redundancy Check for CAN	Mã kiểm tra dư thừa tuần hoàn 15 bit dùng để phát hiện lỗi trong khung CAN.
DMA	Direct Memory Access	Cơ chế truyền dữ liệu trực tiếp giữa ngoại vi và bộ nhớ mà không cần CPU xử lý từng phần tử.
DAC	Digital-to-Analog Converter	Bộ chuyển đổi số–tương tự dùng để tạo điện áp ngưỡng điều khiển được.
EMI	Electromagnetic Interference	Nhiễu điện từ có thể ghép vào đường tín hiệu hoặc nguồn của mạch.
ESD	Electrostatic Discharge	Hiện tượng phóng tĩnh điện có thể tạo xung điện áp lớn tại đầu nối và chân linh kiện.
eFuse	Electronic Fuse	Cầu chì điện tử tích hợp giới hạn dòng, khởi động mềm và các cơ chế bảo vệ nguồn.
ESP32	Espressif ESP32 microcontroller	Vi điều khiển được sử dụng làm thiết bị sinh tín hiệu kiểm thử 16 kênh.
FIFO	First-In, First-Out	Bộ đệm trong đó dữ liệu được đọc theo đúng thứ tự đã ghi.
FPGA	Field-Programmable Gate Array	Mạch tích hợp số có cấu trúc phần cứng được cấu hình lại bằng ngôn ngữ mô tả phần cứng.
GUI	Graphical User Interface	Giao diện đồ họa cho phép người dùng cấu hình, quan sát và phân tích dữ liệu đo.

Từ viết tắt/ thuật ngữ	Tên tiếng Anh	Diễn giải tiếng Việt
GBW	Gain-Bandwidth Product	Tích độ lợi–băng thông đặc trưng cho tốc độ đáp ứng vòng kín của bộ khuếch đại thuật toán.
HIL	Hardware-in-the-Loop	Phương pháp kiểm thử có sử dụng phần cứng thật trong vòng lặp tạo và thu nhận tín hiệu.
HUST	Hanoi University of Science and Technology	Đại học Bách khoa Hà Nội.
I2C	Inter-Integrated Circuit	Giao thức nối tiếp đồng bộ hai dây gồm đường dữ liệu SDA và đường xung nhịp SCL.
I2S	Inter-IC Sound	Ngoại vi truyền dữ liệu nối tiếp của ESP32; trong hệ thống kiểm thử được cấu hình ở chế độ LCD-parallel 16 bit.
Jitter	Timing jitter	Sự biến thiên không mong muốn của thời điểm xuất hiện cạnh tín hiệu so với vị trí lý tưởng.
LA	Logic Analyzer	Máy phân tích logic dùng để lấy mẫu, hiển thị và giải mã nhiều kênh tín hiệu số.
LDO	Low-Dropout Regulator	Bộ ổn áp tuyến tính có điện áp sụt thấp, dùng để tạo nguồn sạch cho mạch tín hiệu.
LPF	Low-Pass Filter	Bộ lọc thông thấp cho phép thành phần tần số thấp đi qua và suy giảm nhiễu tần số cao.
LSB	Least Significant Bit	Bit có trọng số thấp nhất; trong DAC, LSB biểu thị bước điện áp lượng tử nhỏ nhất.
LVDS	Low-Voltage Differential Signaling	Chuẩn tín hiệu vi sai biên độ thấp dùng cho đường dữ liệu tốc độ cao.
MS/s	Mega Samples per Second	Đơn vị biểu thị một triệu mẫu trên giây.
PHY	Physical-Layer Transceiver	Khối thu phát tầng vật lý thực hiện chuyển đổi tín hiệu số của MAC sang tín hiệu đường truyền.
PLL	Phase-Locked Loop	Vòng khóa pha dùng để tổng hợp và ổn định xung nhịp lấy mẫu.
PnR	Place and Route	Quá trình bố trí tài nguyên logic và định tuyến kết nối vật lý trên FPGA.
PRISM	LA16 PRISM host application	Phần mềm máy chủ dùng để thu nhận UDP, hiển thị dạng sóng và giải mã giao thức.
PWM	Pulse-Width Modulation	Phương pháp điều chế độ rộng xung để biểu diễn giá trị hoặc điều khiển công suất.
RLE	Run-Length Encoding	Phương pháp nén biểu diễn một trạng thái cùng số chu kỳ tồn tại của trạng thái đó.

Từ viết tắt/ thuật ngữ	Tên tiếng Anh	Diễn giải tiếng Việt
PSRR	Power-Supply Rejection Ratio	Khả năng suy giảm nhiễu từ nguồn cấp truyền tới đầu ra của mạch tương tự.
RFI	Radio-Frequency Interference	Nhiễu ở miền tần số vô tuyến có thể xâm nhập đầu vào mạch tương tự.
RMII	Reduced Media Independent Interface	Giao tiếp rút gọn giữa logic Ethernet số và PHY 10/100 Mbps.
RRIO	Rail-to-Rail Input and Output	Khả năng đầu vào và đầu ra hoạt động gần cả hai rail nguồn.
RTL	Register-Transfer Level	Mức mô tả phần cứng số dựa trên thanh ghi và luồng truyền dữ liệu giữa các thanh ghi.
SIL	Software-in-the-Loop	Phương pháp kiểm thử thuật toán bằng dữ liệu mô phỏng trước khi kết nối phần cứng.
Skew	Inter-channel skew	Độ chênh thời gian giữa các cạnh hoặc thời điểm lấy mẫu của nhiều kênh.
SPI	Serial Peripheral Interface	Giao thức nối tiếp đồng bộ sử dụng các đường SCK, MOSI, MISO và tín hiệu chọn thiết bị.
TC	Test Case	Kịch bản kiểm thử có đầu vào, điều kiện thực hiện và tiêu chí đánh giá xác định.
Timestamp	Timestamp	Mốc thời gian gắn với một mẫu hoặc một lần chuyển trạng thái của tín hiệu.
Transition	Logic transition	Sự chuyển đổi mức logic từ thấp lên cao hoặc từ cao xuống thấp.
TVS	Transient Voltage Suppressor	Diode triệt xung quá độ dùng để kẹp ESD hoặc surge về mức an toàn.
UART	Universal Asynchronous Receiver/Transmitter	Giao tiếp nối tiếp bất đồng bộ sử dụng bit bắt đầu, các bit dữ liệu và bit kết thúc.
UDP	User Datagram Protocol	Giao thức truyền datagram không kết nối, có độ trễ xử lý thấp và không bảo đảm truyền lại.

DANH SÁCH BIỂU ĐỒ

Biểu đồ 2.1 Sơ đồ kết nối và cấu trúc khung UART 8N1 7
Biểu đồ 2.2 Sơ đồ kết nối và từ dữ liệu SPI Mode 0 8
Biểu đồ 2.3 Sơ đồ bus và cấu trúc một transaction I2C 7 bit 8
Biểu đồ 2.4 Tầng vật lý và cấu trúc Data Frame CAN Classic 2.0A 9
Biểu đồ 2.5 Kết nối đo và các tham số thời gian của CLK/PWM 10
Biểu đồ 2.6 Nền tảng Sipeed Tang Primer 20K Dock sử dụng trong LA16 . 12
Biểu đồ 2.7 NodeMCU ESP32-WROOM-32 sử dụng làm bộ phát tín hiệu kiểm thử 13
Biểu đồ 2.8 Sơ đồ khối tạo bốn điện áp tham chiếu điều khiển qua I2C . . . 24
Biểu đồ 2.9 Mạng RC cách ly tại đầu vào tham chiếu của comparator 26
Biểu đồ 2.10 Sơ đồ khối nguồn 5 V và 3,3 V của mạch bảo vệ 29
Biểu đồ 2.11 Bảo vệ nguồn USB-C 30
Biểu đồ 2.12 Sơ đồ chi tiết nguồn LT3045 và mạch kẹp dòng dội ngược TLVH431 32
Biểu đồ 3.1 Sơ đồ khối hệ thống LA16 33
Biểu đồ 3.2 Sơ đồ quy trình thu nhận và phân tích dữ liệu trên phần mềm Host 35
Biểu đồ 3.3 Sơ đồ tuần tự thu nhận, truyền và hiển thị dữ liệu của hệ thống LA16 37
Biểu đồ 3.4 FSM khử bit nhồi và kiểm tra CRC15 của decoder CAN Classic 2.0A 42
Biểu đồ 3.5 Kiến trúc đa luồng của giao diện PRISM 43
Biểu đồ 3.6 Sơ đồ nguyên lý mạch phân áp bù tần số của một kênh đầu vào 46
Biểu đồ 3.7 Mạng RC cách ly điện áp tham chiếu tại đầu vào comparator . . 49
Biểu đồ 3.8 Sơ đồ nguyên lý khối dịch mức PWM sử dụng LSF0204 50
Biểu đồ 3.9 Mạch kéo lên, hạn dòng và bảo vệ ESD tại ngõ ra PWM 50
Biểu đồ 3.10 Sơ đồ nguyên lý nguồn kéo lên điều khiển được TPS7A8300 . . 51
Biểu đồ 3.11 Sơ đồ nguyên lý bộ mở rộng I2C TCA9535 52
Biểu đồ 3.12 Mạch transistor số DTC114E xả nhanh tụ nguồn kéo lên 53
Biểu đồ 3.13 Sơ đồ bảo vệ đầu vào USB-C bằng SMAJ5.0A và TPS259250 . 54
Biểu đồ 3.14 Sơ đồ nguồn 3,3 V nhiễu thấp LT3045 và mạch kẹp TLVH431 . 55
Biểu đồ 4.1 Bo mạch front-end LA16 sau khi lắp ráp linh kiện 58

Biểu đồ 4.2 V-Model cải tiến cho quá trình thiết kế và kiểm thử hệ thống nhúng LA16 59
Biểu đồ 4.3 Kết quả hiển thị TC0 trên phần mềm PRISM 63
Biểu đồ 4.4 Kết quả hiển thị TC1 với 16 luồng UART đa tốc độ 64
Biểu đồ 4.5 Kết quả hiển thị TC2 với các bus I2C hoạt động đồng thời . . . 66
Biểu đồ 4.6 Kết quả hiển thị TC3 và hiện tượng nhận dạng nhầm SPI thành I2C 67
Biểu đồ 4.7 Kết quả hiển thị TC4 với các khung CAN đa tốc độ 69
Biểu đồ 4.8 Kết quả thực nghiệm HIL của TC5 sau khi chạy lại CLK_SWEEP 70
Biểu đồ 4.9 Kết quả hiển thị TC6 khi nhiều giao thức cùng hoạt động 71
Biểu đồ 4.10 Kết quả hiển thị TC7 với các trường hợp biên của decoder . . . 73
Biểu đồ 4.11 Kết quả hiển thị TC8 với 16 xung vuông 50 kHz lệch pha . . . 74

DANH SÁCH BẢNG

Bảng 2.1 Cấu trúc header ứng dụng UDP của LA16 11
Bảng 2.2 So sánh TCP và UDP trong bài toán streaming LA16 11
Bảng 2.3 Đặc tả nền tảng thu nhận Tang Primer 20K 12
Bảng 2.4 Đặc tả nền tảng phát tín hiệu ESP32 13
Bảng 2.5 Phân chia chức năng và giao diện giữa hai nền tảng 14
Bảng 2.6 Luồng xử lý trong mô hình kiểm thử HIL 15
Bảng 2.7 Ánh xạ CH0–CH7 từ ESP32 tới PMOD2 16
Bảng 2.8 Ánh xạ CH8–CH15 từ ESP32 tới PMOD3 và kết nối GND 16
Bảng 2.9 Trình tự hoạt động của bộ phát I2S0 LCD-Parallel DMA 18
Bảng 2.10 So sánh phương pháp sinh tín hiệu bằng phần mềm và I2S0 DMA 18
Bảng 2.11 So sánh comparator cho 12 kênh front-end tốc độ cao 21
Bảng 2.12 So sánh comparator cho bốn kênh front-end 50 MHz 22
Bảng 2.13 So sánh DAC bốn kênh tạo điện áp tham chiếu 25
Bảng 2.14 So sánh op-amp bốn kênh cho bộ đệm điện áp tham chiếu 26
Bảng 2.15 So sánh IC dịch mức cho xung PWM điều khiển được 27
Bảng 2.16 Ngân sách điện dung tải của một kênh LSF0204 28
Bảng 3.1 Ánh xạ các khối triển khai chính 34
Bảng 3.2 Cấu trúc header UDP 16 byte 40
Bảng 3.3 Quy tắc chính của các decoder 43
Bảng 3.4 Vai trò và yêu cầu bố trí của các phần tử front-end 47
Bảng 3.5 Phân bố kênh, comparator và điện áp tham chiếu 48
Bảng 3.6 Quy ước điều khiển TPS7A8300 bằng TCA9535 52
Bảng 3.7 Trình tự đổi điện áp nguồn kéo lên PWM 53
Bảng 4.1 Kết quả sử dụng tài nguyên sau PnR của LA16 61
Bảng 4.2 Tổng hợp thiết kế, vai trò và kết quả các kịch bản TC0–TC8 . . . 62

TÓM TẮT

Máy phân tích logic là công cụ thiết yếu để quan sát tín hiệu số, gỡ lỗi giao thức và xác minh hệ thống nhúng. Tuy nhiên, thiết bị giá rẻ thường hạn chế về số kênh hoặc tốc độ lấy mẫu. LA16 thu nhận đồng thời 16 kênh cho tín hiệu từ 25 MHz và truyền dữ liệu về máy tính qua Ethernet 100 Mbps, trong khi dòng mẫu thô vượt xa băng thông của PHY RMII RTL8201F.

Hệ thống triển khai trên kit Sipeed Tang Primer 20K sử dụng FPGA Gowin GW2A-LV18PG256C8/I7. rPLL nhân xung nhịp 27 MHz lên 175,5 MHz, tương ứng độ phân giải 5,70 ns. Cả 16 kênh được chốt trên cùng cạnh xung nhịp nên độ lệch kiến trúc giữa các kênh bằng 0 chu kỳ mẫu. Phần cứng áp dụng Run-Length Encoding (RLE), chỉ tạo bản ghi độ lệch thời gian và trạng thái 16 bit khi trạng thái đổi. Bản ghi đi qua FIFO bất đồng bộ Gray-pointer để thực hiện Clock Domain Crossing (CDC) từ miền 175,5 MHz sang 50 MHz, rồi đóng gói UDP qua cổng 5001. Cơ chế này giảm tải đường truyền và báo tràn khi mật độ chuyển trạng thái vượt khả năng Ethernet.

Trên máy tính, PRISM được xây dựng bằng Python, CustomTkinter và Matplotlib để tiếp nhận UDP, lưu mốc thời gian bằng số nguyên chu kỳ mẫu nhằm tránh sai số trôi dấu phẩy động, đồng thời hiển thị dạng sóng. Phần mềm tự động nhận diện và giải mã UART, I2C, SPI, CAN Classic 2.0A và CLK/PWM; bộ giải mã CAN thực hiện bit-destuffing và kiểm tra CRC-15/CAN. Thiết bị kiểm thử ESP32 sử dụng I2S0 ở chế độ LCD-parallel 16 bit với DMA để phát đồng thời các mẫu kiểm thử, loại bỏ jitter do phần mềm trong miền phát. Kết quả thực nghiệm và các kịch bản TC0–TC8 xác nhận tốc độ lấy mẫu 175,5 MS/s, khả năng tái tạo thời gian và hoạt động của chuỗi thu nhận, truyền tải, hiển thị, giải mã.

Từ khóa: Máy phân tích logic, FPGA, RLE, Clock Domain Crossing, bộ giải mã giao thức, HUST.

ABSTRACT

A logic analyzer is essential for observing digital signals, debugging communication protocols, and verifying embedded systems. However, low-cost commercial instruments are commonly restricted in either channel count or sampling rate. This report presents LA16, which concurrently acquires 16 digital channels for signals at or above the required 25 MHz and transfers the results to a host computer through 100 Mbps Ethernet, although the raw sample stream greatly exceeds the capacity of the RTL8201F RMII PHY.

LA16 is implemented on the Sipeed Tang Primer 20K board using the Gowin GW2A-LV18PG256C8/I7 FPGA. An rPLL multiplies the 27 MHz reference clock to 175.5 MHz, providing a temporal resolution of 5.70 ns. All 16 channels are latched on the same clock edge; therefore, their architectural skew is zero sampling cycles. A hardware pipeline applies transition-based Run-Length Encoding (RLE), generating a time-delta and 16-bit state record only when the input changes. A Gray-pointer asynchronous FIFO performs Clock Domain Crossing (CDC) from the 175.5 MHz acquisition domain to the 50 MHz Ethernet domain. The packetizer then sends the records through UDP port 5001. This approach reduces link traffic and reports overflow whenever transition density exceeds the available Ethernet throughput.

On the host computer, PRISM is developed in Python with CustomTkinter and Matplotlib. It receives UDP packets, stores time as integer sampling cycles to prevent floating-point drift, and renders digital waveforms. The software automatically identifies and decodes UART, I2C, SPI, Classic CAN 2.0A, and CLK/PWM; its CAN decoder performs bit destuffing and CRC-15/CAN verification. The test source uses the ESP32 I2S0 peripheral in 16-bit LCD-parallel DMA mode to launch synchronous patterns without software-induced jitter in the generation clock domain. Experimental measurements and scenarios TC0–TC8 confirm the 175.5 MS/s sampling rate, timestamp reconstruction, and operation of the acquisition, transport, visualization, and protocol-decoding chain.

Keywords: Logic Analyzer, FPGA, RLE, Clock Domain Crossing, Protocol Decoder, HUST.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Tổng quan chương. Hệ thống nhúng hiện đại gồm nhiều khối phần cứng và phần mềm hoạt động đồng thời, trao đổi dữ liệu qua các giao thức như UART, I2C, SPI hoặc CAN. Khi xuất hiện sai lệch thời điểm, mất bit hay lỗi khung, trạng thái cuối cùng của chương trình không đủ để xác định nguyên nhân. Kỹ sư cần thu nhận trực tiếp mức logic trên nhiều đường tín hiệu, bảo toàn quan hệ thời gian giữa các cạnh và tái dựng nội dung giao tiếp. Vì vậy, máy phân tích logic là công cụ thiết yếu trong kiểm thử và gỡ lỗi hệ thống nhúng.

Chương này trình bày bối cảnh, giới hạn băng thông và mục tiêu của LA16 Logic Analyzer; từ đó nêu các đóng góp về kiến trúc FPGA, nén dữ liệu, nhận dạng giao thức, tạo tín hiệu kiểm thử và giới thiệu bố cục báo cáo.

1.1 Đặt vấn đề

Máy phân tích logic (Logic Analyzer – LA) là thiết bị đo số lấy mẫu đồng thời nhiều kênh và biểu diễn trạng thái theo thời gian. Dữ liệu thu tại chân giao tiếp phản ánh trực tiếp hoạt động của phần cứng, cho phép xác định thứ tự cạnh, độ rộng xung, chu kỳ, tần số và độ trễ giữa các kênh. Đây là cơ sở để phát hiện các sự kiện ngắn và đánh giá quan hệ timing giữa các khối trong hệ thống nhúng.

Trong phát triển sản phẩm, LA được dùng để kiểm tra tín hiệu đồng hồ, tín hiệu điều khiển và các bus dữ liệu. Với UART, thiết bị hỗ trợ xác định tốc độ baud và lỗi khung; với I2C, SPI, việc quan sát đồng thời dữ liệu, xung nhịp và tín hiệu chọn mạch giúp phát hiện sai địa chỉ, sai chế độ lấy mẫu hoặc thiếu xác nhận. Đối với CAN, chuỗi bit còn cho phép kiểm tra định dạng khung, bit chèn và CRC. Do đó, LA vừa hiển thị dạng sóng vừa cung cấp dữ liệu để giải mã giao thức và khoanh vùng lỗi.

Ở phân khúc chuyên nghiệp, các máy hiện sóng hỗn hợp và máy phân tích logic chuyên dụng của Keysight hoặc Tektronix cung cấp băng thông, độ sâu bộ nhớ và hệ thống trigger mạnh, nhưng chi phí đầu tư cao nên khó tiếp cận đối với phần lớn sinh viên. Ở chiều ngược lại, các thiết bị USB LA giá thấp, điển hình là dòng thường được gọi là Saleae 8ch Clone, chia sẻ băng thông cổng USB và phải đánh đổi giữa số kênh, tốc độ lấy mẫu với độ sâu bộ nhớ. Tốc độ công bố vì vậy không phải lúc nào cũng được duy trì đồng thời trên toàn bộ kênh. Hạn chế này làm giảm khả năng phân tích tin cậy các tín hiệu từ vài MHz đến hàng chục MHz, đặc biệt khi cần bảo toàn quan hệ thời gian giữa nhiều đường tín hiệu.

Từ nhu cầu trên, LA16 được xây dựng trên FPGA của kit Sipeed Tang Primer 20K, hướng tới lấy mẫu đồng thời 16 kênh, truyền kết quả qua Ethernet và nhận dạng giao

thức trên máy tính. Bài toán trọng tâm là tổ chức đường dữ liệu để thông tin thu ở tốc độ cao có thể đi qua giao tiếp mạng 100 Mbit/s của kit [2, 40].

1.2 Cơ sở thực tế và các hạn chế của nghiên cứu hiện tại

Đối với LA lấy mẫu theo thời gian, lượng dữ liệu thô tăng tuyến tính theo số kênh và tần số lấy mẫu. Với 16 kênh được biểu diễn bởi 16 bit tại mỗi chu kỳ và tần số lấy mẫu 175,5 MS/s, tốc độ dữ liệu phát sinh được xác định như sau:

$$R_{\text{raw}} = 175{,}5 \times 10^6 \times 16 = 2{,}808 \times 10^9 \text{ bit/s} = 351 \text{ MB/s}. \tag{1.1}$$

Luồng dữ liệu thô đạt xấp xỉ 2,8 Gbit/s, trong khi Tang Primer 20K Dock sử dụng PHY Ethernet 10/100 Mbit/s kết nối qua RMII 50 MHz [2, 40]. Bỏ qua mọi chi phí giao thức, 100 Mbit/s chỉ tương đương 12,5 MB/s, thấp hơn khoảng 28 lần nhu cầu dữ liệu thô. Thông lượng tải hữu ích thực tế còn nhỏ hơn do tiêu đề gói và khoảng cách giữa các khung.

Chênh lệch này tạo thành nút thắt cổ chai: FPGA có thể lấy mẫu ở 175,5 MHz nhưng không thể truyền toàn bộ mẫu theo thời gian thực. Thay UDP bằng TCP không làm tăng băng thông vật lý. LA16 chọn UDP vì cấu trúc datagram đơn giản, phù hợp với phần cứng và cho phép phát hiện mất gói bằng số thứ tự [41]; lựa chọn này không loại bỏ giới hạn 100 Mbit/s.

Giải pháp truyền thống là lưu mẫu vào RAM trong một cửa sổ đo, sau đó dừng thu và truyền chậm về máy tính. Cách thu có đệm phù hợp với sự kiện trigger hữu hạn nhưng thời gian quan sát phụ thuộc dung lượng RAM và không hỗ trợ giám sát liên tục trong lúc đọc bộ đệm. Giảm tần số lấy mẫu để kéo dài cửa sổ lại làm suy giảm độ phân giải và có thể bỏ sót xung hẹp.

Phần lớn tín hiệu giao thức giữ nguyên trạng thái qua nhiều chu kỳ và chỉ thay đổi tại các cạnh có ý nghĩa. Vì vậy, LA16 áp dụng Run-Length Encoding (RLE) theo chuyển trạng thái: mỗi bản ghi chứa giá trị mới của 16 kênh và số chu kỳ kể từ lần thay đổi trước. Lưu lượng được chuyển từ "số mẫu trên giây" sang "số lần chuyển trạng thái trên giây", còn mốc thời gian vẫn được đo bằng clock 175,5 MHz. Với giao thức có thời gian rỗi hoặc truyền theo cụm, số bản ghi giảm đáng kể.

RLE không bảo đảm tỷ lệ nén cố định: nhiều kênh chuyển mức liên tục vẫn có thể làm tốc độ bản ghi vượt băng thông mạng. FIFO bất đồng bộ hấp thụ các burst ngắn và phát cờ overflow khi quá tải kéo dài. Bộ đếm thời gian tiếp tục tăng khi FIFO đầy nên bản ghi sau đó vẫn đúng mốc, dù có thể mất các cạnh ở giữa. Giải pháp phù hợp với giám sát giao thức nối tiếp nhưng không bảo đảm truyền lossless mọi tín hiệu 16 kênh ở trường hợp xấu nhất.

1.3 Mục tiêu nghiên cứu

Mục tiêu thứ nhất là lấy mẫu đồng thời 16 kênh tại 175,5 MHz, tương ứng $T_s = 1/(175{,}5 \times 10^6) \approx 5{,}70$ ns. Các kênh được chốt bởi cùng một clock và cấu trúc thanh ghi nên không có độ lệch chu kỳ ở cấp kiến trúc. Sai lệch vật lý của đường vào vẫn tồn tại nhưng nhỏ hơn đáng kể chu kỳ mẫu và được xem như thành phần sai số lượng tử thời gian.

Mục tiêu thứ hai là xử lý nút thắt 100 Mbit/s bằng RLE trực tiếp trên FPGA. Hệ thống chỉ tạo bản ghi khi trạng thái 16 bit thay đổi, kèm delta 32 bit để bảo toàn thời gian; sau đó dữ liệu đi qua FIFO chuyển miền clock, được đóng gói và truyền đến máy tính bằng UDP không chặn. Kiến trúc hướng tới quan sát thời gian thực UART, I2C, SPI, CAN và clock/PWM trong miền tải phù hợp, đồng thời báo rõ trạng thái quá tải thay vì che giấu mất dữ liệu bằng cơ chế tồn đọng hoặc truyền lại.

Mục tiêu thứ ba là xây dựng phần mềm máy chủ PRISM để tiếp nhận, lưu trữ và biểu diễn dữ liệu. Bộ phân tích chạy nền khoảng 1 Hz nhằm phân loại tín hiệu, ghép các đường bus và lựa chọn bộ giải mã. Giao diện hỗ trợ tạm dừng, phóng đại, đo bằng con trỏ; giải mã UART, I2C, SPI mode 0, CAN Classic 2.0A và đo tần số, hệ số lấp đầy của clock/PWM. Qua đó hình thành chuỗi xử lý thống nhất từ chân đo đến giao diện người dùng.

1.4 Các đóng góp chính của đề tài

1. Đề tài hiện thực kiến trúc xử lý song song trên FPGA GW2A-18C, trong đó 16 kênh được chốt bằng cùng một cạnh clock, tạo độ lệch chu kỳ bằng không ở cấp logic. Quan hệ thời gian được duy trì qua RLE, FIFO bất đồng bộ và bộ đóng gói UDP; đường dữ liệu nhanh được pipeline để đáp ứng timing 175,5 MHz.

2. Thuật toán RLE theo chuyển trạng thái được triển khai thành pipeline phần cứng một tầng. Khi bus duy trì cùng trạng thái trong nhiều chu kỳ, hệ thống chỉ truyền một bản ghi gồm delta và giá trị 16 bit, nhờ đó tỷ lệ dữ liệu có thể giảm hàng trăm lần đối với các luồng có thời gian rỗi dài hoặc hoạt động theo cụm. Bộ đếm delta 32 bit được tách thành `dlo` 12 bit và `dhi` 20 bit để tránh carry-chain dài trên đường tới hạn nhưng vẫn bảo toàn số chu kỳ lấy mẫu.

3. Bộ phân tích chạy nền khoảng 1 Hz nhận dạng UART, I2C, SPI, CAN, clock và PWM. Decoder CAN hỗ trợ khung Classic 2.0A định danh 11 bit, tự động loại bit chèn và tính CRC-15/CAN [42]; điều kiện CRC hợp lệ giúp hạn chế nhầm CAN với UART hoặc nhiễu.

4. Bộ tạo tín hiệu 16 kênh trên ESP32 sử dụng I2S0 LCD-parallel và DMA. Dạng sóng được tính trước trong bộ đệm 16 bit rồi phát theo clock phần cứng, loại bỏ

jitter do bit-banging. Các tổ hợp UART, I2C, SPI, CAN, clock và PWM cũng được dùng cho kiểm thử software-in-the-loop.

1.5 Bố cục báo cáo

Không kể các phần mở đầu và phụ lục, báo cáo được tổ chức thành năm chương. Chương 1 giới thiệu bối cảnh, vấn đề băng thông, mục tiêu và các đóng góp của đề tài LA16. Chương 2 trình bày cơ sở lý thuyết liên quan đến lấy mẫu tín hiệu số, kiến trúc FPGA, đồng bộ miền clock, nén RLE, truyền thông Ethernet/UDP và nguyên lý của các giao thức cần giải mã. Chương 3 mô tả quá trình triển khai dự án, bao gồm thiết kế phần cứng FPGA, cấu trúc gói dữ liệu, phần mềm máy chủ và bộ tạo tín hiệu kiểm thử. Chương 4 trình bày kết quả thực nghiệm, kiểm tra chức năng, đánh giá timing, khả năng nhận dạng giao thức và các giới hạn băng thông của hệ thống. Cuối cùng, Chương 5 tổng kết kết quả đạt được, phân tích những hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo.

Kết chương. Chương 1 đã xác lập nhu cầu xây dựng máy phân tích logic đa kênh có tốc độ lấy mẫu cao và chỉ ra rằng trở ngại chính của hệ thống không nằm riêng ở quá trình chốt tín hiệu, mà nằm ở sự chênh lệch lớn giữa tốc độ sinh dữ liệu 2,808 Gbit/s và liên kết Ethernet 100 Mbit/s. Từ đó, đề tài lựa chọn kiến trúc lấy mẫu 16 kênh tại 175,5 MHz kết hợp RLE theo chuyển trạng thái, truyền UDP và phân tích giao thức trên máy tính. Các mục tiêu và đóng góp đã nêu là cơ sở để đánh giá tính đúng đắn của thiết kế trong các chương tiếp theo. Chương 2 sẽ trình bày những nền tảng lý thuyết cần thiết cho việc xây dựng và phân tích kiến trúc LA16.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP TÍNH TOÁN

Tổng quan chương

Chương 1 đã xác định yêu cầu xây dựng Logic Analyzer 16 kênh có khả năng lấy mẫu tốc độ cao, truyền dữ liệu qua Ethernet và tự động giải mã các giao thức số. Để chuyển các yêu cầu đó thành kiến trúc có thể triển khai, chương này phân tích mười nhóm cơ sở: (i) lấy mẫu bất đồng bộ, lượng tử hóa thời gian và đồng bộ tín hiệu vào; (ii) nén theo transition và tái dựng timeline; (iii) quy tắc timing của UART, SPI, I2C, CAN và CLK/PWM; (iv) lựa chọn TCP hoặc UDP cho luồng dữ liệu đo liên tục; (v) đặc tả hai nền tảng Tang Primer 20K và ESP32-WROOM-32; (vi) mô hình kiểm thử Hardware-in-the-Loop sử dụng I2S0 LCD-Parallel kết hợp DMA; (vii) mạch phân áp bù tần số và bộ so sánh tốc độ cao; (viii) nguồn điện áp tham chiếu có bộ đệm cách ly; (ix) mạch dịch mức xung PWM; và (x) cây nguồn 5 V/3,3 V, bảo vệ USB-C, eFuse, LDO nhiễu thấp cùng mạch kẹp dòng dội ngược. Trong LA16, FPGA lấy mẫu đồng thời 16 kênh tại 175,5 MS/s, tạo record RLE 6 byte, truyền qua Ethernet RMII 100 Mbps và tái dựng waveform trên Host PRISM. Các kết quả của chương là cơ sở trực tiếp cho triển khai ở Chương 3 và kiểm thử hệ thống ở Chương 4.

2.1 Cơ sở lý thuyết lấy mẫu và đồng bộ tín hiệu

Trong chế độ lấy mẫu bất đồng bộ (*Timing Mode*), logic analyzer sử dụng clock nội tại f_s không có quan hệ pha xác định với thiết bị cần đo. Tại mỗi sườn tích cực của `clk_sample`, toàn bộ vector $\boldsymbol{x}(t) = \{x_{15}(t), \ldots, x_0(t)\}$ được chốt đồng thời. Chuỗi mẫu và chu kỳ lấy mẫu được xác định bởi

$$\boldsymbol{x}[n] = \boldsymbol{x}(nT_s), \qquad T_s = \frac{1}{f_s} = \frac{1}{175{,}5 \times 10^6} \approx 5{,}70\,\text{ns}. \tag{2.1}$$

Một clock chung làm cho 16 kênh có cùng mốc quan sát danh định, nhờ đó quan hệ timing giữa các kênh không chịu ảnh hưởng của độ trễ mạng hoặc lịch xử lý trên máy tính.

Theo định lý Nyquist, tín hiệu giới hạn băng phải thỏa mãn

$$f_s \geq 2 f_{\max} \tag{2.2}$$

để tránh chồng phổ [4]. Tuy nhiên, tín hiệu số chứa các thành phần hài do sườn lên và sườn xuống nên hai mẫu trên một chu kỳ chỉ là giới hạn tối thiểu, không đủ để đo ổn định độ rộng xung. Trong đo lường số, quy tắc kỹ thuật thường dùng là $f_s \geq (4\text{–}5) f_{\max}$. Với yêu cầu 25 MHz của đề tài [1], LA16 cung cấp $175{,}5/25 = 7{,}02$ mẫu trên một chu kỳ, tạo dự trữ cho định vị cạnh và giải mã giao thức.

Transition đầu vào có thể xuất hiện tại bất kỳ vị trí nào giữa hai sườn lấy mẫu. Nếu cạnh được ghi tại sườn clock đầu tiên sau khi mức logic đã ổn định thì

$$\widehat{t_e} = t_e + \varepsilon_t, \qquad 0 \le \varepsilon_t < T_s, \qquad -T_s < \varepsilon_{t,2} - \varepsilon_{t,1} < T_s. \tag{2.3}$$

Vì vậy, sai số thứ tự tương đối giữa hai kênh trong trường hợp xấu nhất xấp xỉ ± 1 chu kỳ, tương ứng $\pm 5{,}70$ ns. Xung hẹp hơn T_s có thể nằm hoàn toàn giữa hai mẫu và bị bỏ sót; xung kéo dài một đến hai mẫu vẫn phụ thuộc mạnh vào pha tức thời. Đây là giới hạn của phép lượng tử hóa thời gian, không phải lỗi hiển thị.

Tín hiệu probe bất đồng bộ cũng có thể thay đổi trong cửa sổ setup/hold của D Flip-Flop. Khi đó, Flip-Flop có thể rơi vào trạng thái metastable và chưa hội tụ ngay về logic 0 hoặc 1. LA16 sử dụng hai tầng Flip-Flop cho từng kênh: FF1 nhận trực tiếp probe, còn FF2 chốt lại kết quả sau một chu kỳ để tăng thời gian phân giải. MTBF (*Mean Time Between Failures*) của bộ đồng bộ có quan hệ gần đúng

$$\mathrm{MTBF} \propto \frac{\exp(T_{res}/\tau)}{f_{clk} f_{data}}, \tag{2.4}$$

trong đó T_{res} là thời gian phân giải, τ là hằng số công nghệ, f_{clk} là tần số lấy mẫu và f_{data} là tốc độ thay đổi đầu vào [5]. Tầng FF2 làm MTBF tăng theo hàm mũ. Cấu trúc này không loại bỏ metastability tuyệt đối nhưng làm xác suất lỗi lan tới logic xử lý giảm mạnh; độ trễ hai chu kỳ là độ trễ chung, còn các cạnh gần như đồng thời vẫn có thể lệch một mẫu.

2.2 Lý thuyết nén dữ liệu RLE và tái dựng thời gian

Nếu truyền trạng thái 16 kênh ở mọi chu kỳ, tốc độ dữ liệu thô là

$$B_{raw} = 16 \times 175{,}5 \times 10^6 = 2{,}808\,\mathrm{Gbit/s} = 351\,\mathrm{MB/s}. \tag{2.5}$$

Giá trị này lớn hơn khoảng 28 lần tốc độ Ethernet 100 Mbps và chưa tính overhead. RLE (*Run-Length Encoding*) tận dụng các khoảng tín hiệu giữ nguyên mức bằng cách chỉ tạo record khi vector 16 kênh thay đổi. Mỗi record dài 6 byte:

$$\mathscr{R}_k = \{\Delta_k, \boldsymbol{v}_k\} = \{\mathtt{delta}_{u32}, \mathtt{value}_{u16}\}, \tag{2.6}$$

trong đó Δ_k là số chu kỳ kể từ record đã ghi thành công trước đó và $\boldsymbol{v}_k$ là trạng thái mới của 16 kênh. Nếu trạng thái giữ nguyên trong N chu kỳ, lưu thô cần $16N$ bit còn RLE cần xấp xỉ 48 bit, nên hệ số nén lý tưởng là $N/3$. Các khoảng idle của UART, I2C hoặc SPI do đó có thể giảm dữ liệu hàng trăm lần mà vẫn giữ độ phân giải cạnh 5,70 ns.

Host tích lũy tổng số chu kỳ bằng số nguyên thay vì cộng các khoảng thời gian dấu phẩy động:

$$C_k = C_{k-1} + \Delta_k, \qquad \Delta t_{k,\mathrm{ns}} = \frac{\Delta_k \times 10^9}{f_s}, \qquad t_{k,\mathrm{ns}} = \left\lfloor \frac{C_k \times 10^9}{f_s} \right\rfloor. \tag{2.7}$$

Phép cộng C_k là phép toán nguyên chính xác; việc đổi sang nano-giây chỉ thực hiện khi hiển thị, xuất CSV hoặc đo khoảng cách. Cách này loại bỏ sai số trôi do cộng lặp số thực, trong khi sai số cuối cùng chỉ còn do lượng tử hóa về đơn vị hiển thị.

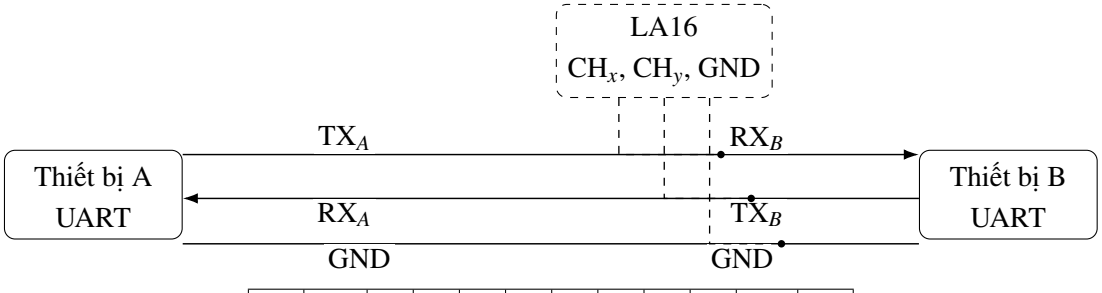
Khi FIFO đầy, tín hiệu ghi record bị chặn nhưng bộ đếm Δ vẫn tăng độc lập theo `clk_sample`. Record hợp lệ tiếp theo vì vậy chứa đúng tổng số chu kỳ kể từ record đã ghi gần nhất. Overflow làm mất các cạnh trung gian nhưng không làm co hoặc dịch timeline của dữ liệu còn lại; các cờ `ovf_recent` và `ovf_ever` cho phép Host công bố trạng thái này [9, 10]. Để đáp ứng timing, bộ đếm 32 bit được tách thành phần thấp 12 bit và phần cao 20 bit, với $\Delta = 2^{12} d_{hi} + d_{lo}$. Khi bus không đổi trong 2^{31} chu kỳ, record keepalive được tạo sau khoảng 12,24 s. Luồng record 48 bit được chuyển từ miền 175,5 MHz sang miền 50 MHz bằng FIFO bất đồng bộ; con trỏ Gray $g = b \oplus (b \gg 1)$ đi qua 2FF trước khi xác định full/empty, tránh chốt tổ hợp trung gian của con trỏ nhị phân.

2.3 Cơ sở lý thuyết các giao thức truyền thông số cần giải mã

PRISM đo khoảng cạnh, độ rộng xung, chu kỳ, duty cycle và quan hệ giữa các kênh trên chuỗi transition. Các đặc trưng này được đối chiếu với cấu trúc khung và timing của từng giao thức để hạn chế nhận dạng nhầm.

2.3.1 Giao tiếp UART

UART là giao tiếp bất đồng bộ điểm–điểm. Hai thiết bị song công dùng TX, RX mắc chéo và GND chung; LA16 chỉ nối song song với đường cần đo. Ở mức TTL/CMOS, khung 8N1 bắt đầu từ trạng thái rảnh HIGH, tiếp theo là Start-bit LOW, tám bit LSB-first và Stop-bit HIGH [4, 45], như Biểu đồ 2.1.



Idle	Start	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	Stop	Idle
1	0	8 bit dữ liệu, LSB-first								1	1

Biểu đồ 2.1 Sơ đồ kết nối và cấu trúc khung UART 8N1

Sau khi loại các glitch có độ rộng nhỏ hơn ngưỡng T_g, decoder tạo tập khoảng cách cạnh hợp lệ $\mathscr{I}$ và chọn khoảng ngắn nhất làm ứng viên bit-time ban đầu:

$$\mathscr{I} = \{\Delta t_j = t_{j+1} - t_j \mid \Delta t_j \geq T_g\}, \qquad T_{bit,ns} = \min_{\Delta t_j \in \mathscr{I}} \Delta t_j. \tag{2.8}$$

Khoảng ứng viên này tiếp tục được kiểm tra bằng điều kiện các khoảng cạnh còn lại gần bội số nguyên của $T_{bit,ns}$. Baudrate ước lượng $\widehat{R}_{baud}$ được snap về baudrate tiêu chuẩn

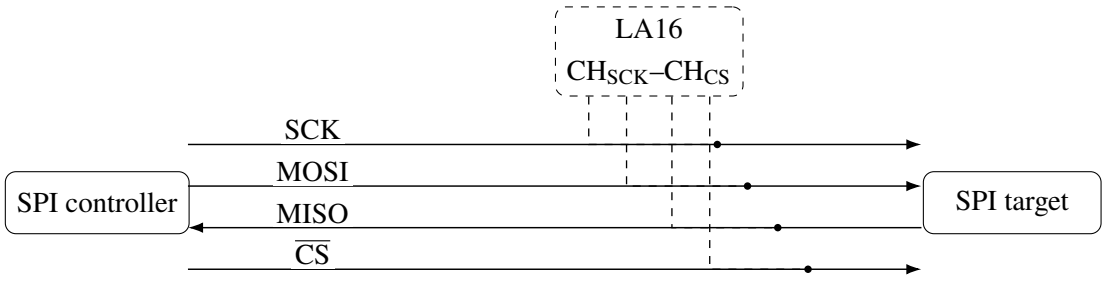
gần nhất R_s khi sai số tương đối không vượt quá 6%:

$$\widehat{R}_{baud} = \frac{10^9}{T_{bit,ns}}, \qquad e_s = \frac{|\widehat{R}_{baud} - R_s|}{R_s}, \qquad R_{baud} = \begin{cases} R_s, & e_s \le 0{,}06, \\ \widehat{R}_{baud}, & e_s > 0{,}06. \end{cases} \tag{2.9}$$

Từ Start-bit, dữ liệu được lấy mẫu tại $t_{start} + (1{,}5 + i)T_{bit}$, $i = 0, \ldots, 7$, và Stop-bit tại $9{,}5T_{bit}$. PRISM hiện hỗ trợ UART TTL/CMOS 8N1 không đảo mức; RS-232 cần tầng chuyển mức riêng.

2.3.2 Giao tiếp SPI

SPI là giao tiếp đồng bộ, song công: controller tạo SCK và CS, MOSI truyền đến target, còn MISO truyền theo chiều ngược lại. SPI không quy định Data Frame cố định; LA16 quy ước một byte gồm tám chu kỳ SCK khi CS active-low [4, 46], như Biểu đồ 2.2.



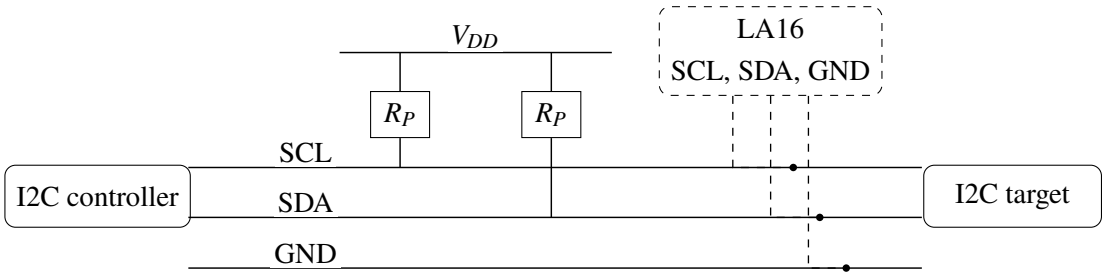
CS↓	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0	CS↑
Bắt đầu	Chốt MOSI/MISO tại tám sườn lên SCK								Kết thúc

Biểu đồ 2.2 Sơ đồ kết nối và từ dữ liệu SPI Mode 0

Trong Mode 0, CPOL=0 và CPHA=0 nên SCK nghỉ LOW; PRISM lấy MOSI tại sườn lên khi CS=0 và ghép byte MSB-first. Sai cấu hình CPOL/CPHA làm thay đổi cạnh chốt và có thể tạo byte sai.

2.3.3 Giao tiếp I2C

I2C dùng hai đường open-drain SCL, SDA với điện trở kéo lên, cho phép nhiều thiết bị dùng chung bus. LA16 chỉ mắc song song với SCL, SDA và GND, không được chủ động lái bus, như Biểu đồ 2.3.



START	Address [6:0]	R/W	ACK	Data [7:0]	ACK/NAK	STOP hoặc Sr
S	Byte địa chỉ, MSB-first		Bit 9	Byte dữ liệu	Bit 9	P hoặc repeated START

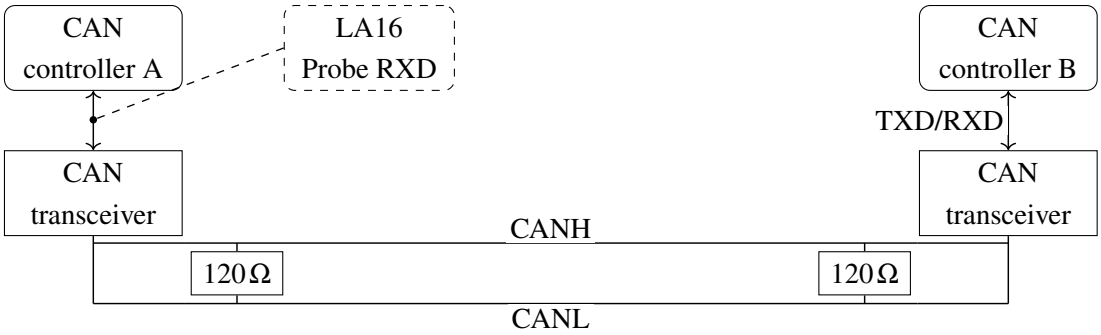
Biểu đồ 2.3 Sơ đồ bus và cấu trúc một transaction I2C 7 bit

Theo UM10204, START là SDA đổi HIGH–LOW khi SCL HIGH, còn STOP là chuyển đổi ngược lại. Ngoài START, STOP và repeated START, SDA phải ổn định khi

SCL HIGH và được chốt tại sườn lên [43]. Byte địa chỉ gồm 7 bit và R/W; bit thứ chín của mỗi byte là ACK=LOW hoặc NAK=HIGH. PRISM dùng các quy tắc này để tách địa chỉ, hướng truyền và dữ liệu.

2.3.4 Giao tiếp CAN Classic 2.0A

CAN dùng cặp vi sai CANH/CANL và transceiver, với điện trở 120 Ω ở hai đầu đường trục [47]. Probe 3,3 V của LA16 phải lấy tại RXD phía logic hoặc qua front-end phù hợp, không nối trực tiếp CANH/CANL, như Biểu đồ 2.4.



SOF	ID	RTR	IDE	r0	DLC	Data	CRC	ACK	EOF	IFS
1	11	1	1	1	4	0–64	15+1	2	7	3 bit

Biểu đồ 2.4 Tầng vật lý và cấu trúc Data Frame CAN Classic 2.0A

Trên RXD, dominant là 0 và recessive là 1. Data Frame chuẩn gồm SOF, ID 11 bit, RTR, IDE=0, r0, DLC, 0–8 byte dữ liệu, CRC, ACK, EOF và IFS [42]. Từ SOF đến hết CRC sequence, một bit ngược cực tính được chèn sau năm bit đồng mức và phải được loại trước khi phân trường. CRC15 dùng đa thức

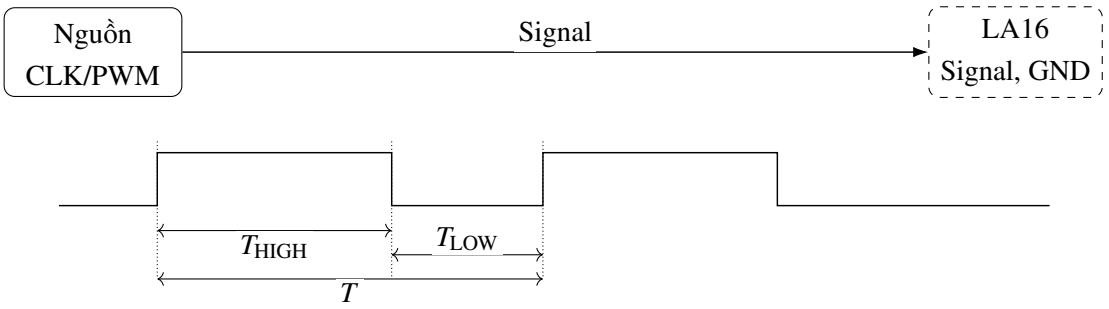
$$G(x) = x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1, \tag{2.10}$$

tương ứng `0x4599`. PRISM chỉ xác nhận frame khi IDE=0, stuffing hợp lệ và CRC khớp. ESP32 kiểm thử tạo trực tiếp chuỗi logic tương đương RXD, vì vậy đánh giá decoder nhưng không thay thế kiểm thử tầng vật lý CAN.

2.3.5 Tín hiệu CLK và PWM

CLK và PWM không có Data Frame mà được đặc trưng bởi chu kỳ và thời gian HIGH. Với $T = T_{HIGH} + T_{LOW}$:

$$f = \frac{1}{T}, \qquad D = \frac{T_{HIGH}}{T_{HIGH} + T_{LOW}} \times 100\%. \tag{2.11}$$



Biểu đồ 2.5 Kết nối đo và các tham số thời gian của CLK/PWM

Gọi $P = 1$ khi chuỗi cạnh thỏa mãn điều kiện tuần hoàn ổn định của detector và $P = 0$ trong trường hợp còn lại. Luật quyết định cuối cùng được toán học hóa bởi

$$\mathscr{C}(P,D) = \begin{cases} \text{CLK}, & P = 1 \ \wedge \ 44\% \leq D \leq 56\%, \\ \text{PWM}, & P = 1 \ \wedge \ (D < 44\% \ \vee \ D > 56\%), \\ \text{DATA}, & P = 0. \end{cases} \tag{2.12}$$

Dải dung sai quanh 50% hấp thụ sai số lượng tử hóa cạnh, trong khi điều kiện P ngăn một chuỗi cạnh không tuần hoàn bị phân loại chỉ dựa trên duty cycle tức thời.

2.4 Giao thức truyền dữ liệu Ethernet thời gian thực: UDP và TCP

Ethernet RMII của LA16 có tốc độ vật lý 100 Mbps. Với record 48 bit và tốc độ transition λ, tải dữ liệu tối thiểu là $B_{RLE} = 48\lambda$; giới hạn lý tưởng tương ứng 2,08 triệu record/s, còn ngân sách liên tục thực dụng xấp xỉ 1,5 triệu record/s sau overhead [9]. Do nguồn đo không thể dừng để nhận backpressure, giao thức phải ưu tiên dữ liệu mới và công bố rõ mọi mất mát.

TCP cung cấp luồng byte tin cậy, đúng thứ tự nhờ ACK, retransmission, flow control và congestion control [6]. Tuy nhiên, segment mất gây nghẽn đầu dòng: dữ liệu sau nó phải chờ truyền lại trong khi sampler vẫn sinh record, làm backlog, độ trễ và mức chiếm FIFO tăng. TCP vì vậy không giải quyết quá tải bền vững của nguồn không thể tạm dừng.

UDP truyền datagram độc lập, không bắt tay hoặc tự truyền lại [41]; phần cứng FPGA nhờ đó ít trạng thái và không chờ phục hồi dữ liệu cũ. LA16 bổ sung header 16 byte chứa sequence và hai cờ overflow, cho phép Host phân biệt mất datagram trên mạng với mất transition do FIFO đầy [10].

Bảng 2.1 Cấu trúc header ứng dụng UDP của LA16

Offset	Dài	Trường	Chức năng
0	4	Magic	Nhận dạng `LA16`.
4	1	Version	Phiên bản giao thức.
5	1	Flags	Overflow gần đây/từ reset.
6	2	Count	Số record; heartbeat bằng 0.
8	4	Sequence	Phát hiện gói thiếu.
12	4	Sample rate	Đổi delta sang thời gian.

Bảng 2.2 So sánh TCP và UDP trong bài toán streaming LA16

Tiêu chí	TCP	UDP trong LA16
Tin cậy	ACK, truyền lại, đúng thứ tự.	Sequence công bố gói mất.
Quá tải	Backpressure và backlog tăng.	Không chặn luồng; flags báo tràn.
Độ trễ	Tăng khi chờ segment cũ.	Không chờ truyền lại.
Phần cứng	Cần kết nối, timer và bộ đệm.	Packetizer ít trạng thái.

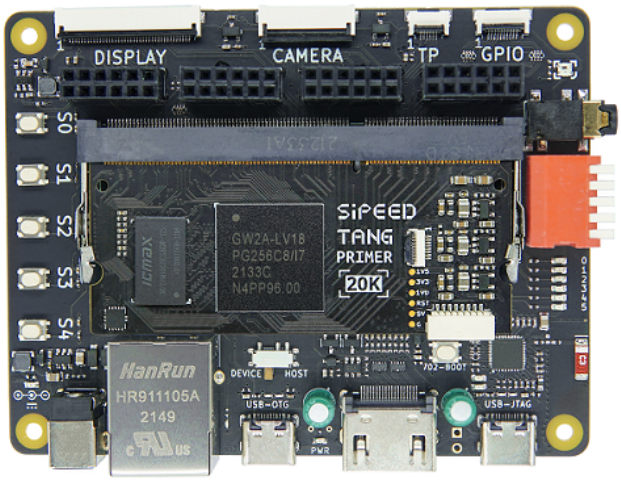
UDP được lựa chọn vì phù hợp nguồn đo liên tục, không phải vì tin cậy hơn TCP. Timestamp được tạo trước tầng mạng nên jitter chỉ ảnh hưởng thời gian hiển thị và mức độ đầy đủ, không làm thay đổi thời điểm transition đã ghi.

2.5 Khảo sát đặc tả phần cứng nền tảng

Hệ thống LA16 sử dụng hai nền tảng phần cứng có chức năng bổ sung cho nhau. Tang Primer 20K thực hiện thu nhận, nén và truyền dữ liệu; bo mạch NodeMCU ESP32 tạo các chuỗi tín hiệu số tham chiếu phục vụ kiểm thử. Việc khảo sát tài nguyên, miền clock và chuẩn điện của hai nền tảng là cơ sở để xác định tính khả thi của kiến trúc và điều kiện đấu nối vật lý.

2.5.1 Nền tảng thu nhận Sipeed Tang Primer 20K

Tang Primer 20K sử dụng FPGA Gowin GW2A-LV18PG256C8/I7 gắn trên Dock có bốn đầu nối PMOD và PHY Ethernet RTL8201F. Biểu đồ 2.6 thể hiện nền tảng phần cứng được sử dụng làm thiết bị đo của LA16.



Biểu đồ 2.6 Nền tảng Sipeed Tang Primer 20K Dock sử dụng trong LA16

Bảng 2.3 Đặc tả nền tảng thu nhận Tang Primer 20K

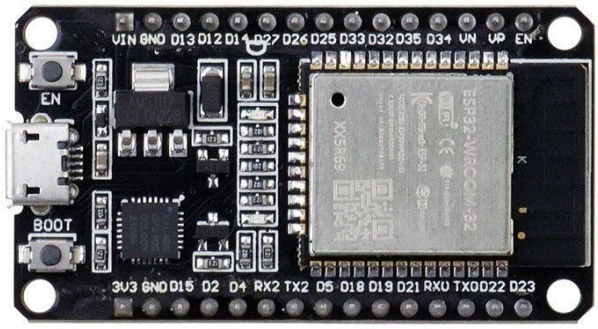
Hạng mục	**Thông số và vai trò trong LA16**
FPGA	Gowin GW2A-LV18PG256C8/I7.
Tài nguyên logic	20.736 LUT; báo cáo triển khai của đúng part đích cung cấp 16.173 tài nguyên thanh ghi.
Bộ nhớ khối	46 BSRAM, dung lượng danh định 18 Kbit mỗi khối; LA16 dùng BSRAM để hiện thực FIFO bất đồng bộ.
Tài nguyên tổng hợp clock	4 PLL; thiết kế sử dụng một rPLL để tổng hợp clock lấy mẫu.
Clock tham chiếu onboard	Thạch anh 27 MHz.
Clock lấy mẫu	175,5 MHz, tương ứng chu kỳ $T_s \approx 5{,}70$ ns.
Đầu vào probe	16 kênh trên PMOD2 và PMOD3; chuẩn LVCMOS 3,3 V, `PULL_MODE=UP`.
Ethernet PHY	Realtek RTL8201F, hỗ trợ Ethernet 10/100 Mbps.
Giao tiếp FPGA–PHY	RMII 2 bit với clock tham chiếu 50 MHz.
Cấu hình mạng	FPGA: `192.168.15.14`; Host: `192.168.15.15`; UDP đích cổng 5001.

Các thông số trong Bảng 2.3 được đối chiếu từ tài liệu bo mạch, datasheet thiết bị và báo cáo triển khai của dự án [2, 44, 7, 9]. LUT và thanh ghi hiện thực sampler, bộ phát hiện transition, bộ đếm delta và máy trạng thái packetizer; BSRAM cung cấp dung lượng đệm burst mà không tạo áp lực lên LUT. Bốn PLL cho phép thiết kế dành riêng một rPLL cho miền lấy mẫu 175,5 MHz, trong khi RMII tiếp tục hoạt động ở miền 50 MHz độc lập.

Ba mốc clock của nền tảng có chức năng khác nhau: (i) 27 MHz là nguồn tham chiếu vật lý; (ii) 175,5 MHz là time base của dữ liệu đo; và (iii) 50 MHz điều khiển packetizer, UDP stack và PHY RMII. Ranh giới 175,5/50 MHz được xử lý bằng FIFO bất đồng bộ, do đó độ trễ của mạng không tham gia vào quá trình xác lập timestamp.

2.5.2 Nền tảng phát tín hiệu NodeMCU ESP32-WROOM-32

NodeMCU ESP32-WROOM-32 được sử dụng làm bộ phát kịch bản thử nghiệm. Thay vì điều khiển từng GPIO bằng lệnh CPU, firmware sử dụng I2S0 ở chế độ LCD-Parallel 16 bit và DMA để cập nhật đồng thời một vector logic 16 kênh. Phần cứng thực tế được thể hiện trong Biểu đồ 2.7.



Biểu đồ 2.7 NodeMCU ESP32-WROOM-32 sử dụng làm bộ phát tín hiệu kiểm thử

Bảng 2.4 Đặc tả nền tảng phát tín hiệu ESP32

Hạng mục	**Thông số và cấu hình sử dụng**
Module phát	NodeMCU sử dụng module ESP32-WROOM-32, bố trí 30 chân.
Bộ xử lý	Hai lõi Xtensa LX6 32 bit, tần số hoạt động tối đa 240 MHz.
Bộ nhớ trên chip	520 KB SRAM; module tích hợp 4 MB SPI Flash.
Chuẩn điện GPIO	Miền logic 3,3 V; các GPIO trong bảng nối dây được cấu hình push-pull output.
Ngoại vi sử dụng	I2S0 ở chế độ LCD-Parallel, đường dữ liệu song song rộng 16 bit.
Tín hiệu nội bộ	`I2S0O_DATA_OUT8` đến `I2S0O_DATA_OUT23`.
Biểu diễn dữ liệu	Mỗi phần tử `uint16_t` là một vector mức logic của CH0–CH15.
Bộ vận chuyển	DMA sử dụng các descriptor `lldesc_t` nối vòng; mỗi descriptor không vượt quá 4092 byte.
Nguồn clock I2S	PLL_D2 160 MHz; bộ chia clock xác định tốc độ phát vector.
Chế độ hoạt động	Buffer được phát lặp vòng cho đến khi CPU chọn test case khác hoặc dừng I2S0.

ESP32 chứa hai ngoại vi I2S và hệ thống GPIO matrix cho phép ánh xạ tín hiệu ngoại vi tới các chân khả dụng [48, 49]. Trong firmware LA16, bit i của phần tử buffer được nối tới `I2S0O_DATA_OUT(8+i)` và tiếp tục ánh xạ tới `CH_PINS[i]` [50]. Vector phát tại mẫu thứ n được biểu diễn bởi

$$u[n] = \sum_{i=0}^{15} b_i[n]2^i, \qquad b_i[n] \in \{0,1\}, \tag{2.13}$$

trong đó $b_i[n]$ là mức logic mong muốn của kênh CHi. Một lần đọc DMA vì vậy chuyển toàn bộ $u[n]$ tới ngoại vi I2S0, thay vì thực hiện 16 phép ghi GPIO tại 16 thời điểm phần mềm khác nhau.

2.5.3 Khả năng tương thích điện và phân chia chức năng

Bảng 2.5 Phân chia chức năng và giao diện giữa hai nền tảng

Thành phần	**ESP32-WROOM-32**	**Tang Primer 20K**
Vai trò	Sinh waveform kiểm thử từ buffer tham chiếu.	Thu nhận, timestamp, nén RLE và truyền UDP.
Hướng dữ liệu GPIO	Đầu ra push-pull 3,3 V.	Đầu vào LVCMOS33 có pull-up nội.
Miền thời gian	Clock I2S0 độc lập, xác định thời điểm phát vector.	rPLL 175,5 MHz độc lập, xác định lưới lấy mẫu.
Khả năng backpressure	DMA phát liên tục theo buffer và không nhận phản hồi từ FPGA.	Chỉ có thể đệm bằng FIFO và công bố overflow.
Kết nối bắt buộc	CH0–CH15 và GND.	PMOD2/PMOD3 và GND tương ứng.

Hai phía cùng sử dụng miền logic danh định 3,3 V nên có thể nối trực tiếp trong cấu hình HIL cơ sở chưa lắp bo front-end. Pull-up nội của FPGA giữ mức HIGH xác định khi dây probe thả nổi, nhưng không thay thế nguồn tham chiếu GND và không được xem là mạch chuyển mức. Tín hiệu 5 V không được đưa trực tiếp vào chân FPGA; ở phiên bản phần cứng hoàn chỉnh, tín hiệu ngoài 0,9–5 V phải đi qua mạch bảo vệ, phân áp và comparator được phân tích tại Mục 2.7.

2.6 Thiết lập mô hình kiểm thử Hardware-in-the-Loop

Mô hình HIL của LA16 sử dụng ESP32 thật để tạo mức điện áp trên 16 dây, FPGA thật để lấy mẫu và Ethernet thật để vận chuyển dữ liệu tới PRISM. Do tín hiệu phải đi qua GPIO matrix, chân vật lý, dây nối PMOD, input buffer, bộ đồng bộ và toàn bộ

datapath FPGA, mô hình này kiểm tra được các sai số và giới hạn không xuất hiện trong mô phỏng phần mềm thuần túy.

2.6.1 Kiến trúc và luồng dữ liệu của vòng kiểm thử

Bảng 2.6 Luồng xử lý trong mô hình kiểm thử HIL

Bước	Khối thực hiện	Dữ liệu/miền điều khiển	Chức năng
1	Hàm dựng test case trên ESP32	Tham số giao thức	Tạo timeline logic của UART, I2C, SPI, CAN hoặc CLK/PWM.
2	Buffer RAM DMA	Mảng `uint16_t`	Lưu trạng thái đồng thời của 16 kênh theo từng mẫu phát.
3	DMA và chuỗi descriptor	AHB/DMA	Cấp dữ liệu tuần tự cho FIFO I2S0 mà không cần CPU ghi từng mẫu.
4	I2S0 LCD-Parallel	Clock phát phần cứng	Đưa đồng thời 16 bit ra GPIO4, GPIO15, . . . , GPIO33.
5	Tang Primer 20K	`clk_sample` 175,5 MHz	Đồng bộ 2FF, phát hiện transition, tạo delta và ghi FIFO.
6	UDP và Host PRISM	RMII 50 MHz/Ethernet	Truyền, tái dựng waveform và so sánh kết quả giải mã với test case.

Chuỗi xử lý trong Bảng 2.6 tạo một vòng kín về mặt kiểm chứng: dữ liệu kỳ vọng được xác định trước trong buffer ESP32, còn dữ liệu quan sát được tái dựng độc lập từ các transition do FPGA thu nhận. CPU ESP32 chỉ tham gia khi dựng buffer, tạo descriptor và chuyển test case; trong giai đoạn phát ổn định, clock I2S0 và DMA duy trì luồng tín hiệu.

2.6.2 Ánh xạ kênh và đấu nối vật lý

Firmware khai báo `CH_PINS[16]` theo đúng thứ tự bit của buffer. Trong hai bảng ánh xạ, chỉ số k biểu thị hậu tố của tín hiệu phần cứng `I2S00_DATA_OUT`k. Bảng 2.7 và Bảng 2.8 đối chiếu đồng thời bit dữ liệu, tín hiệu I2S0, GPIO ESP32, pin package FPGA và vị trí PMOD [51, 50].

Bảng 2.7 Ánh xạ CH0–CH7 từ ESP32 tới PMOD2

CH	Bit $u[n]$	I2S k	GPIO ESP32	Pin FPGA	Vị trí PMOD2
0	0	8	GPIO4 (D4)	A14	Hàng 3, phải.
1	1	9	GPIO15 (D15)	D10	Hàng 4, phải.
2	2	10	GPIO18 (D18)	B13	Hàng 3, trái.
3	3	11	GPIO19 (D19)	B12	Hàng 4, trái.
4	4	12	GPIO21 (D21)	B14	Hàng 2, trái.
5	5	13	GPIO22 (D22)	A15	Hàng 2, phải.
6	6	14	GPIO23 (D23)	D14	Hàng 1, trái.
7	7	15	GPIO25 (D25)	E15	Hàng 1, phải.

Bảng 2.8 Ánh xạ CH8–CH15 từ ESP32 tới PMOD3 và kết nối GND

CH	Bit $u[n]$	I2S k	GPIO ESP32	Pin FPGA	Vị trí PMOD3
8	8	16	GPIO14 (D14)	A11	Hàng 1, trái.
9	9	17	GPIO13 (D13)	B11	Hàng 1, phải.
10	10	18	GPIO16 (RX2)	N6	Hàng 2, trái.
11	11	19	GPIO17 (TX2)	D11	Hàng 2, phải.
12	12	20	GPIO26 (D26)	N9	Hàng 3, trái.
13	13	21	GPIO27 (D27)	N7	Hàng 3, phải.
14	14	22	GPIO32 (D32)	L9	Hàng 4, trái.
15	15	23	GPIO33 (D33)	N8	Hàng 4, phải.
–	**–**	**–**	**GND**	**GND**	**GND bất kỳ, bắt buộc nối chung.**

Hai bảng trên cũng chứng minh rằng các chân probe không trùng với clock 27

MHz, reset, LED hoặc tập chân RMII. Hướng tín hiệu duy nhất là ESP32 → FPGA; LA16 không phát tín hiệu điều khiển ngược lại ESP32, do đó vòng kiểm thử không có cơ chế backpressure ở tầng GPIO.

2.6.3 Cơ sở điện học của kết nối GND chung

Mức logic mà input buffer FPGA quan sát phụ thuộc vào hiệu điện thế giữa dây tín hiệu và GND cục bộ của FPGA:

$$V_{\mathrm{IN,FPGA}} = V_{\mathrm{OUT,ESP32}} + \left(V_{\mathrm{GND,ESP32}} - V_{\mathrm{GND,FPGA}}\right). \tag{2.14}$$

Nếu hai GND không được nối, đại lượng trong ngoặc không có giá trị tham chiếu xác định; điện dung ký sinh và dòng hồi qua các đường không chủ ý có thể làm tín hiệu vượt qua ngưỡng HIGH/LOW tại thời điểm không xác định. Dây GND chung tạo đường hồi dòng có trở kháng thấp và làm giảm sai khác common-mode giữa hai thiết bị. Vì vậy, kết nối này là điều kiện chức năng bắt buộc, không chỉ là biện pháp cải thiện chất lượng tín hiệu.

GND chung không loại bỏ hoàn toàn ground bounce, xuyên nhiễu hoặc sụt áp trên dây. Để giảm các thành phần này, dây tín hiệu và GND cần ngắn, có đường hồi gần nhau và tránh vòng dây lớn. Trong cấu hình nối trực tiếp phục vụ HIL, đầu vào vẫn phải nằm trong miền 0–3,3 V và có thể bổ sung điện trở nối tiếp 220–470 Ω để hạn dòng sự cố [51]. Khi sử dụng bo front-end hoàn chỉnh, miền mức cao 0,9–5 V được chuẩn hóa về mức an toàn trước khi tới FPGA.

2.6.4 Nguyên lý phát song song I2S0 LCD-Parallel và DMA

Ngoại vi I2S0 của ESP32 được cấu hình với `lcd_en=1`, độ rộng từ 16 bit và DMA descriptor mode. Nguồn PLL_D2 160 MHz đi qua bộ chia clock và bộ chia BCK bằng 2, nên tốc độ vector gần đúng là

$$f_g = \frac{160 \times 10^6}{2N_{\mathrm{clk}}}, \qquad 2 \le N_{\mathrm{clk}} \le 63. \tag{2.15}$$

Giá trị N_{clk} được chọn bằng phép làm tròn để tốc độ thực gần tốc độ test case yêu cầu nhất. Vì bộ chia nhận giá trị nguyên, firmware tính và công bố lại tốc độ thực để phục vụ hiệu chuẩn.

Bảng 2.9 Trình tự hoạt động của bộ phát I2S0 LCD-Parallel DMA

Pha	Thao tác	Ý nghĩa thời gian
1	Cấp phát buffer có thuộc tính `MALLOC_CAP_DMA`.	Bảo đảm DMA có thể truy cập trực tiếp vùng mẫu.
2	Hàm painter đặt hoặc xóa bit i trong từng $u[n]$.	Xác định trước toàn bộ timeline của CHi.
3	Hoán vị từng cặp `uint16_t` liền kề.	Bù thứ tự hai nửa từ của FIFO I2S ở mode 16 bit.
4	Chia buffer thành descriptor tối đa 4092 byte và nối vòng.	Cho phép phát liên tục mà không có khoảng dừng giữa hai vòng buffer.
5	Khởi động DMA output link và đặt `tx_start`.	I2S0 tự lấy từng từ và cập nhật 16 chân theo clock phần cứng.

Bảng 2.10 So sánh phương pháp sinh tín hiệu bằng phần mềm và I2S0 DMA

Tiêu chí	Bit-banging/LEDC riêng lẻ	I2S0 LCD-Parallel DMA
Đơn vị cập nhật	Một hoặc một nhóm GPIO theo chuỗi lệnh CPU.	Một từ 16 bit xác định đồng thời CH0–CH15.
Nguồn thời gian	Vòng lặp, timer, ngắt và thời điểm phục vụ của CPU.	Clock phần cứng chung của I2S0.
Jitter phần mềm	Chịu ảnh hưởng ngắt, cache và scheduler.	Không phụ thuộc thời điểm CPU ghi từng mẫu sau khi DMA bắt đầu.
Tải CPU khi phát	Tăng theo tần số cạnh và số kênh.	CPU chủ yếu dựng buffer và chuyển test case.
Khả năng tái lập	Phụ thuộc tải hệ thống tại thời điểm chạy.	Chuỗi đầu ra được xác định bởi buffer và bộ chia clock.

Cơ chế trên loại bỏ jitter tương đối do phần mềm ghi tuần tự từng GPIO, nhưng không triệt tiêu tuyệt đối skew vật lý. Thời điểm cạnh tại chân vẫn chịu chênh lệch nhỏ của GPIO matrix, output pad, đường mạch và dây nối. Tuy nhiên, các sai khác này có tính xác định và nhỏ hơn đáng kể so với biến thiên do lịch CPU, nên nguồn phát phù hợp để đánh giá sai số lượng tử hóa của LA16.

2.6.5 *Liên hệ với lấy mẫu bất đồng bộ và bản chất HIL*

Clock I2S0 của ESP32 và `clk_sample` của FPGA không có quan hệ pha. Nếu ESP32 tạo transition thứ m tại

$$t_m = mT_g + \phi, \tag{2.16}$$

với $T_g = 1/f_g$ và pha tương đối ϕ bất kỳ, LA16 ghi transition tại

$$\widehat{t}_m = \left\lceil \frac{t_m}{T_s} \right\rceil T_s, \qquad 0 \le \widehat{t}_m - t_m < T_s. \tag{2.17}$$

Do đó, I2S0 DMA tạo được quan hệ thời gian ổn định giữa 16 kênh phát nhưng không làm mất bản chất lấy mẫu bất đồng bộ ở phía FPGA. Mỗi cạnh vẫn được lượng tử hóa trên lưới $T_s \approx 5{,}70$ ns như phân tích tại mục 2.1. Đây là đặc điểm cần thiết của phép thử: nguồn phát loại bỏ phần lớn bất định phần mềm để sai số quan sát chủ yếu phản ánh sampler, đồng bộ 2FF và lượng tử hóa của thiết bị đo.

Mô hình được xem là Hardware-in-the-Loop vì đường kiểm thử đi qua mức điện áp và phần cứng thật, không chỉ qua dữ liệu mô phỏng. Nó đồng thời đánh giá: (i) ánh xạ chân và tính toàn vẹn kết nối; (ii) đồng bộ probe và timestamp; (iii) nén RLE cùng FIFO; (iv) truyền UDP; và (v) khả năng tái dựng, nhận dạng giao thức trên Host PRISM. Kết quả HIL vì vậy cung cấp bằng chứng hệ thống từ đầu phát vật lý đến giao diện người dùng.

2.7 Cơ sở lý thuyết của mạch phân áp bù tần số và lựa chọn bộ so sánh tốc độ cao

2.7.1 *Ảnh hưởng của điện dung ký sinh và điều kiện bù tần số*

Tín hiệu số tốc độ cao không chỉ được đặc trưng bởi tần số lặp mà còn bởi phổ hài rộng do thời gian sườn ngắn. Khi một bộ phân áp điện trở được mắc trước comparator, điện dung đầu vào của comparator, điện dung của diode bảo vệ, pad linh kiện và đường mạch tạo thành các nhánh điện dung song song với điện trở. Nếu hai nhánh của bộ phân áp có hằng số thời gian khác nhau, tỷ số truyền trở thành hàm của tần số; các thành phần hài cao bị suy hao khác với thành phần một chiều, làm biên độ sườn giảm và thời điểm vượt ngưỡng bị dịch chuyển.

Với cấu trúc bù gồm R_1 song song C_1 ở nhánh trên và R_2 song song C_2 ở nhánh dưới, trở kháng của mỗi nhánh là

$$Z_i(s) = \frac{R_i}{1 + sR_iC_i}, \qquad i \in \{1,2\}. \tag{2.18}$$

Hàm truyền của bộ phân áp được viết dưới dạng

$$H(s) = \frac{Z_2(s)}{Z_1(s) + Z_2(s)}. \tag{2.19}$$

Điều kiện để $H(s)$ không phụ thuộc s là hai hằng số thời gian bằng nhau:

$$R_1 C_1 = R_2 C_2. \tag{2.20}$$

Khi Phương trình (2.20) được thỏa mãn, tỷ số truyền ở miền tần số thấp và cao đều tiến tới cùng giá trị $R_2/(R_1+R_2)$. Với $R_1 = 1\,\mathrm{M\Omega}$, $R_2 = 1{,}5\,\mathrm{M\Omega}$ và tụ nhánh trên $C_1 = 10$ pF, tổng điện dung yêu cầu tại nhánh dưới là

$$C_{2,\mathrm{req}} = \frac{R_1}{R_2} C_1 = \frac{10^6}{1{,}5\times 10^6} \times 10 \approx 6{,}67\,\mathrm{pF}. \tag{2.21}$$

Giá trị trong Phương trình (2.21) là điện dung tương đương tổng nhìn từ nút comparator xuống nhánh dưới, không phải toàn bộ điện dung của tụ rời cần lắp. Tại nút này, điện dung đầu vào comparator $C_{\mathrm{comp}} = 1$ pF, điện dung tiếp giáp của steering diode BAT15-04W $C_{\mathrm{diode}} = 0{,}35$ pF và điện dung của đường mạch, pad cùng via được ước lượng $C_{\mathrm{PCB}} \approx 2$ pF [52, 71]. Vì các thành phần này mắc song song với tụ bù, điện dung danh định của tụ rời được xác định bởi

$$\begin{aligned} C_{2,\mathrm{actual}} &= C_{2,\mathrm{req}} - C_{\mathrm{comp}} - C_{\mathrm{diode}} - C_{\mathrm{PCB}} \\ &= 6{,}67 - 1 - 0{,}35 - 2 = 3{,}32\,\mathrm{pF}. \end{aligned} \tag{2.22}$$

Do đó, tụ C0G/NP0 danh định 3,3 pF là giá trị chuẩn gần nhất cho nguyên mẫu. Sai số linh kiện và sai số của ước lượng ký sinh làm thay đổi điều kiện cân bằng, vì vậy giá trị cuối cùng cần được kiểm tra bằng mô phỏng có mô hình ký sinh hoặc bằng phép đo đáp ứng sườn trên PCB.

2.7.2 Tỷ số phân áp và miền điện áp đồng pha

Mức logic cao của thiết bị cần đo nằm trong khoảng 0,9–5 V, trong khi front-end comparator và các bank I/O liên quan sử dụng nguồn 3,3 V. Chọn $R_1 = 1\,\mathrm{M\Omega}$ và $R_2 = 1{,}5\,\mathrm{M\Omega}$ tạo hệ số suy giảm

$$K = \frac{R_2}{R_1 + R_2} = \frac{1{,}5}{1 + 1{,}5} = 0{,}6. \tag{2.23}$$

Vì vậy, mức cao lớn nhất được đưa về $5 \times 0{,}6 = 3$ V, thấp hơn nguồn 3,3 V của comparator. Đối với mức cao ngoài nhỏ nhất $V_{\mathrm{IH,min}} = 0{,}9$ V, biên độ tại đầu vào tín hiệu của comparator và ngưỡng đặt giữa hai mức lần lượt là

$$V_{\mathrm{SIG,H,min}} = K V_{\mathrm{IH,min}} = 0{,}6 \times 0{,}9 = 0{,}54\,\mathrm{V}, \qquad V_{\mathrm{REF,min}} = \frac{V_{\mathrm{SIG,H,min}}}{2} = 0{,}27\,\mathrm{V}. \tag{2.24}$$

Điện áp đồng pha phải được kiểm tra tại trạng thái bất lợi nhất của hai đầu vào. Khi đầu vào tín hiệu đang ở mức thấp $V_{\mathrm{SIG,L}} = 0$ V còn đầu vào tham chiếu giữ ở 0,27 V, điện áp đồng pha nhỏ nhất là

$$V_{\mathrm{CM,min}} = \frac{V_{\mathrm{SIG,L}} + V_{\mathrm{REF,min}}}{2} = \frac{0 + 0{,}27}{2} = 0{,}135\,\mathrm{V}. \tag{2.25}$$

Giá trị gần rail âm này buộc comparator phải có đầu vào rail-to-rail hoặc miền đồng pha mở rộng xuống dưới GND. Điện áp offset cũng phải nhỏ so với biên dự trữ quanh ngưỡng, nếu không sai số offset sẽ trực tiếp làm thay đổi mức điện áp ngoài được nhận dạng là logic 1.

2.7.3 Ngân sách timing cho 12 kênh tốc độ cao

Đối với tín hiệu vuông 175 MHz, chu kỳ và độ rộng nửa chu kỳ danh định là

$$T_{175} = \frac{1}{175 \times 10^6} \approx 5{,}71\,\text{ns}, \qquad W_{175} = \frac{T_{175}}{2} \approx 2{,}86\,\text{ns}. \tag{2.26}$$

Nếu thời gian sườn chỉ được chiếm tối đa 30% độ rộng xung thì giới hạn lý thuyết là $0{,}30W_{175} \approx 0{,}86$ ns. Thiết kế sử dụng ngưỡng bảo thủ $t_r, t_f \leq 0{,}8$ ns. Tương tự, giới hạn 50% độ rộng xung cho trễ lan truyền tương ứng khoảng 1,43 ns; ngưỡng thiết kế được siết xuống

$$t_{pd} \leq 1{,}3\,\text{ns}. \tag{2.27}$$

Một chu kỳ vuông tạo hai chuyển mức nên tốc độ chuyển trạng thái yêu cầu tối thiểu là

$$R_{\text{toggle,min}} = 2f_{\text{in,max}} = 350 \times 10^6 \text{ transition/s}. \tag{2.28}$$

Bảng 2.11 So sánh comparator cho 12 kênh front-end tốc độ cao

Tiêu chí	TLV3801	ADCMP605	TLV3604
Trễ lan truyền điển hình	225 ps	1,5–1,6 ns	800 ps
Khả năng nhận xung hẹp	240 ps	Không công bố theo cùng điều kiện	600 ps
Tần số toggle cực đại	3 GHz, tương đương 6 Gtransition/s	Không công bố trực tiếp	1,5 GHz, tương đương 3 Gtransition/s
Miền đồng pha đầu vào	Rail-to-rail; nguồn đơn 2,7–5,25 V	$-0{,}2$ V đến $V_{CCI} + 0{,}2$ V	Mở rộng 0,2 V ngoài hai rail
Offset đầu vào cực đại	$\pm 0{,}5$ mV	Phụ thuộc cấp linh kiện	± 5 mV
Chuẩn đầu ra	LVDS vi sai	LVDS vi sai	LVDS vi sai
Đánh giá theo $t_{pd} \leq 1{,}3$ ns	Đạt, dự trữ lớn	Không đạt ngưỡng bảo thủ	Đạt

Các số liệu trong Bảng 2.11 được lấy theo datasheet nhà sản xuất [52, 53, 54]. ADCMP605 vẫn là comparator rail-to-rail có đầu ra LVDS, nhưng trễ lan truyền khoảng 1,5–1,6 ns vượt ngưỡng bảo thủ của thiết kế. TLV3604 đáp ứng timing song có trễ và offset lớn hơn TLV3801. TLV3801 được lựa chọn cho 12 kênh tốc độ cao nhờ trễ 225 ps, khả năng nhận xung 240 ps, miền đồng pha rộng, offset nhỏ và đầu ra LVDS tương thích trực tiếp với cặp đầu vào vi sai của FPGA. Ngoài lợi thế timing, linh kiện này tạo dự trữ đủ lớn cho sai lệch do overdrive, nhiệt độ và đường mạch.

Cần phân biệt khả năng chuyển mạch của front-end với băng thông đo của toàn hệ thống. Comparator có thể bảo toàn cạnh của tín hiệu 175 MHz, nhưng sampler bất đồng bộ 175,5 MS/s chỉ tạo xấp xỉ một mẫu trên một chu kỳ 175 MHz. Vì vậy, con số 175 MHz biểu thị năng lực điện của front-end và dự trữ thời gian của comparator, không phải cam kết tái dựng lossless một clock 175 MHz liên tục. Miền đo timing tin cậy vẫn phải tuân theo số mẫu trên chu kỳ đã phân tích tại Mục 2.1.

2.7.4 Tiêu chí cho bốn kênh tốc độ thấp

Ở nhóm bốn kênh 50 MHz, chu kỳ là 20 ns và độ rộng nửa chu kỳ là 10 ns. Thiết kế đặt $t_{pd} < 5$ ns để thời điểm cạnh không bị dịch quá nửa độ rộng xung, đồng thời sử dụng tiêu chí bảo thủ $t_r, t_f < 1{,}3$ ns và tốc độ toggle lớn hơn 200 MHz nhằm tạo dự trữ cho xung không đối xứng, jitter và các hài bậc cao.

Bảng 2.12 So sánh comparator cho bốn kênh front-end 50 MHz

Tiêu chí	TLV3601	LTC6752	ADCMP600
Trễ lan truyền	2,5 ns	2,9 ns	3,5 ns danh định; khoảng 5 ns tại overdrive nhỏ
Thời gian sườn	Theo tải đầu ra; nhận xung hẹp 1,25 ns	1,2 ns	Phụ thuộc tải, đầu ra CMOS/TTL
Tần số toggle cực đại	325 MHz	280 MHz	Không công bố trực tiếp
Miền đồng pha	Mở rộng 0,2 V ngoài hai rail	Rail-to-rail, mở rộng ngoài rail	$-0{,}2$ V đến $V_{CC} + 0{,}2$ V
Chuẩn đầu ra	Push-pull CMOS	Push-pull CMOS	TTL/CMOS
Đánh giá	Đạt toàn bộ tiêu chí	Đạt, trễ lớn hơn	Trễ đạt nhưng thiếu chỉ tiêu toggle bảo đảm

TLV3601 được lựa chọn vì trễ 2,5 ns, tần số toggle 325 MHz, đầu vào rail-to-rail và đầu ra push-pull có thể nối tới input logic đơn đầu của FPGA [55]. LTC6752 có thông số gần tương đương nhưng trễ lớn hơn; ADCMP600 có trễ danh định phù hợp nhưng không công bố tốc độ toggle cực đại theo cùng cách đánh giá [56, 57]. Việc tách 12 kênh TLV3801 và bốn kênh TLV3601 cho phép ưu tiên tài nguyên LVDS ở nhóm cần timing chặt, đồng thời giảm số cặp đường vi sai và linh kiện kết cuối ở nhóm 50 MHz.

2.7.5 Giới hạn chịu đựng quá áp một chiều liên tục

Mạch bảo vệ đầu vào sử dụng TVS TPD1E05U06 ở phía trước bộ phân áp và steering diode BAT15-04W tại nút comparator [70, 71]. Trong miền điện áp định mức, TVS hấp thụ các xung quá độ ngắn và giữ cho tầng sau không phải chịu năng lượng trực tiếp. Đối với sự cố quá áp một chiều kéo dài vượt ngưỡng kẹp của TVS, tài liệu thiết kế

xét trường hợp TVS không còn bảo vệ hiệu quả, trong khi BAT15-04W tiếp tục giới hạn nút comparator ở mức xấp xỉ $V_{\text{clamp}} = 3{,}6$ V đối với quá áp dương. Khi đó, $R_1 = 1\,\text{M}\Omega$ trở thành phần tử hạn dòng và công suất tiêu tán của điện trở được mô tả bởi

$$P_{R_1} = \frac{(V_{\text{IN}} - V_{\text{clamp}})^2}{R_1}. \tag{2.29}$$

Với điện trở SMD 0402 có công suất danh định $P_{\max} = 1/16$ W, điện áp rơi cực đại suy ra riêng từ điều kiện nhiệt là

$$V_{\text{res,max}} = \sqrt{P_{\max} R_1} = \sqrt{\frac{1}{16} \times 10^6} = 250\,\text{V}. \tag{2.30}$$

Dòng qua nhánh tại giới hạn này xấp xỉ $250\,\text{V}/1\,\text{M}\Omega = 250\,\mu\text{A}$. Theo mô hình kẹp đối xứng $\pm 3{,}6$ V của tài liệu thiết kế, các giới hạn điện áp đầu vào suy ra từ công suất điện trở là

$$\begin{aligned} V_{\text{IN,max}} &= V_{\text{res,max}} + V_{\text{clamp}} = 250 + 3{,}6 = 253{,}6\,\text{V}, \\ V_{\text{IN,min}} &= -V_{\text{clamp}} - V_{\text{res,max}} = -3{,}6 - 250 = -253{,}6\,\text{V}. \end{aligned} \tag{2.31}$$

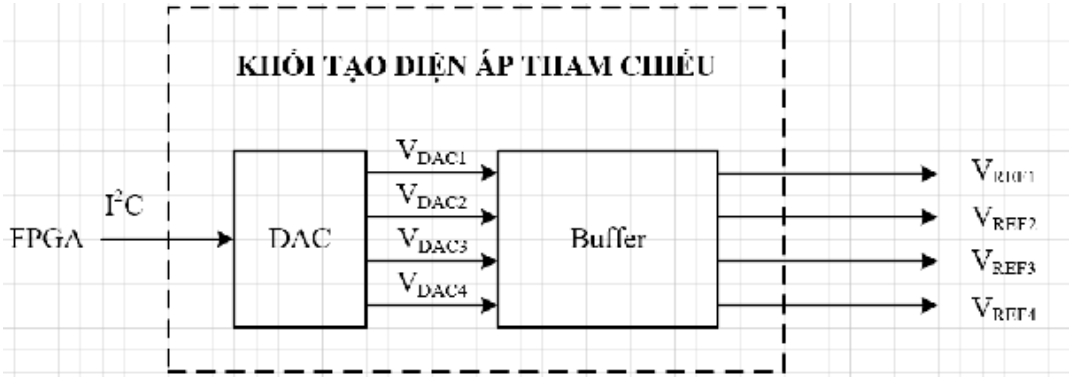
Từ các ngưỡng trên, miền điện áp đầu vào được phân thành ba vùng. (i) Trong vùng an toàn danh định của chuỗi bảo vệ từ $-0{,}6$ V đến 6,4 V, các linh kiện chưa vượt các ngưỡng bảo vệ được lựa chọn. Vùng này không đồng nhất với miền đo chính xác 0,9–5 V; tín hiệu nằm ngoài miền đo có thể bị kẹp hoặc không còn bảo đảm sai số biên độ. (ii) Trong các vùng chuyển tiếp từ 6,4 V đến 253,6 V và từ $-253{,}6$ V đến $-0{,}6$ V, TVS có thể đã chịu quá tải hoặc hỏng, còn điện trở R_1 và steering diode phải duy trì toàn bộ cơ chế hạn dòng; đây chỉ là miền phân tích khả năng tồn tại khi có sự cố, không phải miền vận hành liên tục cho phép. Công suất của R_1 tăng theo bình phương hiệu điện thế nên nhiệt độ linh kiện tăng nhanh khi tiến gần hai biên. (iii) Khi $V_{\text{IN}} > 253{,}6$ V hoặc $V_{\text{IN}} < -253{,}6$ V, công suất tính toán vượt $1/16$ W, điện trở có nguy cơ hỏng hở mạch hoặc cháy và kênh đo không còn được bảo đảm.

Hai giá trị $\pm 253{,}6$ V chỉ là giới hạn nhiệt lý thuyết suy ra từ công suất. Giới hạn thực tế còn phải lấy theo giá trị nhỏ nhất giữa điện áp làm việc cực đại của điện trở 0402, điện áp chịu đựng của BAT15-04W, khoảng cách creepage/clearance trên PCB, khả năng phóng điện bề mặt và trạng thái hỏng của TVS. Vì vậy, không được sử dụng kết quả này để công bố mạch có cấp cách điện hoặc điện áp làm việc liên tục 250 V nếu chưa có chuỗi điện trở phù hợp và kiểm chứng an toàn điện độc lập.

2.8 Cơ sở lý thuyết của khối tạo điện áp tham chiếu tĩnh và bộ đệm cách ly

Khối tạo điện áp tham chiếu nhận lệnh cấu hình từ FPGA qua bus I2C, chuyển mã số thành bốn điện áp analog bằng DAC và cách ly từng đầu ra trước khi phân phối tới các nhóm comparator. Cấu trúc chức năng được trình bày trong Biểu đồ 2.8. Việc sử

dụng bốn kênh độc lập cho phép các nhóm probe đồng thời làm việc với tối đa bốn họ mức logic mà không phải thay đổi mạng phân áp ở đầu vào.



Biểu đồ 2.8 Sơ đồ khối tạo bốn điện áp tham chiếu điều khiển qua I2C

Trong cấu trúc này, MCP4728 thực hiện phép chuyển đổi số–tương tự. TLV9064 được mắc thành bốn bộ lặp điện áp. DAC quyết định giá trị DC của ngưỡng; bộ đệm không khuếch đại điện áp nhưng làm tăng trở kháng nhìn từ phía DAC và giảm trở kháng nguồn nhìn từ phía comparator. Do đó, hai bài toán cần được kiểm tra riêng gồm độ chính xác tĩnh của ngưỡng và khả năng chống nhiễu dội ngược khi comparator chuyển trạng thái.

2.8.1 Ngân sách sai số và độ phân giải DAC

Điện áp tham chiếu quyết định trực tiếp ngưỡng chuyển mức của comparator. Đối với mức cao nhỏ nhất 0,9 V sau bộ phân áp $K = 0{,}6$, biên độ tại comparator là 0,54 V. Nếu tổng sai số ngưỡng chỉ được chiếm tối đa 5% biên độ này thì

$$V_{\text{REF_ERR}} = 0{,}9 \times 0{,}6 \times 0{,}05 = 27\,\text{mV}. \tag{2.32}$$

Do bốn kênh 50 MHz sử dụng TLV3601 có offset cực đại ± 5 mV, thiết kế lấy 5 mV làm trường hợp xấu nhất chung cho cả hai nhóm comparator. Sai số lượng tử cho phép của DAC vì vậy là

$$V_{\text{DAC_ERR}} = V_{\text{REF_ERR}} - V_{OS,\max} = 27 - 5 = 22\,\text{mV}. \tag{2.33}$$

Với DAC n bit và toàn thang V_{FS}, điều kiện cần là

$$V_{\text{LSB}} = \frac{V_{\text{FS}}}{2^n} < 22\,\text{mV}. \tag{2.34}$$

MCP4728 có độ phân giải 12 bit. Khi sử dụng tham chiếu nội $V_{\text{FS}} = 2{,}048$ V, kích thước một bước mã và sai số lượng tử cực đại khi làm tròn tới mã gần nhất là

$$V_{\text{LSB,MCP4728}} = \frac{2{,}048}{2^{12}} = 0{,}5\,\text{mV}, \qquad |e_{q,\max}| = \frac{V_{\text{LSB,MCP4728}}}{2} = 0{,}25\,\text{mV}. \tag{2.35}$$

So với ngân sách $V_{\mathrm{DAC_ERR}} = 22$ mV, bước mã 0,5 mV nhỏ hơn 44 lần, còn sai số làm tròn cực đại 0,25 mV nhỏ hơn 88 lần. Do đó, độ phân giải của MCP4728 tạo dự trữ lượng tử hóa lớn và cho phép hiệu chỉnh ngưỡng theo các bước nhỏ hơn đáng kể so với sai số cho phép.

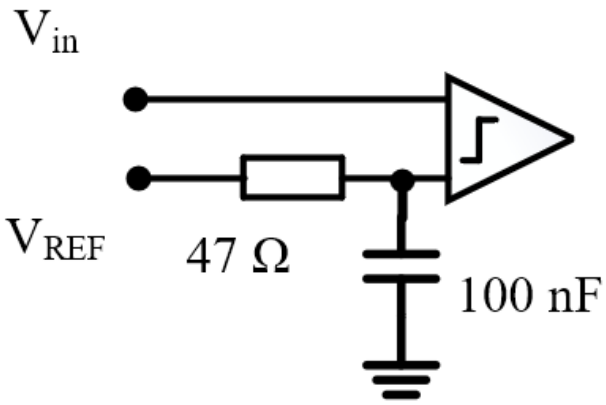
Bảng 2.13 So sánh DAC bốn kênh tạo điện áp tham chiếu

Tiêu chí	MCP4728	LTC2635-10	DAC43204
Độ phân giải	12 bit, 4096 mức	10 bit, 1024 mức	8 bit, 256 mức
Số kênh	4	4	4
Điện áp tham chiếu	Nội 2,048 V hoặc V_{DD}	Nội 2,5 V hoặc 4,096 V tùy biến thể	Nội, ngoài hoặc nguồn cấp
Kích thước LSB điển hình	0,5 mV tại 2,048 V	2,44 mV tại 2,5 V	9,77 mV tại 2,5 V
Bộ nhớ không mất dữ liệu	EEPROM lưu mã DAC và cấu hình	Tùy chọn reset về zero, mid-scale hoặc Hi-Z	NVM lập trình được
Giao tiếp	I2C tới 3,4 MHz	I2C 100/400 kHz	I2C, PMBus hoặc SPI tự nhận dạng
Đánh giá	Dự trữ phân giải lớn; khởi động đúng ngưỡng	Đạt phân giải nhưng không lưu mã ngưỡng như MCP4728	Đạt điều kiện 22 mV nhưng dự trữ nhỏ nhất

Cả ba linh kiện đều thỏa điều kiện lượng tử 22 mV, nhưng MCP4728 cung cấp bước 0,5 mV và EEPROM riêng cho mã của từng kênh [58]. Khi cấp nguồn, mã và cấu hình đã lưu được tự động nạp ra ngõ DAC trước khi FPGA hoàn tất cấu hình, nhờ đó comparator không ở trạng thái ngưỡng không xác định trong giai đoạn khởi động. LTC2635-10 có tham chiếu chính xác và nhiễu thấp nhưng chỉ cung cấp 10 bit; DAC43204 có NVM và nhiều chức năng thông minh nhưng độ phân giải 8 bit tạo dự trữ sai số thấp hơn [59, 60]. Vì vậy MCP4728 được lựa chọn. Tuy nhiên, độ phân giải 0,5 mV không đồng nghĩa với độ chính xác tuyệt đối 0,5 mV; sai số gain, offset, DNL, INL, sai số tham chiếu và sai số bộ đệm vẫn phải được cộng vào ngân sách tổng và xử lý bằng hiệu chuẩn khi đo phần cứng.

2.8.2 Bộ đệm cách ly nhiễu kick-back

Khi comparator chuyển trạng thái, điện tích từ tầng đầu vào có thể dội ngược về chân tham chiếu. Nếu DAC trực tiếp cấp một đường tham chiếu dài, dùng chung cho nhiều comparator, xung dòng này gây biến thiên điện áp cục bộ và làm thay đổi thời điểm chuyển mức của các kênh khác. Một op-amp mắc lặp điện áp, với đầu ra hồi tiếp về đầu vào đảo và DAC nối vào đầu vào không đảo, tạo trở kháng vào cao đối với DAC và trở kháng nguồn thấp đối với mạng comparator. Băng thông vòng kín và khả năng ổn định với tải điện dung quyết định tốc độ bộ đệm phục hồi sau xung kick-back.



Biểu đồ 2.9 Mạng RC cách ly tại đầu vào tham chiếu của comparator

Bảng 2.14 So sánh op-amp bốn kênh cho bộ đệm điện áp tham chiếu

Tiêu chí	TLV9064	2×OPA2333	AD8608
Số kênh trên một IC	4	2; cần hai IC cho bốn kênh	4
Offset cực đại	1,6 mV	10 μV	65 μV
Mật độ nhiễu băng rộng	10 nV/$\sqrt{\text{Hz}}$ điển hình	55 nV/$\sqrt{\text{Hz}}$ tại 1 kHz	8 nV/$\sqrt{\text{Hz}}$
GBW	10 MHz	0,35 MHz	10 MHz
Đặc tính liên quan	RRIO, ổn định gain 1, lọc RFI/EMI nội	Zero-drift, tối ưu tín hiệu DC	RRIO, nhiễu và offset thấp
Đánh giá	Phản hồi kick-back nhanh, chi phí và diện tích thấp	Chính xác DC cao nhưng phản hồi chậm và cần hai IC	Hiệu năng tốt nhưng chi phí cao hơn

Datasheet chính thức của họ OPAx333 chỉ liệt kê biến thể đơn OPA333 và kép OPA2333; không có biến thể bốn kênh OPA4333 của TI. Vì vậy Bảng 2.14 sử dụng hai OPA2333 làm phương án tương đương bốn kênh [62]. TLV9064 được lựa chọn do tích hợp bốn op-amp RRIO, GBW 10 MHz, ổn định ở gain bằng một và có bộ lọc RFI/EMI tại đầu vào [61]. AD8608 có độ chính xác tốt hơn và cùng GBW 10 MHz nhưng không tạo lợi thế tương xứng về chi phí trong ứng dụng này [63].

Điện trở cách ly $R = 47\,\Omega$ được đặt giữa đầu ra op-amp và nút tham chiếu; tụ $C = 100$ nF được đặt sát comparator. Hai linh kiện vừa tách tải điện dung khỏi vòng hồi tiếp op-amp, vừa tạo bộ lọc thông thấp bậc nhất có hàm truyền và hằng số thời gian gần đúng

$$H_{\text{RC}}(s) = \frac{1}{1 + sRC}, \qquad \tau = RC = 47 \times 100 \times 10^{-9} = 4{,}7\,\mu\text{s}. \tag{2.36}$$

Tần số cắt -3 dB của mạng RC là

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \times 47 \times 100 \times 10^{-9}} \approx 33{,}86\,\text{kHz}. \tag{2.37}$$

Điện áp tham chiếu là đại lượng tĩnh nên băng thông 33,86 kHz không làm giảm chức năng điều khiển ngưỡng. Theo mô hình bậc nhất, tại 1 MHz, tỷ số biên độ chỉ còn

$$|H_{\mathrm{RC}}(j2\pi f)|_{f=1\,\mathrm{MHz}} = \frac{1}{\sqrt{1+(f/f_c)^2}} \approx 0{,}0338, \tag{2.38}$$

tương ứng suy giảm khoảng 29,4 dB. Kết quả này cho thấy mạng RC làm giảm đáng kể thành phần kick-back ở miền MHz, trong khi op-amp vẫn duy trì giá trị DC của điện áp ngưỡng.

2.9 Cơ sở lý thuyết của khối dịch mức xung PWM

2.9.1 Yêu cầu dịch mức và lựa chọn kiến trúc

FPGA tạo PWM ở miền 3,3 V, trong khi thiết bị nhận có thể yêu cầu mức cao từ 0,9 V đến 3,3 V. Khối dịch mức phải giữ nguyên duty cycle trong giới hạn timing, hỗ trợ mức thấp gần GND và cho phép thay đổi nguồn kéo lên mà không sửa cấu hình I/O của FPGA.

Bảng 2.15 So sánh IC dịch mức cho xung PWM điều khiển được

Tiêu chí	LSF0204	TXB0104	TXS0104E	SN74LVC2T45
Số kênh	4	4	4	2
Điều khiển hướng	Tự động, pass-FET	Tự động	Tự động	Chân DIR
Kiểu tín hiệu	Open-drain và push-pull	Push-pull	Open-drain và push-pull	Push-pull
Tốc độ công bố	Tới 100 MHz up-translation; 200 Mbps tham số cực đại	100 Mbps	24 Mbps push-pull; 2 Mbps open-drain	Tới 420 Mbps tùy miền áp
Miền điện áp	$V_{refA} = 0{,}8$–$4{,}5$ V; $V_{refB} = 1{,}8$–$5{,}5$ V	A từ 1,2 V; B tới 5,5 V	A từ 1,65 V; B từ 2,3 V	Hai rail 1,65–5,5 V
Hạn chế chính	Cần điện trở kéo lên và tính tải RC	Không phù hợp open-drain hoặc tải ngoài mạnh	Không tạo được miền 0,9 V	Không đạt miền 0,9 V; cần DIR; chỉ hai kênh

LSF0204 được lựa chọn vì pass-FET không cần chân điều khiển hướng, hỗ trợ cả nguồn phát push-pull và open-drain, cho phép đặt V_{refA} tới 0,8 V và có chi phí thấp [64]. TXB0104 có tốc độ cao nhưng chỉ phù hợp push-pull; TXS0104E không hỗ trợ rail A dưới 1,65 V; SN74LVC2T45 có drive mạnh nhưng cần tín hiệu DIR, chỉ có hai kênh và không hỗ trợ nguồn 0,9 V theo điều kiện làm việc khuyến nghị [65, 66, 67].

2.9.2 Mạng định thiên và giới hạn dòng

Trong cấu hình của đề tài, $V_{refB} = 3{,}3$ V được lấy từ miền FPGA và V_{refA} mục tiêu bằng 0,9 V. Mạng định thiên gồm $R_{15} = 1\,\mathrm{M\Omega}$ và R_{14} được xác định theo quan hệ của mạch tham chiếu trong thiết kế:

$$R_{14} = \frac{200\,000 R_{15} V_{refA}}{(200\,000 + R_{15})(V_{refB} - V_{refA}) - 0{,}85 R_{15}}. \tag{2.39}$$

Thay $R_{15} = 1\,\mathrm{M\Omega}$, $V_{refA} = 0{,}9$ V và $V_{refB} = 3{,}3$ V vào Phương trình (2.39) thu được

$$\begin{aligned} R_{14} &= \frac{200\,000 \times 10^6 \times 0{,}9}{(200\,000 + 10^6)(3{,}3 - 0{,}9) - 0{,}85 \times 10^6} \\ &\approx 88{,}67\,\mathrm{k\Omega}. \end{aligned} \tag{2.40}$$

Giá trị chuẩn gần và dễ mua được chọn là $R_{14} = 82\,\mathrm{k\Omega}$. Giải Phương trình (2.39) theo V_{refA} cho phép kiểm tra lại điện áp thực tế:

$$V_{refA} = \frac{R_{14}\left[(200\,000 + R_{15}) V_{refB} - 0{,}85 R_{15}\right]}{200\,000 R_{15} + R_{14}(200\,000 + R_{15})} \approx 0{,}855\,\mathrm{V}. \tag{2.41}$$

Điện áp này thấp hơn mục tiêu 0,9 V khoảng 45 mV nhưng vẫn lớn hơn giới hạn hoạt động thấp 0,8 V của LSF0204. Do LSF0204 tích hợp điện trở 200 kΩ tại nhánh V_{refB}, giá trị cuối cùng phải được kiểm tra lại trên schematic PCB để tránh lắp lặp điện trở tham chiếu [68].

Đầu ra sử dụng $R_{pu} = 220\,\Omega$. Dòng xấu nhất khi kéo xuống tại rail 3,3 V, bỏ qua V_{OL} để đánh giá bảo thủ, là

$$I_{FET,\max} \approx \frac{3{,}3}{220} = 15\,\mathrm{mA}. \tag{2.42}$$

Giá trị này nằm tại giới hạn dòng thiết kế khuyến nghị; với điện áp kéo lên 0,9 V, dòng giảm còn khoảng 4,1 mA. Vì vậy $220\,\Omega$ tạo thỏa hiệp giữa sườn lên nhanh và công suất khi mức LOW [69].

2.9.3 Băng thông theo điện dung tải

Ngân sách điện dung tại một đầu ra được trình bày trong Bảng 2.16.

Bảng 2.16 Ngân sách điện dung tải của một kênh LSF0204

Thành phần	Khoảng điện dung
Chân I/O của FPGA hoặc MCU nhận	3–10 pF
Chân I/O và cấu trúc pass-FET của LSF0204	10,5–13 pF
Đường mạch, pad và via	1–2 pF
Tổng C_{Load}	**14,5–25 pF**

Mô hình gần đúng sử dụng trong thiết kế liên hệ tốc độ truyền với hằng số nạp RC:

$$R_b \approx \frac{1}{6R_{pu}C_{Load}}. \tag{2.43}$$

Với $R_{pu} = 220\,\Omega$, giới hạn dưới của tốc độ xuất hiện tại điện dung tải cực đại, còn giới hạn trên xuất hiện tại điện dung tải cực tiểu:

$$\begin{aligned} R_{b,\min} &= \frac{1}{6 \times 220 \times 25 \times 10^{-12}} \approx 30{,}3\,\text{Mbps}, \\ R_{b,\max} &= \frac{1}{6 \times 220 \times 14{,}5 \times 10^{-12}} \approx 52{,}24\,\text{Mbps}. \end{aligned} \tag{2.44}$$

Theo mô hình này, độ rộng mức HIGH tối thiểu tương ứng được viết dưới dạng

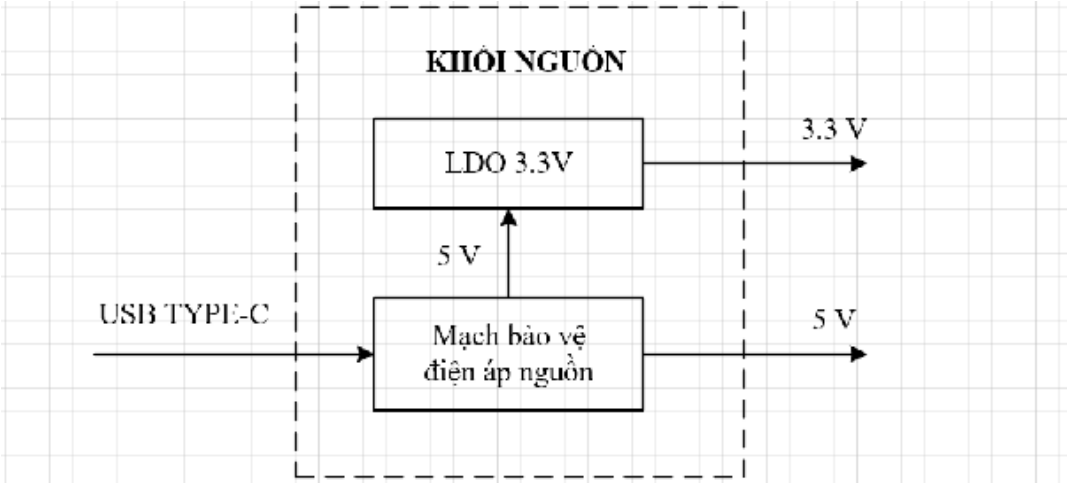
$$t_{\text{H,min}} \approx \frac{1}{R_b} = 6R_{pu}C_{Load} \approx 19{,}1\text{–}33{,}0\,\text{ns}. \tag{2.45}$$

Giá trị 19,1 ns chỉ áp dụng cho trường hợp tải nhẹ $C_{Load} = 14{,}5$ pF. Để bảo đảm hoạt động trên toàn khoảng điện dung đã dự trù, thiết kế phải sử dụng giới hạn bảo thủ $t_{\text{H,min}} \approx 33$ ns tại $C_{Load} = 25$ pF. Đây là ước lượng RC bậc nhất; timing cuối cùng còn phụ thuộc V_{OL} của nguồn phát, ngưỡng đầu vào tải, chiều dịch mức và hình học đường mạch. Datasheet LSF0204 cũng chỉ bảo đảm tốc độ cao khi điện dung tải và điện trở kéo lên được kiểm soát [64, 69].

2.10 Cơ sở lý thuyết của khối nguồn và bảo vệ nguồn

2.10.1 Kiến trúc cây nguồn 5 V và 3,3 V

Khối nguồn tiếp nhận điện áp 5 V từ USB Type-C, bảo vệ đường VBUS trước các sự cố ESD, quá dòng và ngắn mạch, sau đó phân phối thành hai miền. Miền 5 V đã bảo vệ cấp cho các tải cần điện áp vào trực tiếp và làm đầu vào cho bộ ổn áp tuyến tính; miền 3,3 V nhiễu thấp cấp cho comparator, DAC và bộ đệm analog. Kiến trúc tổng quát được trình bày trong Biểu đồ 2.10.

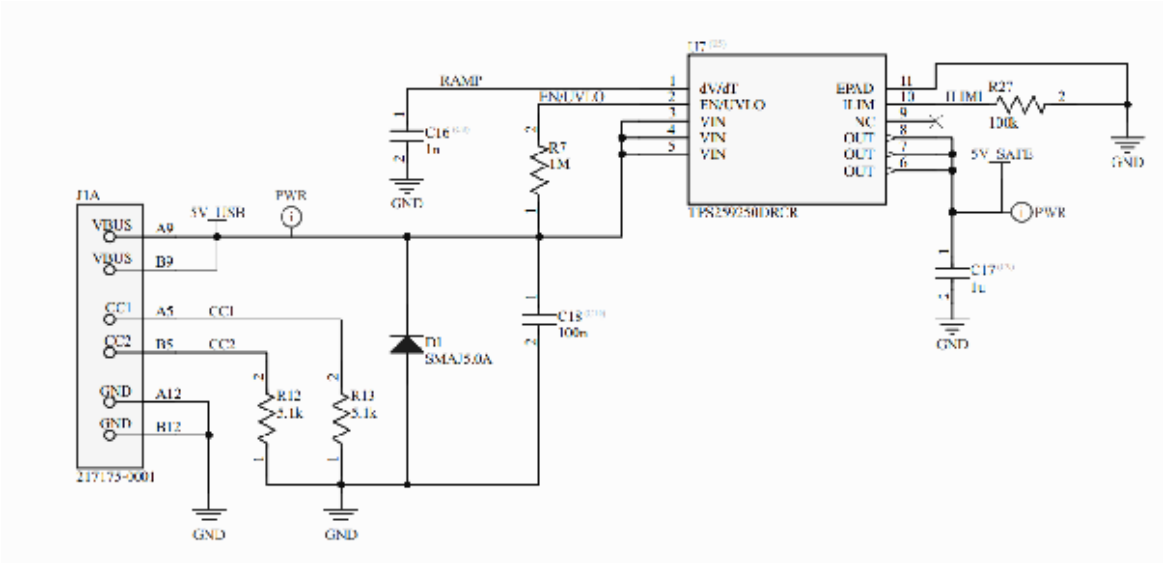


Biểu đồ 2.10 Sơ đồ khối nguồn 5 V và 3,3 V của mạch bảo vệ

Việc đặt mạch bảo vệ trước điểm phân nhánh làm cho cả đầu ra 5 V và LDO 3,3 V cùng được cách ly khỏi sự cố tại cổng USB-C. LDO được đặt sau eFuse để giới hạn năng lượng khi bản thân rail 3,3 V hoặc tải analog bị ngắn mạch. Cấu trúc phân tầng này tách ba chức năng: (i) TVS tạo đường xả cho xung quá độ có biên độ cao; (ii) eFuse giới hạn dòng và năng lượng kéo dài; và (iii) LDO giảm nhiễu cùng độ gợn trước khi cấp cho các phần tử quyết định ngưỡng so sánh.

2.10.2 Bảo vệ đầu vào USB-C bằng TVS và eFuse

Hai chân CC1 và CC2 của cổng USB-C được nối xuống GND bằng các điện trở 5,1 kΩ để công bố thiết bị ở vai trò tải nguồn. Tại VBUS, SMAJ5.0A được mắc song song nhằm dẫn xung ESD và surge xuống GND, trong khi TPS259250 được mắc nối tiếp để điều khiển dòng đi vào hệ thống. Sơ đồ chi tiết của tầng bảo vệ được trình bày trong Biểu đồ 2.11.



Biểu đồ 2.11 Sơ đồ chi tiết bảo vệ nguồn USB-C bằng SMAJ5.0A và TPS259250

TVS không thay thế phần tử giới hạn dòng vì diode chỉ thích hợp hấp thụ xung có thời gian ngắn. Khi quá dòng tồn tại liên tục, eFuse sử dụng MOSFET nội để điều chỉnh hoặc ngắt đường cấp, đồng thời thực hiện soft-start và bảo vệ nhiệt [75, 76]. Với TPS259250, quan hệ lựa chọn điện trở tại chân ILIM trong cấu hình của đề tài là

$$R_{\mathrm{ILIM}} = \frac{I_{\mathrm{LIM}} - 0{,}7}{3 \times 10^{-5}}. \tag{2.46}$$

Đặt $I_{\mathrm{LIM}} = 3{,}7$ A cho kết quả

$$R_{\mathrm{ILIM}} = \frac{3{,}7 - 0{,}7}{3 \times 10^{-5}} = 100\,\mathrm{k\Omega}. \tag{2.47}$$

Điện trở này xác lập ngưỡng danh định; sai số thực tế còn phụ thuộc dung sai nội của IC và nhiệt độ. Với điện trở dẫn nội điển hình khoảng $R_{\mathrm{ON}} = 30\,\mathrm{m\Omega}$, tổn hao dẫn được ước

lượng bởi $P_{\text{eFuse}} = I^2 R_{\text{ON}}$. Tại dòng tải 0,276 A, tổn hao chỉ khoảng 2,3 mW; khi dòng tiến tới 3,7 A, công suất tức thời tăng lên khoảng 0,41 W, do đó vùng đồng tản nhiệt vẫn phải được bố trí theo khuyến nghị của datasheet.

2.10.3 Ngân sách dòng và ổn áp 3,3 V nhiễu thấp

Nguồn 3,3 V cấp cho 12 comparator TLV3801, bốn comparator TLV3601, bốn kênh op-amp TLV9064, MCP4728, TCA9535 và các tải phụ. Sử dụng dòng tĩnh xấu nhất trong tài liệu thiết kế và dành thêm 10 mA cho các thành phần còn lại, dòng tải ước lượng là

$$\begin{aligned} I_{3,3,\text{est}} &= 12 \times 20 + 4 \times 6 + 4 \times 0{,}538 \\ &\quad + 0{,}05 + 0{,}022 + 10 \approx 276{,}2\,\text{mA}. \end{aligned} \tag{2.48}$$

Giá trị này nhỏ hơn giới hạn 500 mA của LT3045 và để lại khoảng 224 mA dự trữ cho biến thiên tải, dòng động cùng sai số mô hình [52, 55, 61, 58, 73, 77]. Đối với LDO lý tưởng, hiệu suất bị giới hạn bởi tỷ số điện áp:

$$\eta_{\text{LDO}} \approx \frac{V_{\text{OUT}}}{V_{\text{IN}}} = \frac{3{,}3}{5} = 66\%. \tag{2.49}$$

Công suất nhiệt của LT3045 tại tải ước lượng và tại tải cực đại lần lượt là

$$\begin{aligned} P_{\text{LDO,est}} &= (5 - 3{,}3) \times 0{,}2762 \approx 0{,}47\,\text{W}, \\ P_{\text{LDO,max}} &= (5 - 3{,}3) \times 0{,}5 = 0{,}85\,\text{W}. \end{aligned} \tag{2.50}$$

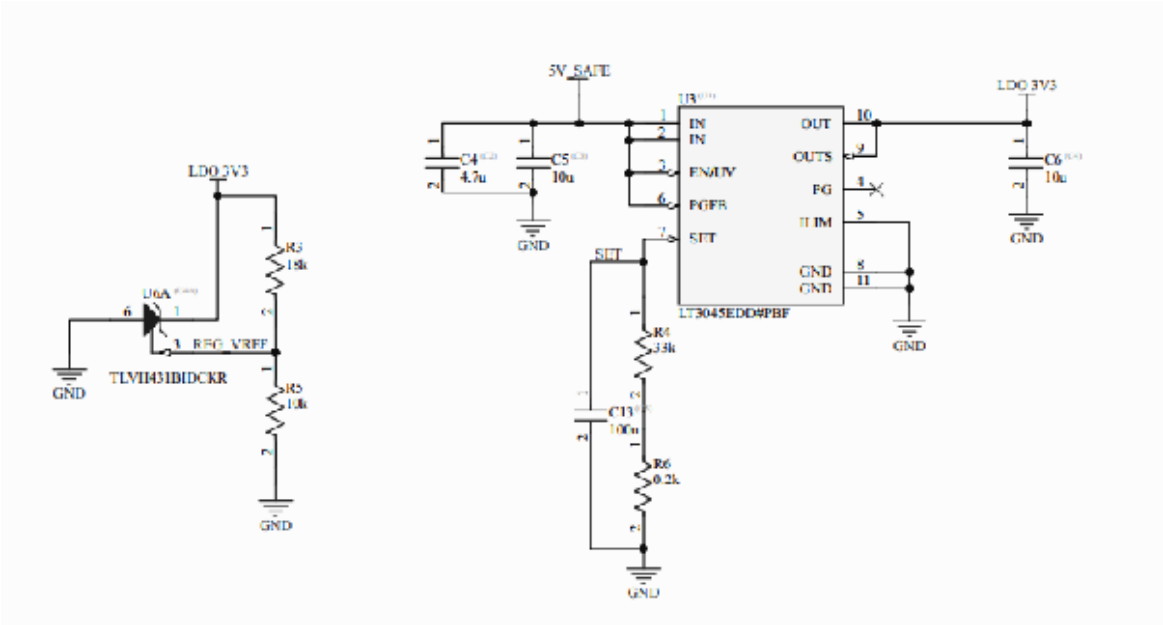
Kết quả cho thấy lựa chọn LDO tạo lợi thế về nhiễu nhưng yêu cầu vùng đồng tản nhiệt và thermal via phù hợp. Với dòng SET danh định 100 μA, điện trở $R_{\text{SET}} = 33\,\text{k}\Omega$ thiết lập điện áp đầu ra

$$V_{\text{OUT}} = I_{\text{SET}} R_{\text{SET}} = 100\,\mu\text{A} \times 33\,\text{k}\Omega = 3{,}3\,\text{V}. \tag{2.51}$$

2.10.4 Kẹp dòng dội ngược trên rail 3,3 V

Sơ đồ LT3045 và phần tử kẹp TLVH431 được trình bày trong Biểu đồ 2.12. LT3045 điều chỉnh tốt theo chiều cấp dòng cho tải nhưng không được sử dụng như một phần tử shunt để hấp thụ liên tục dòng trả về từ các steering diode. Khi nhiều đầu vào đồng thời quá áp, dòng dội ngược nạp tụ đầu ra và làm rail tăng theo quan hệ

$$\Delta V_{3,3} = \frac{I_{\text{inj}} \Delta t}{C_{\text{rail}}}. \tag{2.52}$$



Biểu đồ 2.12 Sơ đồ chi tiết nguồn LT3045 và mạch kẹp dòng dội ngược TLVH431

TLVH431 tạo đường shunt xuống GND khi rail vượt ngưỡng đặt. Với $V_{\mathrm{REF}} = 1{,}24$ V, $R_3 = 18\,\mathrm{k}\Omega$ và $R_5 = 10\,\mathrm{k}\Omega$, điện áp kẹp danh định là

$$V_{\mathrm{clamp}} = V_{\mathrm{REF}}\left(1 + \frac{R_3}{R_5}\right) = 1{,}24\left(1 + \frac{18}{10}\right) = 3{,}472\,\mathrm{V}. \tag{2.53}$$

Mức 3,472 V cao hơn rail danh định 3,3 V nên hạn chế dòng shunt trong chế độ bình thường, đồng thời thấp hơn ngưỡng đánh giá 3,6 V của các linh kiện nhạy cảm. Công suất của TLVH431 và phần tử hạn dòng cathode phải được kiểm tra theo tổng dòng dội ngược của các kênh; mạch kẹp này chỉ bảo vệ cục bộ rail 3,3 V và không thay thế TVS hoặc eFuse ở đầu vào [78].

Kết chương

Chương 2 đã xác lập cơ sở cho LA16 gồm lấy mẫu 175,5 MHz, đồng bộ 2FF, nén RLE, tái dựng timestamp và truyền UDP. Cấu trúc Data Frame cùng timing của UART, SPI, I2C, CAN và CLK/PWM tạo nền tảng cho decoder. Khảo sát phần cứng cho thấy Tang Primer 20K cung cấp tài nguyên logic, BSRAM, rPLL và Ethernet phù hợp, trong khi ESP32-WROOM-32 tạo vector kiểm thử 16 bit bằng I2S0 LCD-Parallel DMA. Bảng nối dây và phân tích GND chung xác lập điều kiện điện của mô hình HIL; quan hệ giữa clock phát và clock lấy mẫu tiếp tục tuân theo sai số lượng tử hóa nhỏ hơn một chu kỳ T_s. Phần cuối chương đã xây dựng điều kiện phân áp bù tần số, ngân sách timing comparator, ngân sách sai số DAC, cơ chế chống kick-back, giới hạn băng thông của mạch dịch mức PWM và cây nguồn bảo vệ gồm TVS, eFuse, LT3045 cùng TLVH431.

CHƯƠNG 3. TRIỂN KHAI DỰ ÁN

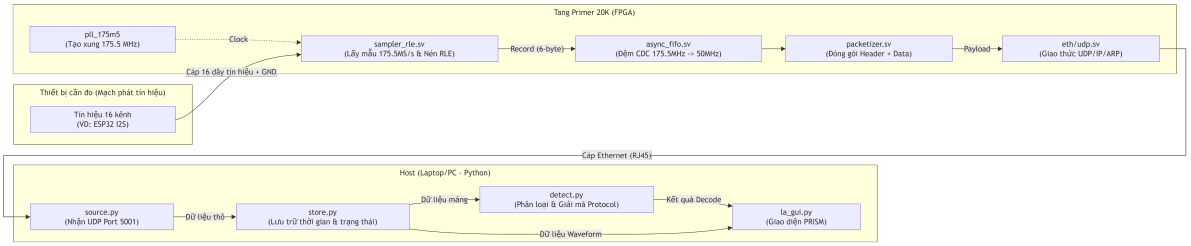
Tổng quan chương

Từ các cơ sở tính toán trong Chương 2, chương này trình bày quá trình hiện thực hóa LA16 thành một Logic Analyzer 16 kênh trên FPGA Tang Primer 20K. Hệ thống lấy mẫu ở 175,5 MHz, nén dữ liệu theo transition, truyền UDP qua Ethernet 100 Mbps và tái dựng waveform trên Host PRISM. Kiến trúc được chia thành front-end bảo vệ và chuẩn hóa mức điện áp, datapath FPGA tốc độ cao, tầng đóng gói mạng 50 MHz, khối nguồn sạch và phần mềm host theo mô hình multi-threading. Việc tạo timestamp hoàn toàn trong FPGA giúp độ trễ mạng và hệ điều hành không làm thay đổi quan hệ thời gian giữa các cạnh đã thu.

Quá trình triển khai được tổ chức thành bốn gói công việc liên kết: (i) Nguyễn Đức Phúc chủ trì RTL FPGA, CDC, packetizer và I2C Master điều khiển MCP4728, TCA9535; (ii) Bùi Đức Công Vinh phụ trách Host backend, heuristic, decoder, Ring Buffer và thiết kế TC0–TC8/SIL; (iii) Trần Quang Khải chủ trì bo mạch analog, nguồn bảo vệ, mô phỏng SPICE và driver I2S0–DMA trên ESP32; (iv) Nguyễn Minh Đức phụ trách GUI PRISM, decimation, PAUSE, Cursor A/B, điều khiển ngưỡng/PWM và đo kiểm HIL/timing. Các giao diện giữa bốn gói được đối chiếu bằng định dạng record RLE, giao thức UDP, ánh xạ 16 kênh và thang kiểm thử chung.

3.1 Sơ đồ khối và luồng dữ liệu hệ thống

3.1.1 Kiến trúc tổng thể



Biểu đồ 3.1 Sơ đồ khối hệ thống LA16

Luồng dữ liệu của LA16 được thể hiện trong Biểu đồ 3.1. Ranh giới Clock Domain Crossing nằm tại FIFO bất đồng bộ; do đó packetizer và UDP stack không nằm trên đường timing 5,698 ns của sampler.

Bảng 3.1 Ánh xạ các khối triển khai chính

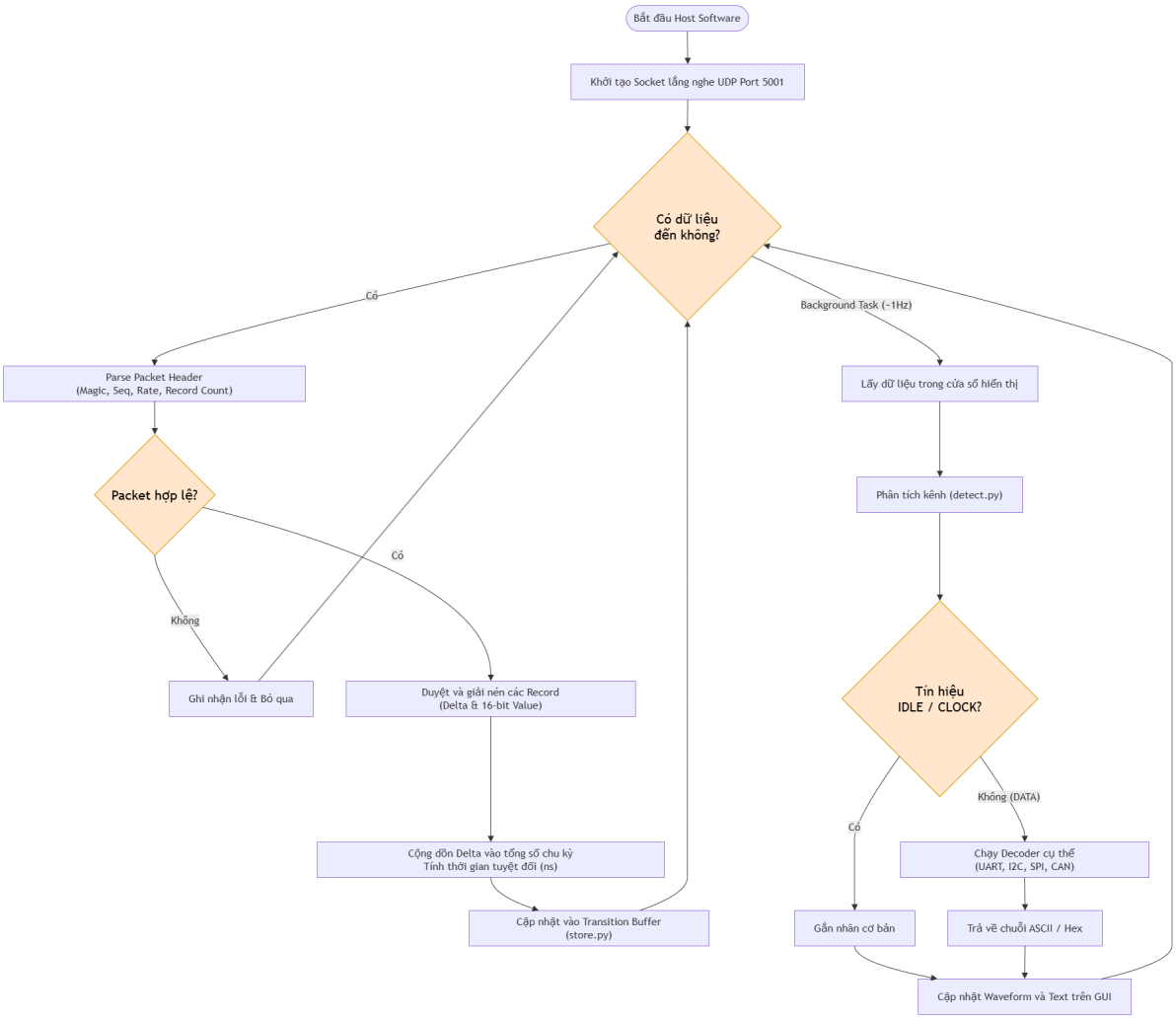
File	Khối	Clock	Vai trò
`pll_175m5.v`	rPLL	27/175,5 MHz	Tạo clock lấy mẫu và tín hiệu khóa pha.
`sampler_rle.sv`	Sampler RLE	175,5 MHz	Đồng bộ probe, phát hiện transition và tạo bản ghi RLE.
`async_fifo.sv`	Async FIFO	175,5/50 MHz	CDC bằng Gray-pointer và đệm burst trong BSRAM.
`packetizer.sv`	Packetizer	50 MHz	Tạo header, gom tối đa 200 bản ghi, flush và heartbeat.
`udp.sv`	UDP stack	50 MHz	Xử lý UDP/IP/ARP và giao tiếp PHY RMII.

Mỗi transition tạo một bản ghi 48 bit gồm số chu kỳ kể từ bản ghi trước và trạng thái mới của toàn bộ 16 kênh. Một thay đổi đồng thời trên nhiều kênh chỉ tạo một bản ghi nên quan hệ giữa các kênh được bảo toàn. Khi bus đứng yên, sampler chỉ tăng bộ đếm thời gian; dữ liệu chỉ được phát khi có transition hoặc keepalive.

```
sample_clock_hz   = 175_500_000
ethernet_clock_hz = 50_000_000
rle_record        = { value[15:0], delta[31:0] }
udp_destination   = "192.168.15.15:5001"
```

3.1.2 Sơ đồ quy trình xử lý dữ liệu (Flowchart)

Quy trình thu nhận và phân tích dữ liệu trên phần mềm Host được mô tả trong Biểu đồ 3.2. Sau khi khởi động, PRISM tạo socket UDP và lắng nghe tại cổng 5001. Từ thời điểm này, hệ thống duy trì hai luồng công việc độc lập: luồng tiếp nhận các gói dữ liệu do FPGA gửi tới và tác vụ nền phân tích cửa sổ tín hiệu đang được hiển thị.



Biểu đồ 3.2 Sơ đồ quy trình thu nhận và phân tích dữ liệu trên phần mềm Host

Khi một datagram được nhận, chương trình phân tích header để lấy các trường `magic`, `sequence`, tần số lấy mẫu và số lượng record. Gói không hợp lệ được ghi nhận lỗi và loại bỏ nhằm ngăn dữ liệu sai làm hỏng timeline. Với gói hợp lệ, từng record được giải nén thành cặp {`delta`, `value`}. Processing Core cộng dồn `delta` vào tổng số chu kỳ đã thu, chuyển kết quả sang thời gian tuyệt đối theo nano-giây và cập nhật `TransitionStore`. Sau khi hoàn tất, luồng mạng quay lại trạng thái chờ gói tiếp theo.

Song song với quá trình nhận gói, background task chạy với tần suất khoảng 1 Hz để lấy snapshot thuộc cửa sổ hiển thị. Module `detect.py` phân tích đặc trưng cạnh và trước hết phân loại kênh thành IDLE, CLOCK hoặc DATA. Các kênh IDLE/CLOCK được gán nhãn cơ bản và tính tần số, duty cycle; các kênh DATA tiếp tục được kiểm tra bởi decoder UART, I2C, SPI hoặc CAN. Waveform, nhãn kênh và chuỗi giải mã ASCII/hexadecimal sau đó được chuyển tới GUI. Cách tổ chức này tách đường nhận UDP khỏi công việc phân tích và render, nhờ đó hạn chế mất gói khi giao diện hoặc decoder cần nhiều thời gian xử lý.

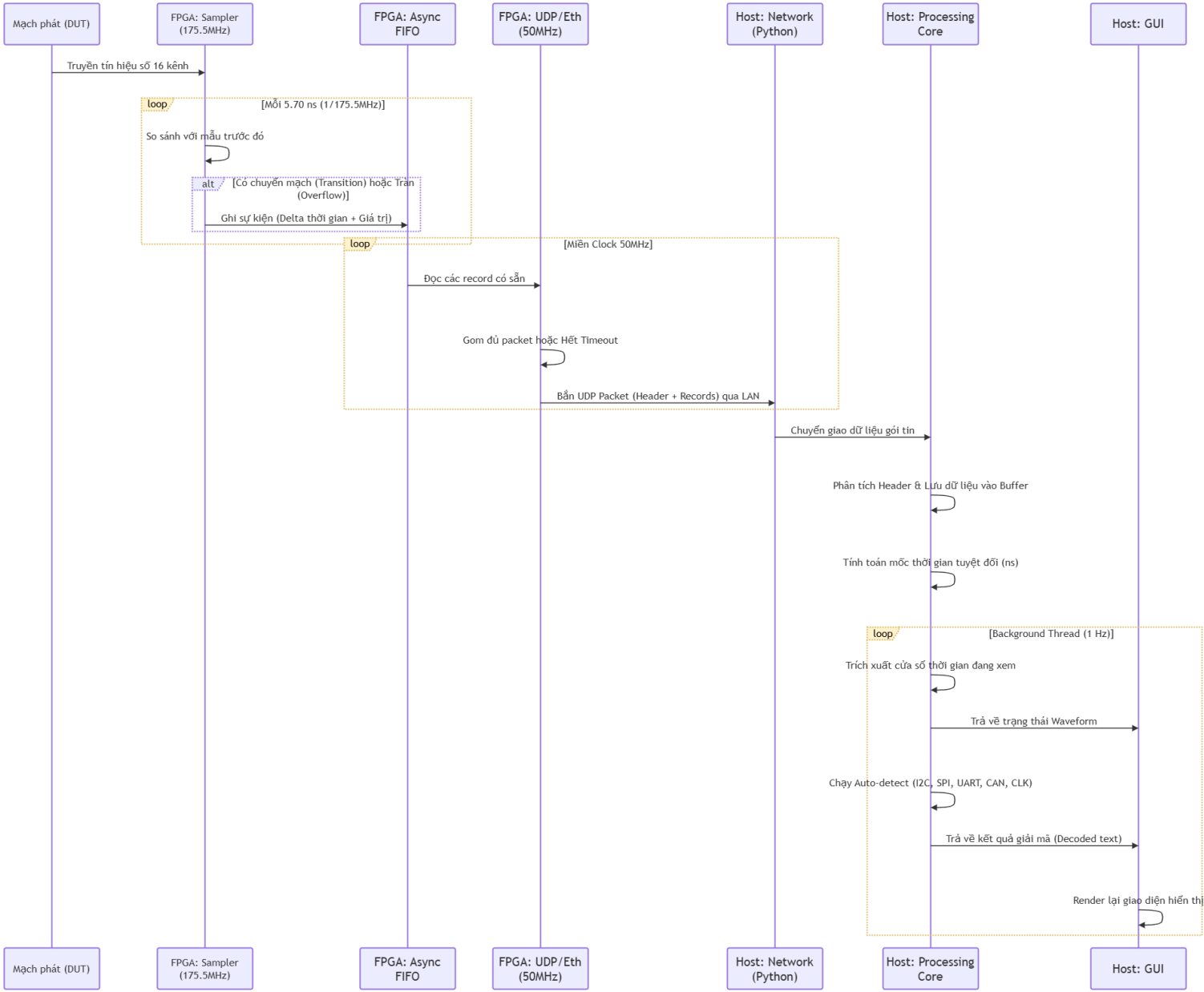
3.1.3 Sơ đồ tuần tự của hệ thống (Sequence Diagram)

Biểu đồ 3.3 thể hiện thứ tự tương tác giữa mạch phát tín hiệu, datapath FPGA và các thành phần phần mềm Host. Bảy thực thể tham gia gồm mạch phát DUT, sampler 175,5 MHz, FIFO bất đồng bộ, khối UDP/Ethernet 50 MHz, luồng mạng Python, Processing Core và GUI. Sơ đồ đồng thời làm rõ ba nhịp xử lý khác nhau: lấy mẫu tốc độ cao trong FPGA, truyền gói ở miền RMII và phân tích nền trên máy tính.

Đầu tiên, DUT đưa đồng thời 16 tín hiệu số tới sampler. Sau mỗi chu kỳ 5,70 ns, sampler chốt trạng thái đầu vào và so sánh với mẫu trước. Khi phát hiện transition hoặc sự kiện overflow cần báo cáo, FPGA tạo record gồm độ lệch thời gian và giá trị logic 16 bit rồi ghi vào FIFO bất đồng bộ. FIFO vừa hấp thụ các burst dữ liệu ở miền 175,5 MHz, vừa chuyển record an toàn sang miền Ethernet 50 MHz mà không đưa đường timing của packetizer vào datapath lấy mẫu.

Trong miền 50 MHz, packetizer liên tục đọc các record sẵn có. Khi đã gom đủ số record quy định hoặc bộ định thời flush hết hạn, hệ thống tạo datagram gồm header và payload rồi truyền qua LAN. Luồng Network trên Host nhận datagram và chuyển dữ liệu hợp lệ cho Processing Core. Tại đây, header được kiểm tra, record được lưu vào buffer và các `delta` được cộng dồn để khôi phục mốc thời gian tuyệt đối.

Cuối cùng, background thread định kỳ trích xuất cửa sổ dữ liệu đang quan sát. Processing Core gửi trạng thái waveform cho GUI, đồng thời chạy auto-detect và các bộ giải mã giao thức trước khi trả về nội dung decoded text. GUI chỉ đảm nhiệm cập nhật phần hiển thị. Vì timestamp được tạo ngay tại sampler trước các bước FIFO, mạng và GUI, sự thay đổi về độ trễ truyền hoặc thời gian render không làm sai lệch khoảng thời gian giữa các transition đã thu; các lớp phía sau chỉ ảnh hưởng đến độ trễ quan sát và độ đầy đủ của dữ liệu khi xảy ra quá tải.



Biểu đồ 3.3 Sơ đồ tuần tự thu nhận, truyền và hiển thị dữ liệu của hệ thống LA16

3.2 Thiết kế phần cứng FPGA

3.2.1 Clock, Synthesis, PnR và Timing Constraints

rPLL nhân clock thạch anh 27 MHz theo tỷ lệ 13/2 để tạo 175,5 MHz. Sampler chỉ thoát reset khi PLL đã khóa; tín hiệu `ready` của Ethernet cũng đi qua bộ đồng bộ hai tầng trước khi enable đường lấy mẫu. File `la16.sdc` khai báo clock cơ sở 27 MHz, clock RMII 50 MHz và generated clock 175,5 MHz. Các false path hai chiều giữa miền lấy mẫu và miền Ethernet chỉ được khai báo sau khi mọi Clock Domain Crossing đã đi qua FIFO hoặc 2FF sync.

```
create_clock clk_27m         period=37.037ns
create_clock netrmii_clk50m  period=20.000ns
create_generated_clock clk_sample multiply_by=13 divide_by=2
set_false_path clk_sample <-> netrmii_clk50m
```

Trong quy trình Synthesis, RTL được ánh xạ sang LUT, thanh ghi và BSRAM của GW2A-18; bước PnR tiếp theo thực hiện placement, routing và Static Timing Analysis. Báo cáo triển khai ngày 21/06/2026 cho biết Fmax của `clk_sample` đạt 182,142 MHz so với yêu cầu 175,5 MHz, không có endpoint vi phạm setup hoặc hold. Toàn thiết kế sử dụng 4078/20736 logic, 1648/16173 thanh ghi và 13/46 BSRAM, lần lượt tương ứng 20%, 11% và 29% tài nguyên chip.

3.2.2 Khối lấy mẫu pipeline và nén RLE

`sampler_rle.sv` là khối nhạy timing nhất và được chia thành ba tầng. Tầng 1 dùng mạch 2FF sync để chốt đồng thời 16 probe. Sau đó, hai byte được so sánh song song qua `chg_b`; cách chia nhỏ phép so sánh giảm fan-in và propagation delay so với một đường so sánh 16 bit dài.

Tầng 2 rút gọn hai cờ byte thành `changed_r` và chốt `value_r` từ `sync1`. Các phép gán non-blocking làm cờ thay đổi và dữ liệu có cùng tuổi pipeline. Vì vậy, nhiều bit đổi trong một chu kỳ vẫn được biểu diễn bằng đúng một trạng thái 16 bit.

Tầng 3 thực hiện ghi FIFO và quản lý `delta`. Bộ đếm 32 bit được tách thành `dlo` 12 bit và `dhi` 20 bit. Điều kiện `dlo==0xFFF` được chốt qua `carry_r`; `dhi_eff_r` bù trước carry đang bay. Đường tạo `wr_data` nhờ đó không chứa carry-chain 32 bit nhưng số chu kỳ vẫn tăng chính xác.

```
sync0 <= probe
sync1 <= sync0
chg_b <= { sync0[15:8] != sync1[15:8],
           sync0[7:0]   != sync1[7:0] }
changed_r <= chg_b[0] | chg_b[1]
value_r   <= sync1
```

```
dlo_wrap  = (dlo == 0xFFF)
carry_r  <= dlo_wrap
dhi      <= dhi + carry_r
dhi_eff_r <= dhi + carry_r + dlo_wrap
wr_data   = { value_r, dhi_eff_r, dlo }
```

Bản ghi đầu sau enable là baseline. Khi không có transition, bit `dhi[19]` trở thành điều kiện keepalive mà không cần bộ so sánh 32 bit; tại 175,5 MHz, ngưỡng này tương ứng khoảng 12,24 giây. Nếu FIFO đầy, `wr_en` bị chặn và `ovf_event` được phát, nhưng `delta` vẫn chạy. Bản ghi hợp lệ tiếp theo do đó giữ đúng thời điểm so với bản ghi đã ghi thành công gần nhất; chỉ các cạnh trong thời gian quá tải bị mất. Cờ `ovf_ever` giữ đến reset, còn `ovf_recent` được kéo dài để đưa vào header và GUI.

3.2.3 FIFO bất đồng bộ Gray-pointer

`async_fifo.sv` duy trì con trỏ nhị phân cục bộ để truy cập RAM và con trỏ Gray để truyền sang miền clock còn lại. Mỗi con trỏ Gray đi qua bộ đồng bộ 2FF sync hai chiều trước khi dùng để xác định `full` hoặc `empty`. Do hai giá trị Gray liên tiếp chỉ đổi một bit, miền nhận không quan sát tổ hợp trung gian nguy hiểm như khi truyền trực tiếp bộ đếm nhị phân.

```
gray_ptr = binary_ptr ^ (binary_ptr >> 1)
write_full = next_write_gray == inverted_synced_read_gray
read_empty = next_read_gray  == synced_write_gray
```

Ở miền ghi, các giá trị `wr_bin+1`, Gray đứng yên, Gray tăng và con trỏ đọc đảo hai bit cao đều được tính trước; đường `full` chỉ còn multiplexer và phép so sánh. Trong `la16_top.sv`, tham số `W=48`, `AW=10` tạo FIFO 1024 bản ghi 48 bit. Mẫu truy cập synchronous làm công cụ infer mảng nhớ thành Block RAM/BSRAM tốc độ cao. Phía đọc dùng request/valid: `fifo_rd` yêu cầu đọc và `fifo_rd_valid` xác nhận dữ liệu ở chu kỳ sau.

3.2.4 Packetizer và UDP stack

Packetizer đọc record ở 50 MHz, lưu vào `pkt_buf` và chỉ chốt gói khi không còn yêu cầu FIFO đang bay. Máy trạng thái lần lượt thu thập dữ liệu, phát 16 byte header, nạp từng record, phát sáu byte little-endian và kích hoạt `tx_req`. Sequence tăng với mọi gói, kể cả heartbeat.

Bảng 3.2 Cấu trúc header UDP 16 byte

Offset	Dài	Trường	Nội dung
0	4	Magic	Chuỗi `LA16`.
4	1	Version	Phiên bản bằng 1.
5	1	Flags	Bit 0: overflow gần đây; bit 1: từng overflow.
6	2	Count	Số record, tối đa 200; heartbeat bằng 0.
8	4	Sequence	Phát hiện mất gói.
12	4	Sample rate	175.500.000 Hz.

```
packet = header_16_bytes + records
record = delta_u32_le + value_u16_le
send_if count == 200
send_if oldest_record_wait >= 5ms
send_heartbeat_if idle_time >= 200ms
```

`udp.sv` xử lý UDP/IP/ARP và điều khiển PHY qua RMII. FPGA dùng địa chỉ `192.168.15.14`, host dùng `192.168.15.15`, cổng UDP 5001. Khi stack chưa `ready`, FIFO được xả để sau khi link lên chỉ truyền dữ liệu mới. Flush 5 ms hạn chế trễ hiển thị khi gói chưa đầy; heartbeat 200 ms giúp host phân biệt bus im lặng với mất kết nối.

3.3 Thiết kế phần mềm Host PRISM

3.3.1 Kiến trúc multi-threading và Ring Buffer thread-safe

`UdpSource` chạy trong daemon thread, mở socket cổng 5001 và đặt bộ đệm nhận 4 MiB. `packet.py` kiểm tra magic, độ dài header và kích thước payload trước khi trả về danh sách record. Sequence được so sánh theo số học 32 bit; khoảng nhảy lớn hơn một được cộng vào `drops`. Heartbeat vẫn cập nhật thời điểm nhận cuối nên trạng thái kết nối còn sống khi probe không đổi.

`TransitionStore` sử dụng hai `deque` đồng bộ, giới hạn 2 triệu transition cho thời gian và giá trị 16 bit. Một `threading.Lock` bảo vệ thao tác ghi packet, lấy snapshot và đọc thống kê. Snapshot chỉ giữ khóa trong lúc sao chép sang NumPy; vẽ waveform và chạy decoder diễn ra ngoài critical section, tránh chặn luồng UDP. Khi cắt cửa sổ, store giữ thêm transition ngay trước điểm đầu để xác định mức nền.

Timeline được lưu dưới dạng tổng số chu kỳ nguyên. Phép chuyển đổi sang nano-giây chỉ thực hiện khi ghi CSV, hiển thị hoặc tính khoảng cách, qua đó triệt tiêu độ trôi tích luỹ dấu phẩy động.

```
cycles += delta
time_ns = cycles * 1_000_000_000 // sample_rate
times.append(time_ns)
```

```
values.append(value_u16)
```

Overflow FPGA và mất gói mạng được thống kê riêng. `ovf_recent`/`ovf_ever` phản ánh FIFO quá tải trước packetizer, còn `drops` phản ánh sequence UDP bị thiếu; người dùng vì thế có thể xác định đúng lớp gây mất dữ liệu.

3.3.2 Heuristic algorithm tự động nhận dạng

`detect.py` tách timeline sparse thành chuỗi cạnh của từng kênh, sau đó đo số cạnh, chu kỳ, độ biến thiên chu kỳ, duty cycle và khoảng nghỉ. Kênh không có cạnh được gán IDLE; chu kỳ đều và phủ phần lớn cửa sổ được gán CLOCK; clock chỉ đều trong nhiều cụm được gán BURSTY_CLOCK; phần còn lại là DATA. Ở bước cuối, luật quyết định trong Phương trình (2.12) biến CLOCK có duty cycle từ 44% đến 56% thành CLK, còn tín hiệu tuần hoàn nằm ngoài khoảng này thành PWM.

```
classify_channel:
    no_edge       -> IDLE
    stable_period -> CLOCK
    stable_bursts -> BURSTY_CLOCK
    otherwise     -> DATA

finalize CLOCK:
    duty in [44%, 56%] -> CLK
    otherwise           -> PWM
```

I2C và SPI được ghép trước từ BURSTY_CLOCK và các kênh đối tác. CAN được thử trước UART vì cả hai có thể idle-high và bắt đầu bằng sườn xuống; CRC15 là điều kiện mạnh hơn để xác nhận CAN. Thứ tự này giảm việc một decoder đơn kênh chiếm nhầm tín hiệu thuộc bus khác.

3.3.3 Các bộ giải mã giao thức

UART ước lượng bit-time từ nhóm khoảng cạnh ngắn sau khi loại glitch dưới 100 ns theo Phương trình (2.8), kiểm tra các khoảng còn lại có gần bội số nguyên hay không và áp dụng luật snap 6% trong Phương trình (2.9). Decoder lấy mẫu tám bit tại tâm bit theo thứ tự LSB-first, kiểm tra start thấp và stop cao, tương ứng khung 8N1.

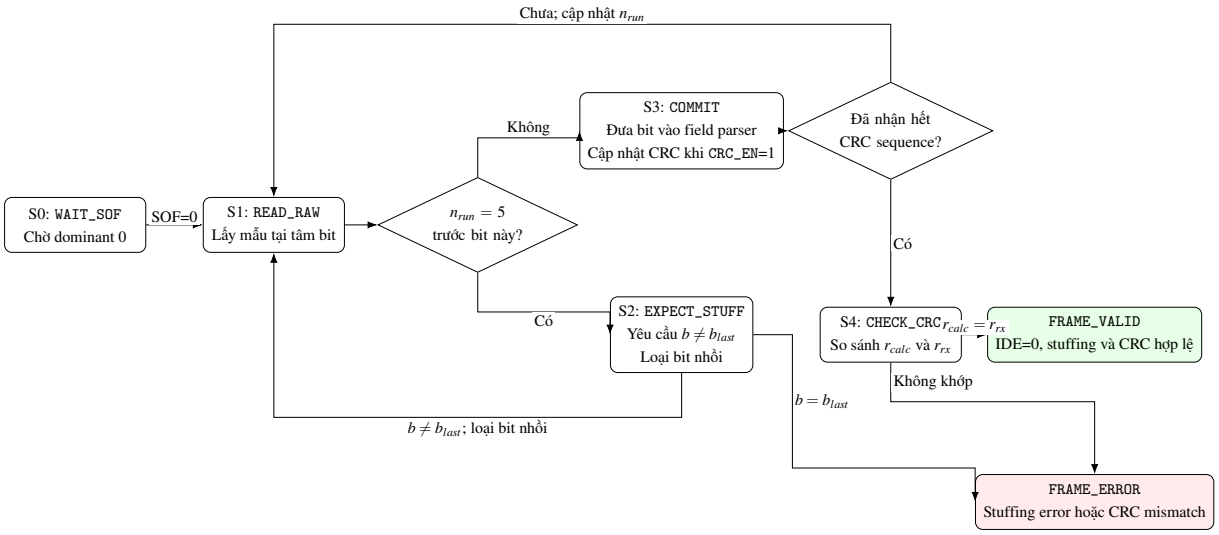
I2C trộn các cạnh SCL/SDA theo thời gian. START là SDA chuyển cao xuống thấp khi SCL cao; STOP là SDA chuyển thấp lên cao khi SCL cao theo NXP UM10204. SDA được chốt trên sườn lên SCL. Tám bit đầu tạo byte, bit thứ chín cho ACK khi thấp hoặc NAK khi cao; byte đầu được tách thành địa chỉ bảy bit và hướng R/W.

SPI hỗ trợ Mode 0: SCK nghỉ thấp, MOSI được chốt trên sườn lên khi CS active-low, tám bit ghép theo MSB-first. Heuristic chọn CS nếu tín hiệu thấp bao phủ burst

SCK và cao giữa các burst; MOSI phải có phần lớn transition nằm trong các burst tương ứng.

CAN hỗ trợ Classic 2.0A chuẩn 11-bit. Sau SOF, bộ đọc bỏ bit nhồi sau mỗi chuỗi năm bit đồng mức, rồi giải mã ID, RTR, IDE, DLC và tối đa tám byte. CRC15 được tính lại với đa thức `0x4599`; frame CRC sai hoặc IDE khác không không được dùng để xác nhận CAN. Nhờ đó nhiễu và chuỗi cạnh ngẫu nhiên không đạt CRC bị loại khỏi kết quả.

Biểu đồ 3.4 mô tả máy trạng thái xử lý chuỗi bit CAN. Bộ đếm n_{run} theo dõi số bit vật lý liên tiếp cùng mức. Khi $n_{run} = 5$, trạng thái `EXPECT_STUFF` yêu cầu bit kế tiếp phải đảo mức; bit này chỉ có chức năng đồng bộ nên bị loại khỏi luồng dữ liệu logic. Nếu bit thứ sáu vẫn cùng mức, FSM phát hiện stuffing error và loại frame. Các bit hợp lệ được chuyển tới field parser; riêng khoảng từ SOF đến hết Data field đồng thời kích hoạt khối CRC15.



Biểu đồ 3.4 FSM khử bit nhồi và kiểm tra CRC15 của decoder CAN Classic 2.0A

Với thanh ghi CRC $\boldsymbol{r}^{(k)}$ rộng 15 bit và bit dữ liệu đã khử nhồi b_k, bit hồi tiếp và phép cập nhật LFSR được biểu diễn bởi

$$a_k = b_k \oplus r_{14}^{(k)}, \qquad \boldsymbol{r}^{(k+1)} = \Big[\Big(\boldsymbol{r}^{(k)} \ll 1\Big) \,\&\, 0\text{x}7\text{FFF}\Big] \oplus \begin{cases} 0\text{x}4599, & a_k = 1, \\ 0\text{x}0000, & a_k = 0. \end{cases} \tag{3.1}$$

Giá trị `0x4599` tương ứng đa thức trong Phương trình (2.10). Điều kiện xác nhận cuối cùng là

$$\text{valid}_{CAN} = \neg\text{stuff_error} \;\wedge\; (\text{IDE} = 0) \;\wedge\; (\boldsymbol{r}_{calc} = \boldsymbol{r}_{rx}). \tag{3.2}$$

Bảng 3.3 Quy tắc chính của các decoder

Giao thức	Quy tắc lấy mẫu	Kết quả
UART	Tâm bit sau start; LSB-first; snap baud ±6%	Byte 8N1 và frame lỗi.
I2C	SDA tại sườn lên SCL; START/STOP khi SCL cao	Địa chỉ, R/W, ACK/NAK, dữ liệu.
SPI	Sườn lên SCK khi CS thấp; MSB-first	Byte của từng burst Mode 0.
CAN	Tâm bit; destuff sau năm bit; kiểm tra CRC15	ID, RTR, DLC, data và CRC.

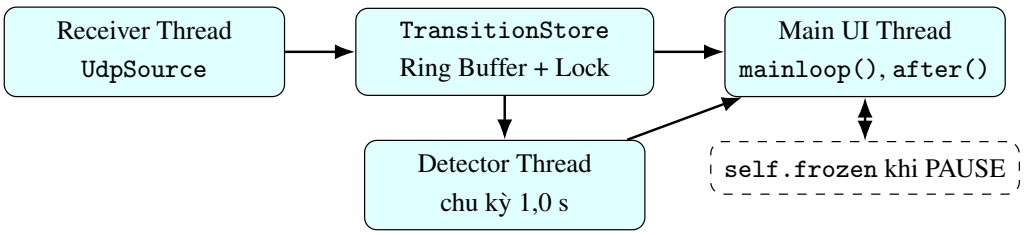
Timestamp được tạo trước tầng mạng nên packet đến chậm không làm sai timing. Sequence, overflow flags và kiểm tra frame không khôi phục transition đã mất, nhưng cho phép GUI cảnh báo capture không đầy đủ thay vì trình bày dữ liệu thiếu như một kết quả hợp lệ.

3.4 Thiết kế và Hiện thực hóa Giao diện Đồ họa PRISM

Giao diện PRISM được hiện thực trong `la_gui.py` bằng CustomTkinter kết hợp Matplotlib. Phần mềm phải đồng thời tiếp nhận UDP, duy trì timeline, phân tích giao thức và phản hồi thao tác người dùng. Vì vậy, các công việc có khả năng chặn được tách khỏi vòng lặp giao diện; dữ liệu vẽ được giới hạn và các thao tác đo sử dụng snapshot nhất quán.

3.4.1 Kiến trúc đa luồng và đồng bộ dữ liệu thời gian thực

Ba luồng thực thi chính được mô tả trong Biểu đồ 3.5. Receiver Thread do `UdpSource` quản lý mở socket cổng 5001, đặt `SO_RCVBUF` bằng 4 MiB, kiểm tra datagram và ghi record hợp lệ vào `TransitionStore`. Hai `deque` giới hạn hai triệu transition trong store được bảo vệ bằng `threading.Lock`, nên thao tác ghi không xung đột với việc lấy snapshot.



Biểu đồ 3.5 Kiến trúc đa luồng của giao diện PRISM

Detector Thread là daemon thread, nghỉ 1,0 s giữa hai lần gọi `detect.analyze()`. Luồng này phân tích nguồn live, file hoặc snapshot PAUSE trên cửa sổ hiện hành rồi thay thế toàn bộ `det_result`. Main UI Thread chạy `mainloop()` và lập lịch `_tick()` bằng `after(150, ...)`, tương ứng nhịp cập nhật danh định $f_{\mathrm{UI}} \approx 1/0{,}150 = 6{,}67$ Hz. Các thao tác sửa widget và yêu cầu vẽ Matplotlib đều nằm trong Main UI Thread, tránh truy cập Tkinter từ luồng nền. Cấu hình thực tế ưu tiên độ ổn định thay vì đặt mục tiêu 30 FPS.

3.4.2 Bộ dựng dạng sóng Matplotlib và Decimation

PRISM nhúng `FigureCanvasTkAgg` vào CustomTkinter và tạo 16 trục Matplotlib xếp dọc. `numpy.searchsorted` giới hạn record theo cửa sổ; bit thứ i của `value` tạo mức $y_i \in \{0,1\}$ và `ax.step(..., where="post")` giữ trạng thái giữa hai transition. Có thể diễn giải điện áp logic danh định bởi $V_i(t) = 3{,}3y_i(t)$ V, nhưng đồ thị thực tế dùng tọa độ chuẩn hóa 0/1, không phải phép đo tương tự.

Trước khi vẽ, engine loại các record không làm thay đổi kênh đang xét bằng phép so sánh `diff(bits)`. Nếu số cạnh còn lại là N và vượt `MAX_DRAW_EDGES`=4000, hệ số lấy cách quãng được chọn theo

$$s = \left\lfloor \frac{N}{4000} \right\rfloor + 1, \tag{3.3}$$

sau đó chỉ các phần tử $0, s, 2s, \ldots$ được chuyển cho Matplotlib. Đây là subsampling, không phải nội suy. Hai điểm biên làm đường bậc thang phủ hết cửa sổ; `draw_idle()` trì hoãn render tới khi Tk rảnh, còn switch kênh loại bỏ công việc vẽ của kênh bị ẩn.

3.4.3 Cơ chế đóng băng Snapshot không chặn khi PAUSE

Khi PAUSE, `_toggle_pause()` sao chép `store.snapshot()` vào `frozen`; waveform và Detector Thread cùng đọc ảnh chụp này nên người dùng có thể zoom, pan, đặt cursor và xem decode trên một timeline tĩnh. Receiver Thread vẫn ghi dữ liệu mới vào Ring Buffer. Khi RESUME, `frozen` được giải phóng, `follow=True` và giao diện trở lại cuối luồng live. PAUSE không tự gây mất cạnh, nhưng UDP drop, overflow FPGA hoặc việc Ring Buffer ghi đè dữ liệu quá cũ vẫn có thể xảy ra độc lập.

3.4.4 Cursor A/B đo thời gian và tần số

Matplotlib cung cấp tọa độ x theo mili-giây; chuột trái đặt Cursor A và chuột phải đặt Cursor B. Phép đổi đơn vị, khoảng thời gian và tần số được xác định bởi

$$t_A = x_A 10^6, \quad t_B = x_B 10^6, \quad \Delta t = |t_B - t_A|, \quad f = \frac{10^9}{|t_B - t_A|}\ \text{Hz}, \tag{3.4}$$

với t_A, t_B tính bằng nano-giây. `fmt_t()` tự chọn s, ms, μs hoặc ns. Hai đường cursor được đồng bộ trên 16 trục để đo chu kỳ clock, bit-time UART, CAN hoặc khoảng sự

kiện I2C.

3.4.5 Phân vùng điều khiển và bảng đọc DECODE

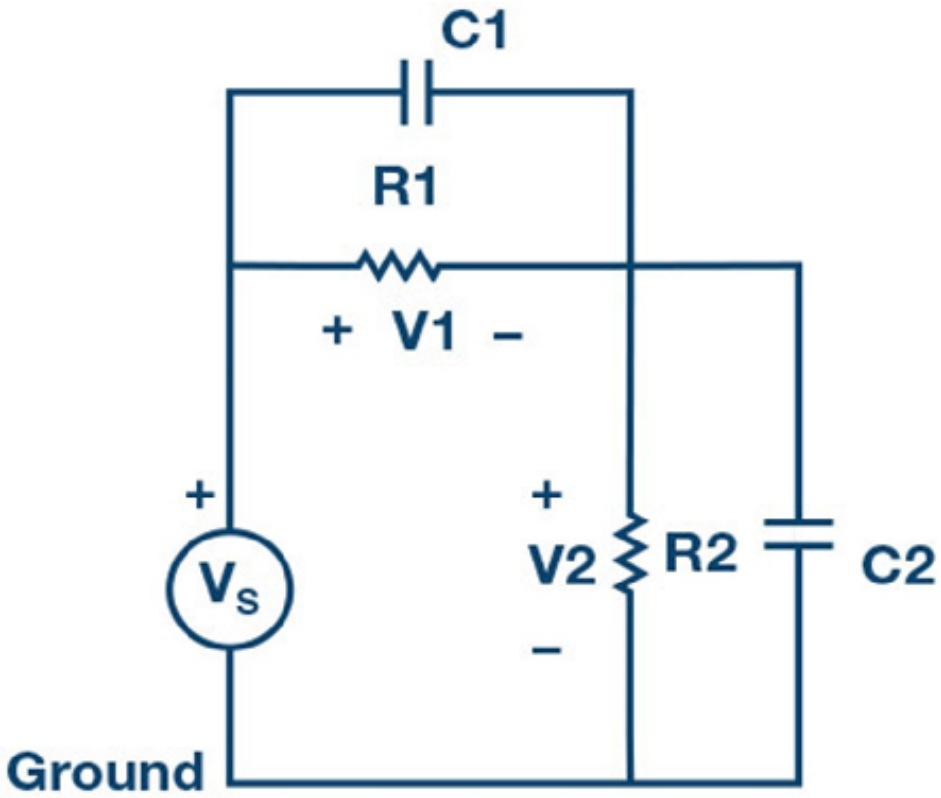
Giao diện gồm: (i) Top Bar với PAUSE, REC, nguồn, zoom và xuất PNG; (ii) Metric Strip hiển thị tốc độ, drops, overflow và cursor; (iii) Channel Card; (iv) waveform cùng thanh pan; (v) DECODE; (vi) Status Bar. Channel Card là khung cuộn gồm 16 hàng; mỗi hàng có màu kênh, tên, biến Boolean điều khiển switch, nhãn giao thức và số cạnh.

DECODE là `CTkTextbox` chỉ đọc và hiển thị kết quả gần nhất từ `det_result`: UART gồm baudrate, lỗi frame, ASCII/hexadecimal; I2C gồm START, STOP, địa chỉ, R/W, ACK/NAK và data; SPI gồm SCK, MOSI, CS và byte; CAN gồm bitrate, ID, RTR, DLC, data và CRC. Detector lấy thêm ngữ cảnh trước mép trái để không cắt mất đầu frame, đồng thời giới hạn số sự kiện trình bày nhằm duy trì thời gian cập nhật.

3.5 Thiết kế và hiện thực hóa mạch bảo vệ và khối thu nhận đầu vào

3.5.1 Cấu trúc đường tín hiệu và các lớp bảo vệ

Front-end được đặt giữa đầu nối probe và các chân nhận của FPGA nhằm mở rộng miền mức cao đo được từ 0,9 V đến 5 V mà không đưa điện áp ngoài trực tiếp vào bank I/O 3,3 V. Trên mỗi kênh, tín hiệu lần lượt đi qua diode ESD TPD1E05U06, bộ phân áp bù tần số, cặp diode kẹp BAT15-04W, điện trở nối tiếp 47 Ω và comparator. Sơ đồ nguyên lý của mạng phân áp bù tần số được trình bày trong Hình 3.6.



Biểu đồ 3.6 Sơ đồ nguyên lý mạch phân áp bù tần số của một kênh đầu vào

TPD1E05U06 được đặt sát đầu nối để dòng ESD có đường xả ngắn xuống mặt phẳng GND trước khi lan vào vùng analog. Linh kiện có điện dung điển hình khoảng 0,5 pF nên tải bổ sung lên đường tín hiệu nhỏ hơn đáng kể so với diode TVS công suất dùng cho nguồn [70]. Sau lớp ESD, tín hiệu đi qua bộ phân áp gồm R1 bằng 1 MΩ, R2 bằng 1,5 MΩ và hai tụ bù C1, C2. Các giá trị tụ được chọn theo điều kiện cân bằng hằng số thời gian đã xác lập tại Mục 2.7, nhờ đó điện áp 5 V được suy giảm về 3 V mà không làm biến dạng đáng kể sườn xung trong dải thiết kế.

Cặp Schottky BAT15-04W được mắc theo cấu hình steering diode từ nút tín hiệu tới GND và rail 3,3 V. Điện dung tiếp giáp nhỏ và điện áp thuận thấp làm linh kiện phù hợp với đường tín hiệu tốc độ cao [71]. Điện trở 47 Ω sau nút kẹp giới hạn dòng tức thời khi diode dẫn, đồng thời làm giảm ringing do chênh lệch trở kháng giữa đường mạch và đầu vào comparator.

Bảng 3.4 Vai trò và yêu cầu bố trí của các phần tử front-end

Phần tử	Chức năng điện	Yêu cầu bố trí PCB
TPD1E05U06	Xả xung ESD từ đầu nối với điện dung tải thấp.	Đặt sát connector; via GND ngắn và không dùng chung cổ chai với đường hồi tín hiệu.
R_1, R_2, C_1, C_2	Suy giảm biên độ theo hệ số 0,6 và cân bằng đáp ứng tần số.	Giữ hai nhánh gọn, pad tụ nhỏ; tránh chạy gần clock hoặc cặp LVDS.
BAT15-04W	Kẹp quá áp còn lại về rail nguồn sạch và GND.	Đặt sát comparator để giảm cảm kháng của vòng dòng kẹp.
Điện trở 47 Ω	Hạn dòng diode, giảm ringing và cách ly điện dung đầu vào.	Đặt ngay trước chân comparator; không tạo nhánh stub tại nút tín hiệu.
Tụ decoupling	Cấp dòng chuyển mạch cục bộ và giảm bounce trên rail 3,3 V.	Mỗi comparator có tụ 100 nF sát chân nguồn; bổ sung tụ khối cho từng cụm kênh.

3.5.2 Phân tích dịch mức DC tại dải đầu vào 0,9 V

Khi kiểm tra kênh đo với xung đầu vào có mức cao 0,9 V, điện áp danh định sau bộ chia hệ số 0,6 chỉ bằng 0,54 V và điện áp tham chiếu tương ứng xấp xỉ 0,27 V. Trong miền điện áp thấp này, dòng rò ngược của cặp diode BAT15-04W trở nên đáng kể so với dòng chạy qua mạng phân áp có trở kháng lớn. Trở kháng tương đương nhìn từ nút đầu vào comparator được xác định bởi

$$R_{eq} = R_1 \parallel R_2 = \frac{1\,\mathrm{M\Omega} \times 1{,}5\,\mathrm{M\Omega}}{1\,\mathrm{M\Omega} + 1{,}5\,\mathrm{M\Omega}} = 600\,\mathrm{k\Omega}. \tag{3.5}$$

Với dòng rò ngược cực đại dùng trong mô phỏng là $I_R = 200$ nA, độ dịch của điểm làm việc tĩnh có độ lớn

$$V_{drift} = I_R R_{eq} = 200\,\mathrm{nA} \times 600\,\mathrm{k\Omega} = 120\,\mathrm{mV}. \tag{3.6}$$

Kết quả này giải thích thành phần dịch mức DC quan sát được khi đo tín hiệu 0,9 V. Dấu và giá trị tức thời của V_{drift} còn phụ thuộc chiều phân cực diode, nhiệt độ và sai số linh kiện, nhưng mô hình cho thấy nguyên nhân chính không xuất phát từ mạch bù tần số. Dịch mức DC làm thay đổi nhẹ thời điểm điện áp đầu vào đi qua ngưỡng so sánh, do đó có thể tạo một sai lệch sườn nhỏ ở miền điện áp thấp. Tuy nhiên, sau khi comparator đã quyết định trạng thái, cặp đầu ra LVDS vẫn chuyển giữa hai trạng thái vi sai xác định; vì vậy hiện tượng này không làm mất tính toàn vẹn logic của dạng sóng đầu ra. Sai lệch tĩnh còn lại có thể được bù bằng mã điện áp tham chiếu của MCP4728 khi hiệu chuẩn từng nhóm kênh.

3.5.3 Kết cuối song song đường truyền LVDS tại FPGA

Đối với 12 kênh sử dụng TLV3801, mỗi tín hiệu được truyền tới FPGA Gowin dưới dạng một cặp LVDS. Điện trở kết cuối song song R3 bằng 100 Ω được mắc trực tiếp giữa hai dây của cặp vi sai tại phía nhận. R3 tạo tải gần bằng trở kháng vi sai danh định 100 Ω của đường truyền, hấp thụ năng lượng của sóng tới và hạn chế phản xạ quay trở lại đầu phát. Nhờ đó, biên độ ringing và nguy cơ xuất hiện nhiều lần vượt ngưỡng tại chân FPGA được giảm xuống.

R3 phải được đặt sát cặp chân LVDS của FPGA Gowin, với hai nhánh từ đường truyền tới hai pad điện trở có chiều dài ngắn và gần đối xứng. Điện trở này là *parallel termination resistor*, không phải điện trở nối tiếp trên từng dây. Đường OUTP và OUTN phải được định tuyến thành cặp vi sai có trở kháng mục tiêu 100 Ω, giữ khoảng cách tương đối ổn định, hạn chế via và không tạo nhánh stub trước điểm kết cuối. Cấu hình R3 chỉ áp dụng cho đầu ra LVDS của TLV3801; các kênh đơn đầu sử dụng TLV3601 không mắc điện trở 100 Ω theo cấu hình này.

3.5.4 Phân bố 12 kênh tốc độ cao và bốn kênh 50 MHz

Mười hai kênh CH0–CH11 sử dụng 12 comparator TLV3801. Mỗi đầu ra là một cặp LVDS vi sai và phải được đưa tới một cặp chân FPGA có hỗ trợ chuẩn LVDS trong cùng bank điện áp; điện trở kết cuối vi sai 100 Ω được đặt tại phía nhận theo cấu hình I/O của FPGA. Bốn kênh CH12–CH15 sử dụng TLV3601 với đầu ra push-pull đơn đầu, nối tới bốn input LVCMOS 3,3 V. Sau input buffer vật lý, mỗi cặp LVDS hoặc đường đơn đầu vẫn được ánh xạ thành đúng một bit của vector `probe[15:0]`, nên datapath RLE không phải thay đổi.

Bảng 3.5 Phân bố kênh, comparator và điện áp tham chiếu

Kênh	Comparator	Đầu ra tới FPGA	Nhóm ngưỡng	Mục tiêu bố trí
CH0–CH3	TLV3801	4 cặp LVDS	V_{REF0}	Cụm tốc độ cao 0; cùng chiều truyền và chiều dài gần bằng nhau.
CH4–CH7	TLV3801	4 cặp LVDS	V_{REF1}	Cụm tốc độ cao 1; tách khỏi đường I2C và ANY-OUT.
CH8–CH11	TLV3801	4 cặp LVDS	V_{REF2}	Cụm tốc độ cao 2; ưu tiên đường hồi GND liên tục.
CH12–CH15	TLV3601	4 đường LVCMOS	V_{REF3}	Cụm 50 MHz; dùng điện trở nối tiếp đầu ra nếu đo thấy ringing.

Bốn điện áp tham chiếu cho phép đồng thời đo bốn họ mức logic khác nhau; mỗi

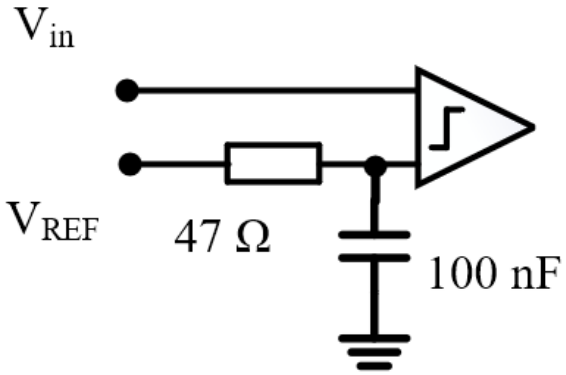
nhóm bốn kênh dùng chung một ngưỡng. Các đường $V_{REF0...3}$ không được chạy song song dài với đầu ra comparator. Cặp LVDS phải được định tuyến vi sai có trở kháng mục tiêu 100 Ω, hạn chế via và không tách qua khe mặt phẳng GND. Sai lệch cố định do comparator và đường mạch có thể được hiệu chuẩn theo từng kênh sau khi hoàn thiện PCB.

Ánh xạ PMOD trong mô hình HIL ở Mục 2.6 là cấu hình nối trực tiếp dùng để kiểm thử datapath. Khi lắp bo front-end TLV3801, constraint chân phải được chuyển sang các cặp LVDS hợp lệ của carrier hoặc adapter; không được giả định mọi cặp chân PMOD đơn đầu hiện tại đều tạo thành cặp LVDS vật lý.

3.6 Triển khai khối tạo điện áp tham chiếu và nguồn kéo lên điều khiển được

3.6.1 Điện áp tham chiếu comparator

MCP4728 được nối với FPGA qua bus I2C và lưu bốn mã điện áp ngưỡng trong EEPROM. Ngay sau khi nguồn 3,3 V ổn định, DAC tự nạp các mã đã lưu ra bốn chân analog mà không phải chờ bitstream hoàn tất khởi động [58]. Mỗi đầu ra DAC đi qua một kênh TLV9064 mắc voltage follower, sau đó qua điện trở 47 Ω tới nút tham chiếu có tụ 100 nF đặt gần cụm comparator. Cấu trúc này bảo vệ DAC trước dòng kick-back và giữ trở kháng nguồn thấp tại ngưỡng so sánh. Mạng RC cách ly tại đầu vào tham chiếu của comparator được thể hiện trong Hình 3.7.

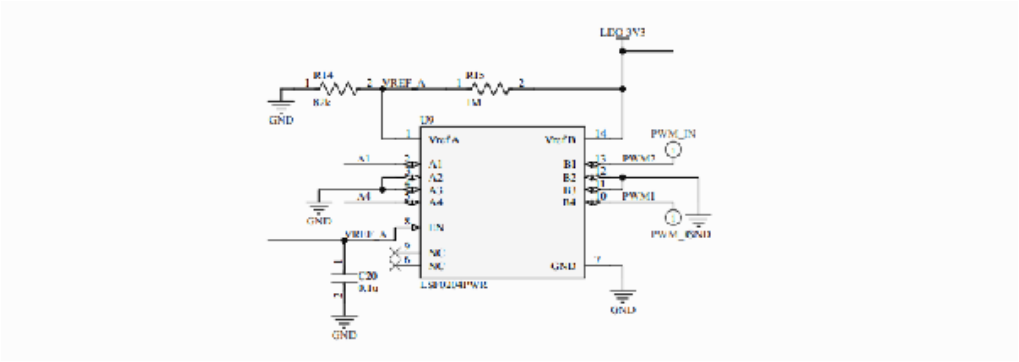


Biểu đồ 3.7 Mạng RC cách ly điện áp tham chiếu tại đầu vào comparator

TLV9064 được cấp từ nguồn sạch 3,3 V của LT3045 và có tụ decoupling riêng. Mặc dù GBW 10 MHz lớn hơn nhiều tần số cắt 33,86 kHz của mạng RC, điện trở cách ly vẫn cần thiết để bảo đảm ổn định khi op-amp nhìn thấy tổng điện dung của tụ 100 nF, đường mạch và nhiều đầu vào comparator.

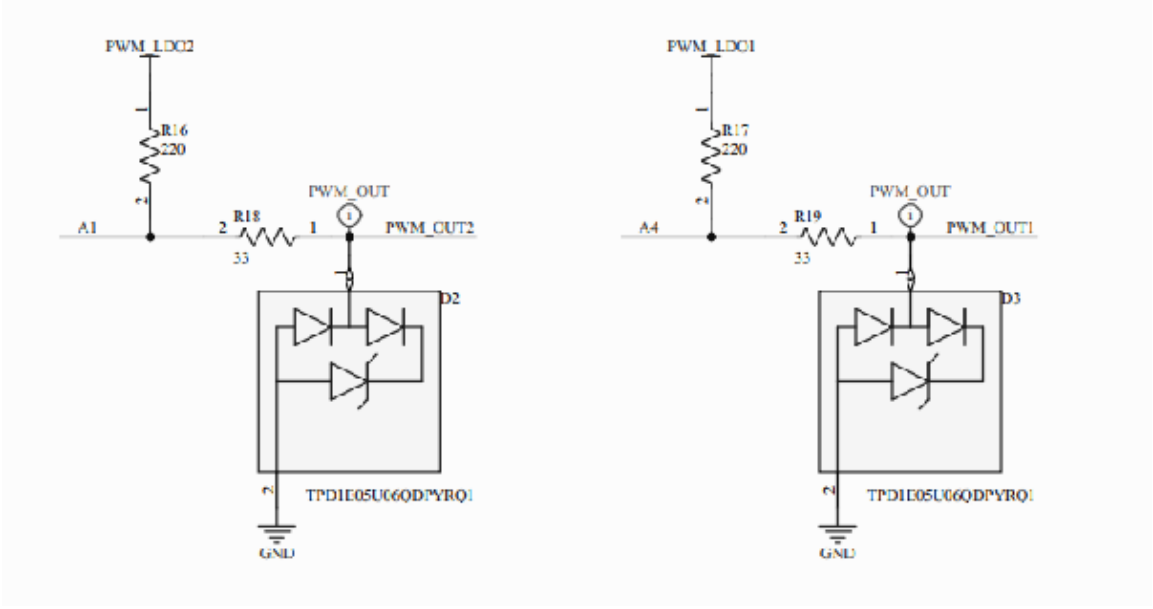
3.6.2 Dịch mức và bảo vệ ngõ ra PWM

Bốn tín hiệu PWM 3,3 V từ FPGA được đưa vào phía B của LSF0204, trong khi phía A sử dụng nguồn kéo lên điều khiển được trong miền 0,9–3,3 V. Điện trở R15 bằng 1 MΩ và R14 được lắp giá trị chuẩn 82 kΩ để thiết lập thiên áp cho chân tham chiếu; lựa chọn này được rút ra từ phép tính tại Mục 2.9. Mỗi kênh sử dụng điện trở kéo lên 220 Ω nhằm bảo đảm tốc độ sườn và giữ dòng dẫn qua MOSFET nội trong giới hạn cho phép. Sơ đồ kết nối LSF0204 được trình bày trong Hình 3.8.



Biểu đồ 3.8 Sơ đồ nguyên lý khối dịch mức PWM sử dụng LSF0204

Sau tầng dịch mức, mỗi đường PWM đi qua điện trở nối tiếp 33 Ω để giảm ringing và giới hạn dòng xung. Diode TPD1E05U06 được đặt sát đầu nối ngõ ra để tạo đường xả ESD ngắn về GND. Hình 3.9 thể hiện hai kênh điển hình; hai kênh còn lại được triển khai đồng nhất để duy trì đặc tính tải và độ trễ giữa các ngõ ra.



Biểu đồ 3.9 Mạch kéo lên, hạn dòng và bảo vệ ESD tại ngõ ra PWM

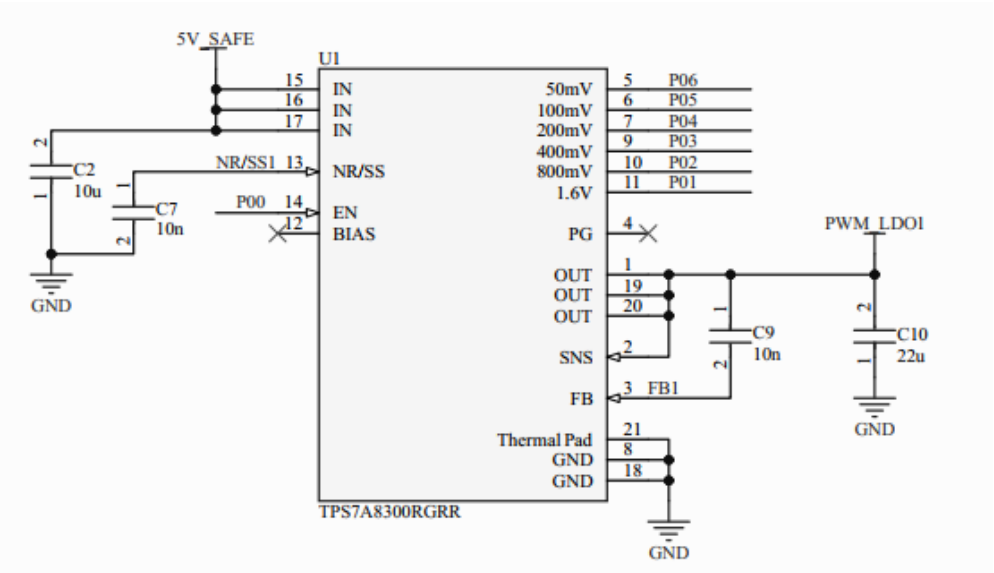
3.6.3 Điều khiển ANY-OUT của TPS7A8300 bằng TCA9535

Nguồn kéo lên của LSF0204 được tạo bởi TPS7A8300. Biến thể ANY-OUT cung cấp miền 0,8–3,95 V với các bước nhị phân 50 mV; trong ứng dụng này phần mềm giới hạn mã để tạo miền 0,9–3,3 V [72]. TCA9535 cung cấp 16 GPIO qua I2C 400 kHz, đủ để điều khiển các chân ANY-OUT và tín hiệu tuần tự nguồn [73].

Chân ANY-OUT được kích hoạt bằng cách kéo xuống GND. Do TCA9535 có cấu trúc đầu ra push-pull, việc đưa GPIO lên mức HIGH 3,3 V sẽ chủ động bơm dòng vào nút ANY-OUT và không phù hợp với cơ chế chọn điện áp bằng nối GND của TPS7A8300. Vì vậy firmware chỉ sử dụng hai trạng thái điều khiển. Khi cần chọn một trọng số điện áp, chân TCA9535 được cấu hình là output và thanh ghi dữ liệu tương ứng được đặt ở mức LOW. Khi cần nhả trọng số, chân được cấu hình lại thành input để tạo trạng thái trở kháng cao (Hi-Z), tuyệt đối không dùng output HIGH làm trạng thái "không chọn".

Để tránh xung mức HIGH ngắn trong lúc chuyển hướng GPIO, firmware ghi trước mức LOW vào output latch, sau đó mới chuyển bit direction từ input sang output. Theo chiều ngược lại, bit direction được chuyển về input trước khi cập nhật mã điện áp tiếp theo. Các I/O TCA9535 mặc định là input sau power-on, vì vậy toàn bộ bit ANY-OUT ở trạng thái nhả khi mới cấp nguồn; TPS7A8300 vẫn cần điện trở phần cứng đặt EN ở trạng thái an toàn cho tới khi FPGA hoàn tất cấu hình.

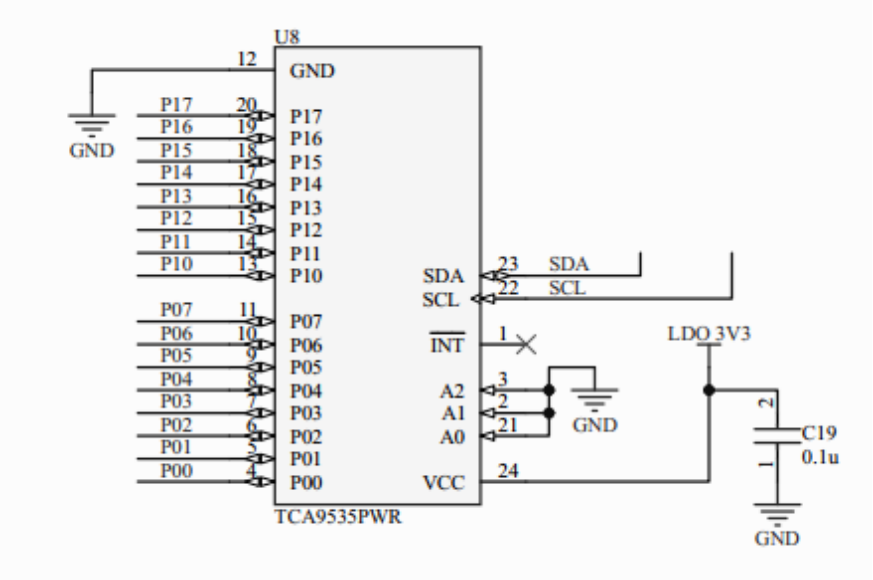
Sơ đồ nguyên lý TPS7A8300 trong Hình 3.10 cho thấy các chân ANY-OUT được đưa ra thành bus cấu hình và đường EN được điều khiển độc lập. Tụ đầu vào, tụ đầu ra và tụ giảm nhiễu phải đặt sát chân IC, còn đường nguồn kéo lên tới LSF0204 được đi rộng và tránh chạy song song với các đường tham chiếu comparator.



Biểu đồ 3.10 Sơ đồ nguyên lý nguồn kéo lên điều khiển được TPS7A8300

TCA9535 trong Hình 3.11 nhận hai đường SCL và SDA từ FPGA, sau đó phân

phối các tín hiệu chọn mức, EN và điều khiển transistor xả. Các điện trở kéo lên bus I2C được nối về miền 3,3 V; các chân địa chỉ được cố định bằng phần cứng để tránh thay đổi địa chỉ do nhiễu lúc khởi động.



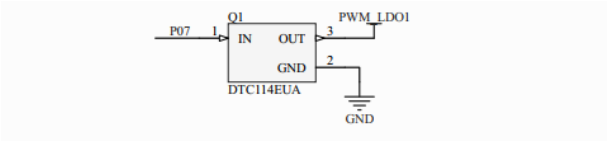
Biểu đồ 3.11 Sơ đồ nguyên lý bộ mở rộng I2C TCA9535

Bảng 3.6 Quy ước điều khiển TPS7A8300 bằng TCA9535

Tín hiệu	Trạng thái chọn	Trạng thái nhả	Chức năng
ANY-OUT 50–3200 mV	GPIO output LOW	GPIO input Hi-Z	Cộng trọng số điện áp tương ứng vào mức cơ sở 0,8 V.
EN của LDO	Mức logic cho phép	Mức vô hiệu hóa	Tách nguồn kéo lên trước khi đổi mã điện áp.
P07 điều khiển DTC114E	GPIO output HIGH	GPIO output LOW	Dẫn transistor để xả tụ hoặc ngắt đường xả.
POWER-GOOD	Input	Input	Xác nhận rail mới đã ổn định trước khi bật PWM.

3.6.4 Mạch xả nhanh bằng transistor số DTC114E

Khi EN của LDO bị ngắt, tụ decoupling vẫn giữ điện tích của mức điện áp trước đó. Nếu mã ANY-OUT được đổi từ mức cao xuống mức thấp trong khi tụ chưa xả, rail kéo lên tiếp tục nằm trên giá trị cũ và tạo xung PWM sai biên độ. DTC114E là transistor NPN tích hợp điện trở đầu vào và điện trở base–emitter, chịu dòng collector tới 100 mA theo biến thể đóng gói [74]. Collector nối với rail cần xả, emitter nối GND, còn input do TCA9535 điều khiển như Hình 3.12. Khi transistor tắt, không tồn tại điện trở xả thường trực nên không gây tổn hao dòng tĩnh như phương án dùng điện trở xả cố định.



Biểu đồ 3.12 Mạch transistor số DTC114E xả nhanh tụ nguồn kéo lên

Thời gian chờ xả được firmware đặt theo điện dung đầu ra và trở kháng thực tế của đường xả, sau đó rail phải giảm dưới ngưỡng an toàn trước khi mã ANY-OUT mới được ghi. Điện trở giới hạn dòng được lựa chọn để dòng xả không vượt định mức DTC114E và khả năng chịu dòng xung của tụ. Trong mạch điều khiển, P07 của TCA9535 được dành riêng cho chức năng này: P07 ở mức HIGH làm DTC114E dẫn, còn P07 ở mức LOW khóa transistor và tách đường xả khỏi rail kéo lên.

Bảng 3.7 Trình tự đổi điện áp nguồn kéo lên PWM

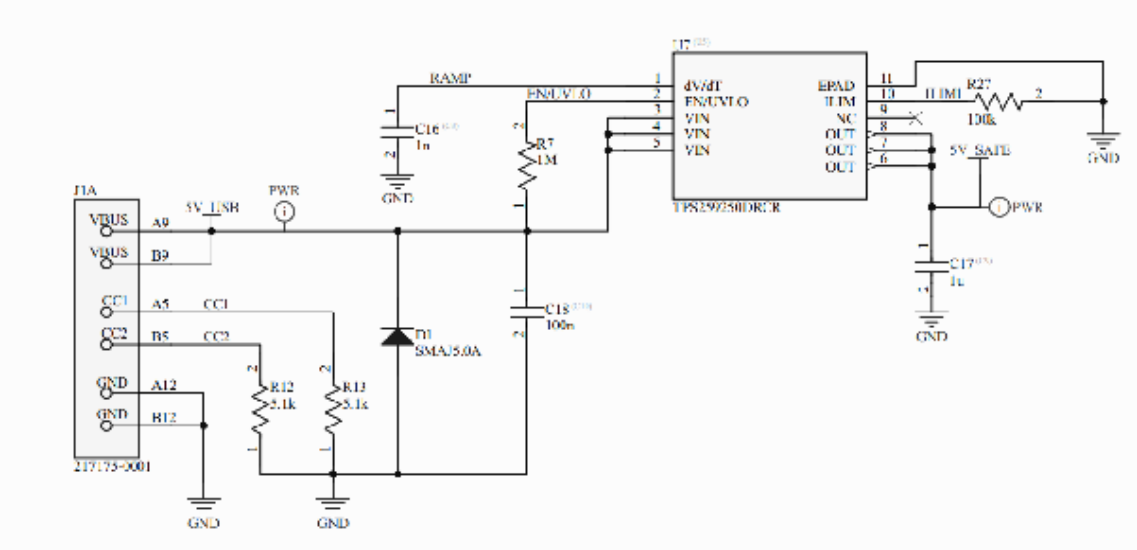
Pha	Trạng thái điều khiển	Mục đích
1	Vô hiệu hóa PWM trên FPGA và đưa EN của LSF0204 xuống LOW.	Cách ly hoàn toàn ngõ ra khỏi tải và ngăn xung không xác định.
2	Đưa EN của TPS7A8300 xuống LOW.	Ngắt nguồn dòng chính đang nạp cho tụ đầu ra.
3	Đưa P07 của TCA9535 lên HIGH và chờ rail giảm dưới ngưỡng an toàn.	Mở DTC114E để xả nhanh năng lượng còn lại trong các tụ decoupling xuống GND.
4	Đưa P07 xuống LOW; cấu hình lại các chân ANY-OUT bằng hai trạng thái input Hi-Z hoặc output LOW.	Khóa đường xả và thiết lập mã điện áp mới mà không lái mức HIGH vào chân ANY-OUT.
5	Đưa EN của TPS7A8300 lên HIGH, đợi POWER-GOOD ổn định, sau đó bật lại LSF0204 và PWM.	Chỉ nối tải và phát xung khi rail kéo lên đã đạt điện áp mục tiêu.

Trình tự trên phải được thực hiện như một giao dịch nguyên tử trong firmware điều khiển. Nếu mất kết nối I2C hoặc reset FPGA giữa chừng, điện trở định thiên phần cứng phải đưa LDO, LSF0204 và DTC114E về trạng thái không gây ngắn mạch.

3.7 Triển khai khối bảo vệ nguồn đầu vào USB-C

3.7.1 Cây nguồn và bảo vệ ESD

LA16 nhận 5 V từ cổng USB-C. Hai chân CC1 và CC2 dùng điện trở kéo xuống 5,1 kΩ để công bố thiết bị tiêu thụ nguồn. Tại nút VBUS, SMAJ5.0A tạo đường xả ESD và surge xuống GND [75]. TVS phải được đặt sát connector, với đường từ VBUS qua TVS xuống mặt phẳng GND ngắn và rộng; nếu đặt sau đường mạch dài, cảm kháng ký sinh làm tăng điện áp kẹp tại eFuse.



Biểu đồ 3.13 Sơ đồ bảo vệ đầu vào USB-C bằng SMAJ5.0A và TPS259250

SMAJ5.0A có công suất xung cao hơn diode ESD siêu nhỏ, phù hợp nút nguồn nhưng có điện dung lớn nên không được dùng trên probe tốc độ cao. TVS là lớp hấp thụ xung đầu tiên; eFuse ở sau chịu trách nhiệm giới hạn năng lượng kéo dài do ngắn mạch, quá dòng hoặc nguồn vào bất thường.

3.7.2 Giới hạn dòng bằng eFuse TPS259250

TPS259250 tích hợp MOSFET công suất, giới hạn dòng có thể lập trình, soft-start và bảo vệ nhiệt. So với PTC, eFuse phản ứng nhanh hơn và có ngưỡng dòng xác định; so với cầu chì đứt, mạch có thể phục hồi sau khi nguyên nhân lỗi được loại bỏ. Dòng giới hạn mục tiêu của hệ thống được chọn bằng 3,7 A nhằm cho phép toàn bộ mạch khởi động bình thường nhưng vẫn ngắt đường 5 V trước các sự cố quá dòng kéo dài.

Trong tài liệu tính toán ban đầu, biểu thức cấu hình chân ILIM được ghi dưới dạng

$$R_{ILIM}^{(\text{initial})} = \frac{I_{LIM} - 0{,}73}{10^{-5}} = \frac{3{,}7 - 0{,}73}{10^{-5}} = 297\,\text{k}\Omega. \tag{3.7}$$

Tuy nhiên, kết quả 297 kΩ xuất phát từ việc chép thiếu hệ số 3 trong mẫu số và không phù hợp với phương trình lựa chọn R_{ILIM} của TPS259250. Quan hệ được nhà sản xuất

sử dụng cho thiết kế danh định là

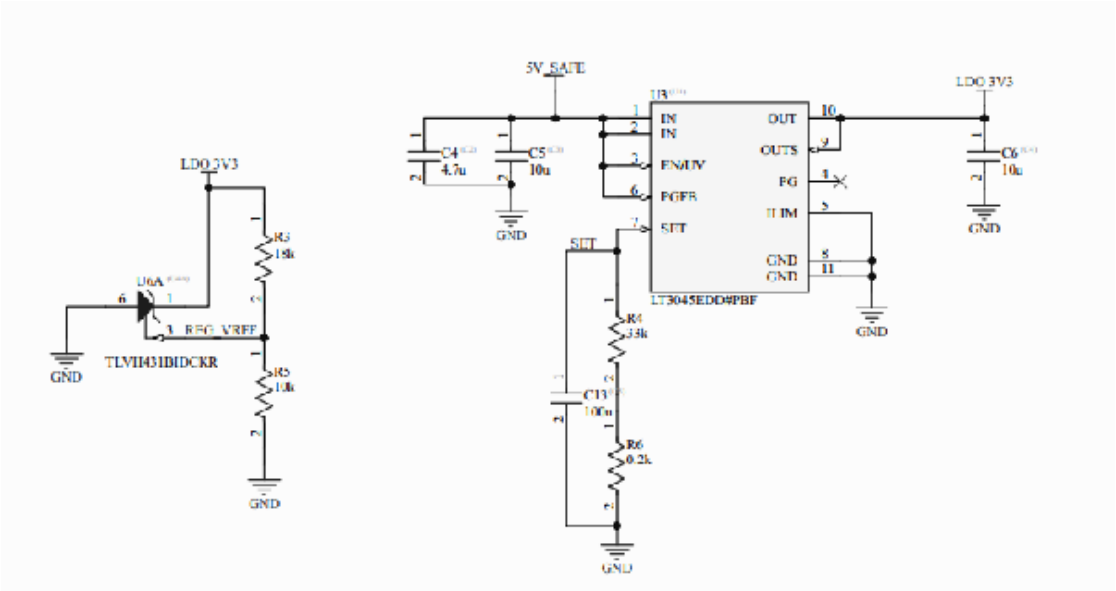
$$R_{ILIM} = \frac{I_{LIM} - 0{,}7}{3 \times 10^{-5}} = \frac{3{,}7 - 0{,}7}{3 \times 10^{-5}} = 100\,\text{k}\Omega. \tag{3.8}$$

Vì vậy điện trở thực tế tại chân ILIM được lắp bằng 100 kΩ, dung sai 1%. Giá trị này vừa tương ứng trực tiếp với ví dụ thiết kế 3,7 A của TPS259250, vừa tạo khoảng dự trữ so với khả năng cấp dòng cực đại 5 A của họ linh kiện. Do ngưỡng giới hạn còn phụ thuộc sai số nội tại của IC và nhiệt độ, mức đo thực tế quanh 3,7–3,75 A được xem là đạt yêu cầu; kết quả hiệu chuẩn phải được ghi ở Chương 4 thay vì suy ra chỉ từ dung sai điện trở [76].

3.7.3 Nguồn 3,3 V nhiễu thấp bằng LT3045

Sau eFuse, LT3045 chuyển đổi 5 V thành 3,3 V cho comparator, MCP4728 và TLV9064. Linh kiện cung cấp dòng tối đa 500 mA, nhiễu tích phân khoảng 0,8 μV_{RMS} và PSRR 76 dB tại 1 MHz trong điều kiện đo của datasheet [77]. Nguồn sạch làm giảm biến thiên ngưỡng comparator và hạn chế jitter do nhiễu nguồn điều chế vào tầng vào/ra tốc độ cao.

Điện trở SET được lắp giá trị 33 kΩ để tạo mức 3,3 V theo dòng SET danh định của LT3045. Tụ SET và tụ đầu ra được chọn theo datasheet, đặt sát IC và nối về điểm GND yên tĩnh. Với điều kiện tải cực đại 500 mA, công suất nhiệt xấu nhất của LDO vào khoảng 0,85 W; vì vậy PCB phải có vùng đồng tản nhiệt và nhiều thermal via, đồng thời dòng thực tế của toàn bộ comparator, DAC và op-amp phải được kiểm tra để không vượt giới hạn nhiệt. Sơ đồ LT3045 và tầng kẹp TLVH431 được thể hiện trong Hình 3.14.



Biểu đồ 3.14 Sơ đồ nguồn 3,3 V nhiễu thấp LT3045 và mạch kẹp TLVH431

3.7.4 Kẹp dòng dội ngược bằng TLVH431

Khi nhiều đầu vào cùng quá áp, các diode steering BAT15-04W có thể đồng thời bơm dòng về rail 3,3 V. Dòng dội ngược này sạc các tụ decoupling ở đầu ra LT3045 và làm điện áp rail tăng theo quan hệ $\Delta V = I\Delta t/C$. LT3045 điều chỉnh nguồn theo chiều cấp dòng cho tải nhưng không được dùng như một phần tử shunt để hấp thụ liên tục dòng trả về từ các kênh probe. Nếu không có đường tiêu tán bổ sung, rail 3,3 V có thể vượt miền an toàn của các comparator tốc độ cao.

TLVH431 được mắc song song với đầu ra LT3045 để tạo mạch kẹp quá áp tĩnh cục bộ. Khi điện áp rail thấp hơn ngưỡng đặt, TLVH431 gần như không dẫn và không ảnh hưởng đến chế độ nguồn bình thường. Khi điện áp vượt ngưỡng, dòng cathode tăng và dẫn phần dòng dội ngược xuống GND. Hai điện trở $R_3 = 18$ kΩ và $R_5 = 10$ kΩ trong mạng hồi tiếp của TLVH431 thiết lập điện áp kẹp

$$V_{clamp} = V_{REF}\left(1 + \frac{R_3}{R_5}\right) = 1{,}24\left(1 + \frac{18}{10}\right) = 3{,}472\,\text{V}. \tag{3.9}$$

Mức 3,472 V cao hơn rail danh định 3,3 V nên tránh làm TLVH431 dẫn đáng kể trong điều kiện bình thường, đồng thời thấp hơn giới hạn đánh giá 3,6 V, tương ứng $V_{CC} + 0{,}3$ V của các linh kiện nhạy cảm. Ký hiệu R_3 trong mạch TLVH431 thuộc mạng chia áp nguồn và độc lập với điện trở R3 bằng 100 Ω dùng để kết cuối LVDS ở khối thu nhận đầu vào.

Điện trở hoặc phần tử giới hạn dòng cathode phải được chọn sao cho dòng shunt và công suất tiêu tán không vượt định mức TLVH431 trong trường hợp nhiều kênh cùng dội dòng. Mạch kẹp này xử lý quá áp tĩnh cục bộ trên rail 3,3 V; nó không thay thế eFuse TPS259250, TVS tại USB-C hoặc điện trở hạn dòng trên từng đầu probe [78].

Kết chương

Chương 3 đã trình bày đường triển khai hoàn chỉnh từ đầu nối probe đến Host PRISM. Front-end sử dụng TPD1E05U06, bộ phân áp bù tần số, BAT15-04W và comparator để bảo vệ, suy giảm và chuẩn hóa 16 kênh; MCP4728 cùng TLV9064 tạo bốn ngưỡng tĩnh có bộ đệm. LSF0204, TPS7A8300, TCA9535 và DTC114E cung cấp miền PWM 0,9–3,3 V với trình tự đổi áp an toàn. Cây nguồn USB-C được bảo vệ bởi SMAJ5.0A, TPS259250, LT3045 và TLVH431. Trong FPGA, pipeline ba tầng, bộ đếm tách đôi, FIFO Gray-pointer và Block RAM giúp phần cứng đáp ứng 175,5 MHz; packetizer cung cấp sequence, overflow flags, flush và heartbeat. Phần mềm sử dụng multi-threading, Ring Buffer thread-safe, thời gian số nguyên và heuristic algorithm để thu liên tục và giải mã UART, I2C, SPI, CAN. Chương 4 tiếp theo sẽ kiểm thử timing, độ chính xác decoder, mất gói và giới hạn vận hành của hệ thống.

CHƯƠNG 4. KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

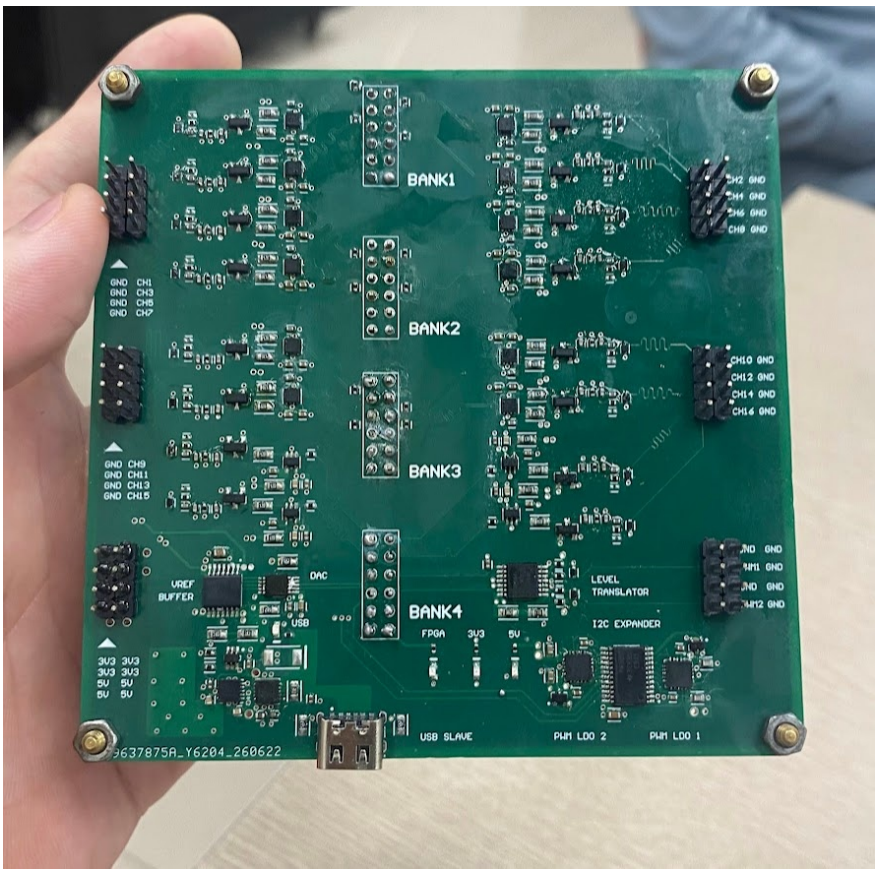
Tổng quan chương

Chương 3 đã trình bày kiến trúc LA16, gồm khối lấy mẫu 16 kênh, mã hóa theo biến đổi trạng thái, FIFO bất đồng bộ, đường truyền UDP và phần mềm PRISM. Chương này chuẩn hóa quy trình đánh giá theo V-Model cải tiến cho hệ thống nhúng, sau đó đánh giá kết quả triển khai ở ba mức: tài nguyên và timing FPGA, độ chính xác giải mã giao thức, và khả năng làm việc của toàn chuỗi đo. Nội dung lần lượt phân tích nguồn phát I2S0–DMA trên ESP32, báo cáo PnR Gowin, thiết kế và kết quả của chín kịch bản TC0–TC8, đồng thời trình bày mô hình Software-in-the-Loop (SIL) dùng để kiểm tra decoder khi chưa cần phần cứng vật lý.

4.1 Chuẩn hóa kiểm thử theo V-Model cho hệ thống nhúng

4.1.1 Mạch phần cứng thực tế được đưa vào kiểm thử HIL

Đối tượng của quá trình kiểm thử Hardware-in-the-Loop không chỉ gồm FPGA và phần mềm PRISM mà còn bao gồm bo mạch front-end đã được lắp ráp thực tế trong Hình 4.1. Bo mạch tiếp nhận 16 tín hiệu cần đo, thực hiện bảo vệ và chuẩn hóa mức điện áp trước khi chuyển các tín hiệu logic tới Tang Primer 20K. Việc đưa bo mạch thật vào vòng kiểm thử cho phép đánh giá đồng thời đường tín hiệu, kết nối đất, sai khác giữa các kênh và hoạt động của các khối điều khiển phụ trợ, thay vì chỉ xác minh RTL bằng dữ liệu mô phỏng.



Biểu đồ 4.1 Bo mạch front-end LA16 sau khi lắp ráp linh kiện

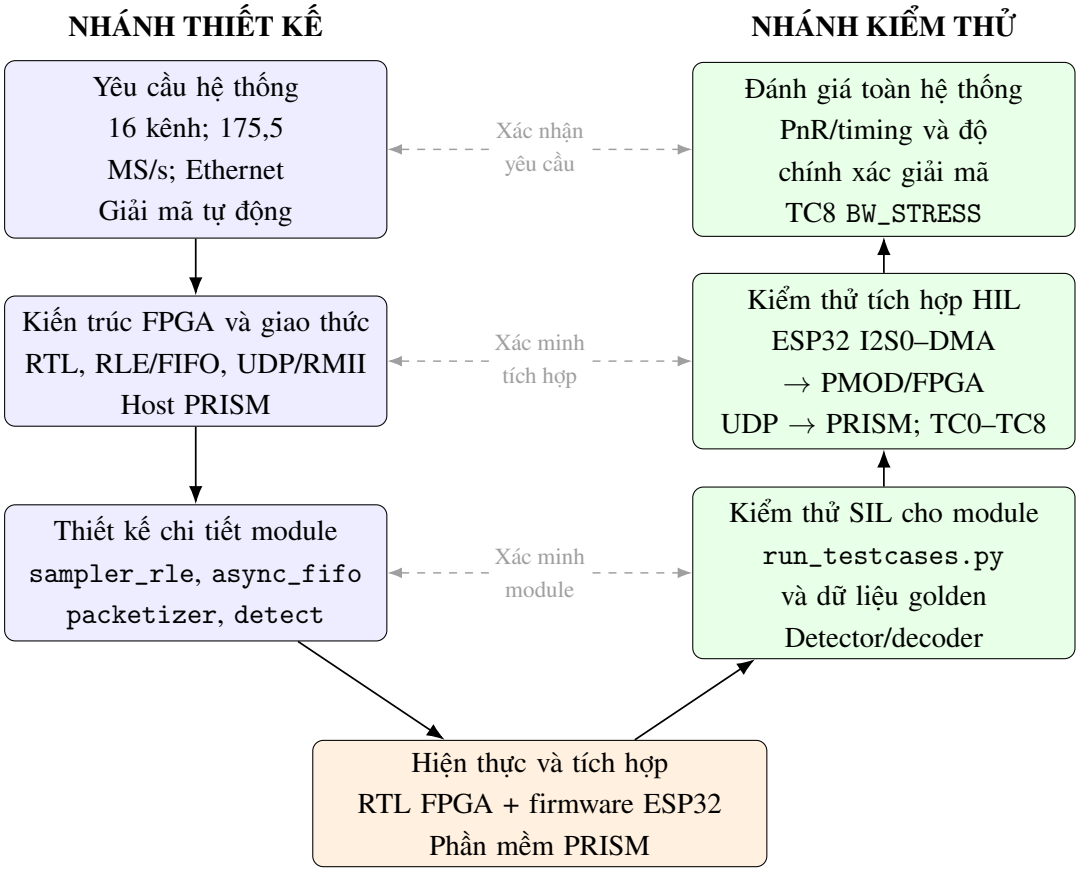
Trên bo mạch, 16 kênh được bố trí đối xứng thành bốn nhóm BANK1–BANK4. Các đầu nối ở hai cạnh trái và phải đưa từng kênh CH0–CH15 đi kèm chân GND, nhờ đó dây đo có đường hồi dòng gần tín hiệu và giảm vòng dòng lớn trong quá trình thử nghiệm. Mỗi kênh sử dụng một chuỗi linh kiện lặp lại gồm mạng bảo vệ, suy giảm và chuẩn hóa đầu vào; cách bố trí tương đồng giữa các kênh giúp hạn chế sai khác ký sinh và tạo điều kiện so sánh timing đa kênh.

Khu vực phía dưới bo mạch tập trung các khối dùng chung. Cụm `DAC` và `VREF BUFFER` tạo, đệm các mức ngưỡng cho front-end; cụm `LEVEL TRANSLATOR` và `I2C EXPANDER` thực hiện chuyển mức và điều khiển các đường cấu hình. Hai khối `PWM LDO 1` và `PWM LDO 2` cung cấp miền điện áp kéo lên có thể lựa chọn cho phép thử PWM, còn cổng `USB SLAVE` cùng các đầu 3,3 V/5 V phục vụ cấp nguồn và kết nối điều khiển. Các header BANK ở giữa bo tạo giao diện gọn để chuyển các tín hiệu đã chuẩn hóa tới khối FPGA.

Ảnh lắp ráp xác nhận các khối chức năng đã được hiện thực trên PCB và các nhóm kênh có thể được đấu nối vào vòng HIL. Tuy nhiên, ảnh ngoại quan không tự chứng minh băng thông hoặc độ chính xác điện áp; các đặc tính đó phải được kết luận từ dạng sóng, kết quả giải mã, kiểm thử timing và các kịch bản TC0–TC8 ở những phần tiếp theo.

4.1.2 Ánh xạ thiết kế và kiểm thử theo V-Model

V-Model truyền thống được điều chỉnh cho LA16 nhằm phản ánh đồng thời phần cứng FPGA, firmware tạo tín hiệu, giao thức Ethernet và phần mềm Host. Nhánh trái phân rã yêu cầu thành kiến trúc và module chi tiết; nhánh phải tích hợp dần các bằng chứng kiểm thử từ thuật toán độc lập đến toàn bộ chuỗi vật lý. Tính đối xứng giữa hai nhánh được thể hiện trong Biểu đồ 4.2.



Biểu đồ 4.2 V-Model cải tiến cho quá trình thiết kế và kiểm thử hệ thống nhúng LA16

Ba cặp đối xứng cung cấp ba mức bằng chứng. Thứ nhất, thiết kế chi tiết của `sampler_rle` và `detect` được đối chiếu với SIL bằng `run_testcases.py`; dữ liệu đầu vào được lượng tử hóa theo clock 175,5 MHz nhưng không phụ thuộc kết nối vật lý. Thứ hai, kiến trúc FPGA–UDP–Host được đối chiếu với HIL: ESP32 phát đồng thời 16 mức logic qua I2S0 LCD-Parallel và DMA, tín hiệu đi qua dây PMOD, input buffer, datapath FPGA, PHY Ethernet và PRISM. Thứ ba, yêu cầu hệ thống được xác nhận bằng kết quả PnR, độ chính xác giải mã của TC0–TC7 và thử tải TC8 `BW_STRESS`. Cách tổ chức này phân biệt rõ lỗi thuật toán, lỗi tích hợp và giới hạn hiệu năng hệ thống.

4.2 Giải pháp tạo tín hiệu kiểm thử không jitter trên ESP32

Trong phương án truyền thống, xung được tạo bằng LEDC còn UART được phát bằng `SoftwareSerial` hoặc thao tác GPIO. Cách này phù hợp với ít kênh nhưng không đáp ứng tốt 16 tín hiệu độc lập: số timer hữu hạn, nhiều kênh phải chia sẻ cấu hình, còn bit-banging phụ thuộc ngắt và lịch thực thi CPU. Các chân vì thế không đổi trạng thái tại cùng một mốc clock và xuất hiện jitter tương đối, làm giảm độ tin cậy khi đánh giá pha và giải mã đa kênh.

Firmware `la16_testgen` khắc phục hạn chế trên bằng cách cấu hình ngoại vi I2S0 ở chế độ LCD-Parallel 16 bit. Một phần tử `uint16_t` trong RAM biểu diễn trạng thái logic của cả 16 kênh tại cùng một thời điểm, trong đó bit thứ *i* tương ứng với kênh CH*i*. Các hàm sinh UART, I2C, SPI, CAN và CLK/PWM không trực tiếp điều khiển GPIO mà "vẽ" trước dạng sóng vào mảng dữ liệu này. Vì vậy, mọi quan hệ pha, độ rộng xung và thứ tự cạnh đều được xác định trước khi quá trình phát bắt đầu.

Mảng mẫu được chia thành các descriptor DMA không quá 4092 byte và nối vòng. DMA tự đọc từng từ 16 bit để đưa ra `I2S0O_DATA_OUT8..23`; CPU chỉ chọn test case hoặc thay đổi cấu hình. Cả 16 kênh vì vậy cập nhật trên cùng sườn clock I2S0, loại bỏ jitter do lịch phần mềm và giải phóng CPU. "Không jitter" ở đây được hiểu là không có jitter tương đối do phần mềm giữa các kênh; sai số tuyệt đối còn lại phụ thuộc clock nguồn và bộ chia I2S.

Do FIFO 16 bit của I2S0 có thể hoán đổi từng cặp từ, firmware cung cấp cờ pair-swap và hệ số tinh chỉnh clock. TC0 xác định các tham số này: PRISM phải đo được 10 kHz trên CH0 và giải mã đúng `LA16-CAL` ở 9600 bps trên CH1. Khi hai điều kiện cùng thỏa mãn, TC1–TC8 sử dụng time base và thứ tự bit đã hiệu chuẩn.

4.3 Kết quả tài nguyên tổng hợp phần cứng trên FPGA

Thiết kế được PnR cho FPGA Gowin GW2A-18C trên Tang Primer 20K. Bảng 4.1 lấy số liệu từ `la16.rpt.txt`. Gowin dùng *Logic* cho tổng tài nguyên đã ánh xạ, nên 4078 không phải toàn bộ là LUT thuần; nhóm LUT/ALU/ROM16 gồm 2970 LUT và 1090 ALU.

Bảng 4.1 Kết quả sử dụng tài nguyên sau PnR của LA16

Tài nguyên	Sử dụng	Tỉ lệ	Vai trò chính
Logic	4078/20736	20%	Lấy mẫu, RLE, packetizer, UDP
LUT thuần	2970	–	Logic tổ hợp
ALU	1090	–	Bộ đếm và số học
Thanh ghi	1648/16173	11%	Pipeline và đồng bộ clock
CLS	2489/10368	24%	Cụm logic đã bố trí
BSRAM	13/46	29%	FIFO bất đồng bộ
Cổng I/O	32/207	16%	Probe, RMII và tín hiệu điều khiển
rPLL	1/4	25%	Tạo clock lấy mẫu 175,5 MHz

Mức sử dụng Logic xấp xỉ 20% cho thấy thiết kế vẫn còn khoảng dự trữ lớn cho các chức năng mở rộng như trigger phần cứng, bộ lọc đầu vào hoặc bộ đệm bắt mẫu sâu hơn. Khối chiếm tỉ lệ tương đối cao nhất là BSRAM với 13/46 khối, tương đương 29%. Đây là lựa chọn có chủ đích vì FIFO bất đồng bộ phải hấp thụ các burst dữ liệu từ miền lấy mẫu 175,5 MHz trước khi packetizer đọc dữ liệu ở miền RMII 50 MHz. Việc sử dụng BSRAM thay cho LUT RAM đồng thời hạn chế áp lực lên logic tổ hợp và cải thiện khả năng định tuyến.

rPLL nhân clock 27 MHz theo hệ số $13/2$ để tạo tần số lấy mẫu 175,5 MHz. Chu kỳ lấy mẫu tương ứng là

$$T_s = \frac{1}{175{,}5 \times 10^6} = 5{,}698 \text{ ns} \approx 5{,}70 \text{ ns}. \tag{4.10}$$

Báo cáo timing sau PnR cho biết Fmax của miền `clk_sample` đạt 182,142 MHz, cao hơn ràng buộc 175,5 MHz. Total Negative Slack bằng 0 trên các miền clock và đường setup xấu nhất còn slack dương 0,208 ns. Như vậy, thiết kế đã đóng timing tại tần số mục tiêu; mỗi cạnh hợp lệ được lượng tử hóa với độ phân giải 5,70 ns và quan hệ thời gian giữa 16 kênh được bảo toàn trong cùng miền clock lấy mẫu.

4.4 Thiết kế và kết quả các kịch bản kiểm thử

Theo nhánh kiểm thử tích hợp của V-Model, các test case được sắp xếp từ hiệu chuẩn, kiểm thử từng giao thức, kiểm thử tích hợp đến thử biên và thử quá tải. ESP32 phát lặp dạng sóng đã dựng sẵn qua 16 dây dữ liệu và mass chung đến Tang Primer 20K. FPGA tạo record {`delta`, `value`}, đệm qua FIFO rồi gửi UDP. Với mỗi test case, kết quả được trình bày trước tại waveform, bảng CHANNELS và khung DECODE của PRISM, sau đó mới đánh giá mức độ đạt yêu cầu. Các đoạn mã dưới đây được trích từ hàm `paint_build()` trong firmware `la16_paint.cpp`.

Trước khi phân tích riêng từng kịch bản, Bảng 4.2 tổng hợp phạm vi tín hiệu, vai trò và kết quả chính của toàn bộ thang kiểm thử TC0–TC8.

Bảng 4.2 Tổng hợp thiết kế, vai trò và kết quả các kịch bản TC0–TC8

TC	Tên	Cấu hình tín hiệu	Vai trò	Kết quả chính
0	CALIBRATION	CH0: CLK 10 kHz; CH1: UART 9600 bps, chuỗi `LA16-CAL`.	Hiệu chuẩn time base, ánh xạ bit và pair-swap của I2S0.	Ảnh đo đúng 10 kHz và 9600 bps; cửa sổ chỉ chứa một đoạn payload UART.
1	UART_SWEEP	16 UART tại tám mức 4800–230400 bps; mỗi mức xuất hiện trên hai kênh.	Kiểm tra auto-baud và giải mã nhiều UART đồng thời.	Ảnh nhận 13/16 kênh là UART; ba kênh tốc độ thấp còn là DATA. SIL đạt 48/48.
2	I2C_SWEEP	Bốn bus I2C tại 100 kHz, 400 kHz, 1 MHz và 200 kHz.	Kiểm tra START/STOP, ACK, payload và Fast-mode Plus.	Ảnh ghép được hai bus nhưng tần số và dữ liệu còn sai lệch; SIL đạt 13/13.
3	SPI_SWEEP	Ba bus SPI mode 0 tại 1, 5 và 10 MHz; dữ liệu `A5 5A 3C`.	Đánh giá bắt cạnh và giải mã SPI tốc độ cao.	Waveform được thu nhưng PRISM ghép nhầm thành I2C; SIL đạt 7/7.
4	CAN_SWEEP	Năm CAN classic tại 125, 250, 500, 800 kbps và 1 Mbps.	Kiểm tra đồng bộ bit, destuffing, ID, payload và CRC15.	Ảnh xác nhận 3/5 kênh tốc độ cao với CRC đúng; SIL đạt 25/25.

Bảng 4.2 (tiếp theo)

TC	Tên	Cấu hình tín hiệu	Vai trò	Kết quả chính
5	CLK_SWEEP	16 CLK/PWM từ 1–20 kHz; duty 10%, 25%, 50%, 75% và 90%.	Kiểm tra phép đo tần số, duty cycle và phân loại CLK/PWM.	HIL nhận đúng 12 CLK và 4 PWM; SIL đạt 36/36 tiêu chí.
6	MIX_ALL	UART, I2C, SPI, CAN và CLK/PWM cùng hoạt động trên 16 kênh.	Đánh giá auto-detect và tải xử lý trong phép đo hỗn hợp.	Ảnh nhận được UART, I2C và CLK; SPI/CAN còn là DATA. SIL đạt 10/10.
7	EDGE_CASES	Xung 150 ns, UART 250000 bps và 8E1, SPI mode 2, CAN stuffing, I2C NAK.	Xác định giới hạn lấy mẫu và phạm vi hỗ trợ của decoder.	Bắt được xung và UART 250000; các cấu hình ngoài phạm vi bị nhận sai như dự kiến.
8	BW_STRESS	16 CLK 50 kHz lệch pha đều, tạo khoảng 1,6 triệu cạnh/s.	Xác định giới hạn RLE–FIFO–Ethernet khi tải cực đại.	Dạng sóng đúng 50,1 kHz nhưng 12 kênh bị nhận thành UART 100000 bps.

Các ảnh PRISM sử dụng cửa sổ quan sát khác nhau. Giá trị `SAMPLE RATE` được GUI làm tròn thành 176 MS/s, tương ứng clock phần cứng 175,5 MHz. Hai chỉ báo `DROPS` và `FIFO OVF ever` có tính tích lũy trong toàn phiên chạy, do đó không thể tự chúng chứng minh lỗi phát sinh trong test case đang hiển thị. Chỉ báo `RECENT!` cho biết overflow vừa xảy ra gần cửa sổ hiện tại. Khi đối chiếu kết quả, báo cáo vì vậy ưu tiên dạng sóng, nhãn kênh và nội dung DECODE; các bộ đếm tích lũy chỉ được dùng như thông tin bổ trợ.

4.4.1 TC0 – CALIBRATION

Thiết kế và vai trò. TC0 tạo xung vuông 10 kHz trên CH0 và UART 9600 bps mang chuỗi `LA16-CAL` trên CH1. Đây là bước hiệu chuẩn bắt buộc nhằm kiểm tra time base của I2S0, thứ tự ánh xạ bit và cấu hình pair-swap trước khi chạy các kịch bản còn lại.

```
prepare(2000000, 24e6);
paint_clk(0, 10000, 0.5, 0, -1);
const uint8_t cal[] = "LA16-CAL\r\n";
double t = 1e9 / 9600 * 2;
while (t < end)
    t = paint_uart(1, 9600, cal, sizeof(cal)-1, t, 1, -1, 3.0);
```



Biểu đồ 4.3 Kết quả hiển thị TC0 trên phần mềm PRISM

Phân tích dạng sóng. Hình 4.3 sử dụng cửa sổ 2,834 ms. CH0 tạo xung vuông tuần hoàn, thời gian mức cao và thấp xấp xỉ nhau nên được PRISM gán nhãn CLK 10 kHz. CH1 nghỉ ở mức cao; mỗi cụm truyền bắt đầu bằng sườn xuống của start bit, tiếp theo là các bit dữ liệu có độ rộng khoảng $1/9600$ s và kết thúc ở mức cao của stop bit. Cấu trúc này phù hợp với UART 8N1 nên bộ auto-baud trả về 9600 bps. CH2–CH15 không được kích hoạt và giữ mức cao, đúng với trạng thái IDLE trong thiết kế.

Kết quả. Hai thông số hiệu chuẩn chính đều đạt: CH0 được đo bằng 10 kHz và CH1 được nhận dạng là UART 9600 bps. Khung DECODE trong ảnh chỉ chứa đoạn `4C 0D 0A 4C`, tương ứng phần cuối và phần đầu của hai lần lặp liên tiếp; nguyên nhân là cửa sổ quan sát cắt ngang chuỗi `LA16-CAL\r\n`, không phải firmware phát sai payload.

Việc tín hiệu xuất hiện đúng trên CH0 và CH1 cũng cho thấy ánh xạ bit sau cấu hình pair-swap không bị đảo kênh.

Đánh giá. TC0 đạt mục tiêu hiệu chuẩn tần số, baudrate và vị trí kênh. Ảnh hiện tại chưa chứa trọn một chuỗi `LA16-CAL`, vì vậy bằng chứng về payload đầy đủ được bổ sung bởi kết quả SIL; còn ảnh thực nghiệm đủ để xác nhận time base và ánh xạ vật lý trước khi chạy TC1–TC8.

4.4.2 TC1 – UART_SWEEP

Thiết kế và vai trò. TC1 phát đồng thời 16 luồng UART tại tám baudrate từ 4800 đến 230400 bps, mỗi tốc độ được lặp trên hai kênh. Payload ba byte của từng kênh được thay đổi theo số kênh để tránh trường hợp decoder chỉ nhận đúng một mẫu dữ liệu cố định.

```
for (int ch = 0; ch < 16; ch++) {
    uint8_t pl[3]; uart_payload(ch, pl);
    double t = 1e9 / TC1_BAUDS[ch] * 2;
    while (t < end)
        t = paint_uart(ch, TC1_BAUDS[ch], pl, 3, t, 1, -1, 3.0);
}
```



Biểu đồ 4.4 Kết quả hiển thị TC1 với 16 luồng UART đa tốc độ

Phân tích dạng sóng. Trong Hình 4.4, mật độ cạnh tăng rõ từ các kênh baud thấp sang baud cao. CH2–CH7 và CH9–CH15 được gán đúng các nhãn UART từ 19200 đến 230400 bps hoặc từ 9600 đến 230400 bps. Hai kênh 4800 bps là CH0 và CH8 cùng CH1

ở 9600 bps mới được gán DATA. Cửa sổ 2,722 ms chỉ chứa khoảng $4800 \times 2{,}722$ ms $=$ 13,1 bit ở 4800 bps. Trong khi đó, payload ba byte cần tối thiểu 30 bit UART 8N1, tương ứng $30/4800 = 6{,}25$ ms, chưa kể khoảng nghỉ giữa hai lần phát. Nếu cửa sổ bắt đầu giữa khung, decoder không quan sát được đủ chuỗi start–data–stop liên tiếp để xác nhận cấu trúc UART và ước lượng baudrate. Ngược lại, các kênh tốc độ cao chứa nhiều khung hoàn chỉnh trong cùng cửa sổ nên được nhận dạng ổn định hơn.

Kết quả. Ảnh thực nghiệm nhận dạng được 13/16 kênh là UART và cho thấy đúng quy luật mật độ cạnh theo baudrate. Ba kênh còn lại vẫn giữ waveform vật lý nhưng chỉ được phân loại DATA. Khung DECODE có một số lỗi frame do cửa sổ bắt đầu hoặc kết thúc giữa byte và do phiên thu đã tích lũy mất gói. Trong mô hình SIL, nơi cửa sổ có đủ dữ liệu liên tục, TC1 đạt 48/48 tiêu chí về loại giao thức, baudrate và payload.

Đánh giá. Tầng lấy mẫu và nguồn phát đáp ứng đúng yêu cầu đa kênh, nhưng ảnh live chỉ đạt một phần mục tiêu auto-detect 16/16 do độ dài cửa sổ chưa phù hợp với baud thấp. Cách khắc phục trực tiếp là tăng *Window size* trên GUI lên khoảng 10 ms và xóa buffer trước khi chụp; khi đó ngay cả kênh 4800 bps cũng chứa ít nhất một payload ba byte hoàn chỉnh. Kết quả SIL cho thấy decoder đạt yêu cầu khi đầu vào đầy đủ.

4.4.3 TC2 – I2C_SWEEP

Thiết kế và vai trò. TC2 sử dụng tám kênh để tạo bốn bus I2C độc lập tại 100 kHz, 400 kHz, 1 MHz và 200 kHz. Mỗi bus có địa chỉ và payload riêng, đồng thời sinh đầy đủ START, địa chỉ ghi, ACK, dữ liệu và STOP.

```
Bus B[] = {{0,1,100000,0x3C,{0x42,0xA7},2},
           {2,3,400000,0x55,{0x10,0x20},2},
           {4,5,1000000,0x6B,{0xDE},1},
           {6,7,200000,0x48,{0xAB,0xCD},2}};
for (auto &b : B)
    paint_i2c(b.scl, b.sda, b.f, b.addr, b.d, b.dn, t, true);
```



Biểu đồ 4.5 Kết quả hiển thị TC2 với các bus I2C hoạt động đồng thời

Phân tích dạng sóng. I2C có SCL ở mức cao khi bus rỗi, tạo các cụm chín xung cho một byte và bit ACK; SDA chỉ được phép đổi khi SCL thấp, ngoại trừ START và STOP. Hình 4.5 thể hiện rõ cấu trúc burst trên CH2/CH3 và CH4/CH5, vì vậy PRISM ghép hai cặp này thành I2C. Tuy nhiên cửa sổ chỉ dài 147,7 μs: một giao dịch nhiều byte ở 100 kHz cần vài trăm micro-giây, nên CH0/CH1 chưa đủ chuỗi START–data–STOP để được xác nhận. CH6/CH7 cũng bị tách thành UART/DATA thay vì bus I2C.

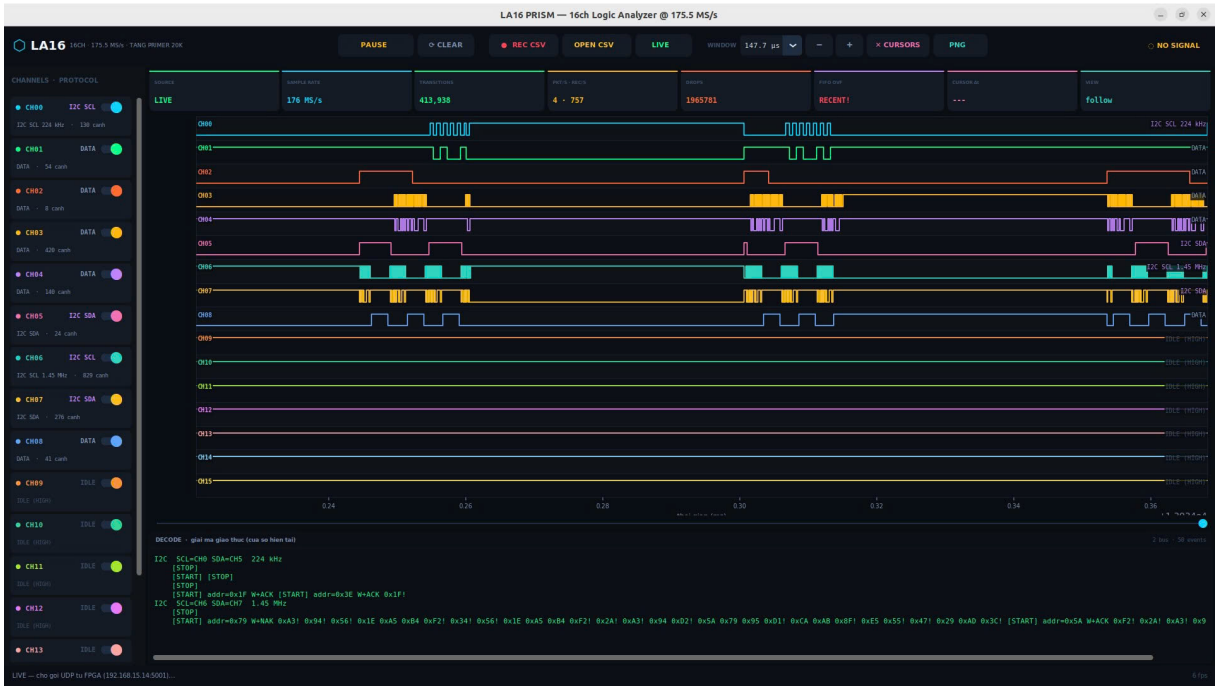
Kết quả. Ảnh nhận dạng được hai trong bốn bus, với tần số hiển thị khoảng 302 kHz trên CH2/CH3 và 574 kHz trên CH4/CH5. Khung DECODE có địa chỉ 0x54 và 0x6B, trong đó bus thứ hai vẫn cho thấy payload `DE`; các giá trị còn lại không khớp hoàn toàn cấu hình 400 kHz/1 MHz và địa chỉ 0x55/0x6B. Đây là dấu hiệu một số transition đã thiếu hoặc cửa sổ không bao trọn giao dịch. Dữ liệu SIL liên tục đạt 13/13 tiêu chí với đủ bốn bus.

Đánh giá. Ảnh thực nghiệm chứng minh được khả năng ghép cặp SCL–SDA và giải mã một phần giao dịch, nhưng chưa chứng minh đủ bốn bus và Fast-mode Plus 1 MHz. TC2 trên ảnh được đánh giá đạt một phần; cần chụp lại với buffer sạch, cửa sổ dài hơn và không có drops để đối chiếu chính xác tần số, địa chỉ và ACK.

4.4.4 TC3 – SPI_SWEEP

Thiết kế và vai trò. Ba bus SPI mode 0 được bố trí trên CH0–CH8 tại 1, 5 và 10 MHz. Mỗi bus truyền cùng chuỗi `A5 5A 3C`, nhờ đó có thể so sánh trực tiếp ảnh hưởng của tốc độ đến kết quả giải mã.

```
SpiBus B[] = {{0,1,2,1000000},
              {3,4,5,5000000},
              {6,7,8,10000000}};
const uint8_t d[3] = {0xA5, 0x5A, 0x3C};
for (auto &b : B)
    paint_spi(b.sck, b.mosi, b.cs, b.f, d, 3, 0, t, 2.0);
```



Biểu đồ 4.6 Kết quả hiển thị TC3 và hiện tượng nhận dạng nhầm SPI thành I2C

Phân tích dạng sóng. Theo ánh xạ thiết kế, CH0, CH3 và CH6 là SCK; CH1, CH4 và CH7 là MOSI; CH2, CH5 và CH8 là CS. Hình 4.6 vẫn thể hiện các burst clock, dữ liệu thay đổi trong burst và các khoảng chọn thiết bị. Tuy nhiên PRISM không xác lập được bộ ba SCK–MOSI–CS, sau đó heuristic I2C ghép chéo CH0 với CH5 và ghép CH6 với CH7 thành hai bus SCL–SDA. Cờ `FIFO OVF RECENT!` và số drops lớn cho biết dữ liệu gần cửa sổ đã bị thiếu.

Về cơ chế, bộ lọc SPI chỉ chấp nhận một nhóm kênh khi CS duy trì active-low bao trùm toàn bộ burst SCK và phần lớn transition của MOSI nằm trong khoảng CS được kích hoạt. Khi overflow hoặc mất gói làm biến mất một số sườn CS, cửa sổ tái dựng có thể xuất hiện CS bị đứt đoạn hoặc burst SCK không còn nằm trọn trong CS; điểm tin cậy của bộ ba SPI vì thế rơi dưới ngưỡng. Bộ lọc I2C sử dụng điều kiện ghép lỏng hơn: tìm một kênh clock dạng burst và một kênh dữ liệu có transition liên quan. Do đó nó vẫn có thể lấy SCK làm SCL và MOSI hoặc CS làm SDA, tạo ra bus I2C rác dù tín hiệu nguồn là SPI. Đây là giới hạn của heuristic trên Host khi dữ liệu truyền bị nhiễu hoặc thiếu, không phải giới hạn tốc độ lấy mẫu của FPGA.

Kết quả. Ảnh thực nghiệm không có nhãn SPI và không giải mã được chuỗi `A5 5A 3C`; thay vào đó khung DECODE xuất hiện các giao dịch I2C giả. Về lý thuyết, tại bus nhanh nhất số mẫu trên một chu kỳ SCK vẫn là

$$N_{SPI} = \frac{175{,}5\ \text{MS/s}}{10\ \text{MHz}} = 17{,}55\ \text{mẫu/chu kỳ}. \tag{4.11}$$

Mật độ 17,55 mẫu/chu kỳ cho thấy giới hạn không nằm ở tốc độ lấy mẫu, mà ở tính đầy đủ của transition và thuật toán ghép kênh. SIL sử dụng dữ liệu không mất cạnh vẫn đạt 7/7 tiêu chí. **Đánh giá.** Tầng thu waveform có đủ độ phân giải, nhưng kết quả live của TC3 chưa đạt mục tiêu auto-detect và giải mã SPI. Ảnh này đồng thời chỉ ra nhu cầu ưu tiên kiểm tra CS theo ánh xạ lân cận và từ chối ghép I2C khi SDA vi phạm điều kiện ổn định lúc SCL cao.

4.4.5 TC4 – CAN_SWEEP

Thiết kế và vai trò. TC4 phát năm kênh CAN classic 2.0A tại 125, 250, 500, 800 kbps và 1 Mbps. Các khung sử dụng ID, DLC và payload khác nhau để kiểm tra toàn bộ quá trình đồng bộ bit, bit-destuffing và CRC15.

```
CanCase C[] = {
 {0,125000,0x123,{0x11,0x22,0x33},3},
 {1,250000,0x201,{0xAA,0xBB},2},
 {2,500000,0x305,{0xDE,0xAD,0xBE,0xEF},4},
 {3,1000000,0x7FF,{0x01},1}, {4,800000,0x010,{},0}};
for (auto &c : C) paint_can(c.ch, c.br, c.id, c.d, c.dn, t);
```



Biểu đồ 4.7 Kết quả hiển thị TC4 với các khung CAN đa tốc độ

Phân tích dạng sóng. CAN classic dùng mức recessive cao khi rỗi và bắt đầu khung bằng SOF dominant thấp. Các đoạn có nhiều cạnh trên CH2–CH4 là trường arbitration, control, data và CRC sau bit stuffing. Trong cửa sổ 147,7 μs của Hình 4.7, PRISM nhận được khung hoàn chỉnh ở 500 kbps, 800 kbps và 1 Mbps. Ngược lại, khung 125 kbps và 250 kbps có thời gian truyền dài hơn cửa sổ, nên CH0 và CH1 chỉ hiện một phần waveform và được giữ ở nhãn DATA.

Kết quả. CH2 được giải mã CAN 500 kbps với ID 0x305, DLC=4 và payload `DE AD BE EF`; CH3 là CAN 1 Mbps với ID 0x7FF, DLC=1, dữ liệu `01`; CH4 là CAN 800 kbps với ID 0x010 và DLC=0. Khung DECODE báo CRC hợp lệ cho cả ba kênh. Hai kênh 125/250 kbps chưa được xác nhận trong ảnh, trong khi SIL với cửa sổ đầy đủ đạt 25/25 tiêu chí cho năm kênh.

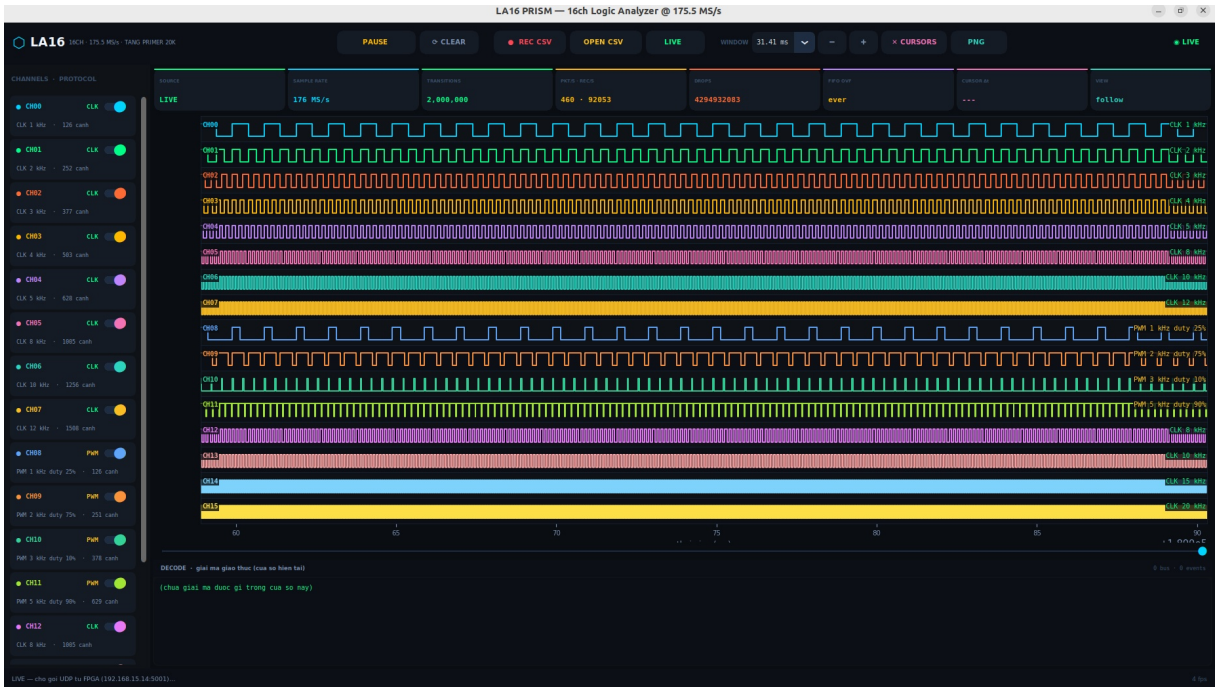
Đánh giá. TC4 đạt hoàn toàn trên ba kênh tốc độ cao và đạt một phần trên toàn bộ cấu hình năm kênh. CRC15 đúng chứng minh destuffing và vị trí các trường không bị lệch. Để kiểm chứng hai bus chậm bằng ảnh live, cửa sổ cần tăng lên tối thiểu khoảng 1 ms để chứa trọn khung cùng khoảng nghỉ.

4.4.6 TC5 – CLK_SWEEP

Thiết kế và vai trò. TC5 phủ toàn bộ 16 kênh bằng CLK/PWM từ 1 đến 20 kHz. Các kênh duty 50% dùng để kiểm tra phép đo clock, trong khi duty 10%, 25%, 75% và 90% kiểm tra ngưỡng phân loại PWM.

```
ClockCase K[] = {
 {0,1000,.5}, {6,10000,.5}, {7,12000,.5},
```

```
 {8,1000,.25}, {9,2000,.75},
 {10,3000,.10}, {11,5000,.90}, {15,20000,.5}};
for (auto &k : K)
    paint_clk(k.ch, k.f, k.duty, 0, -1);
```



Biểu đồ 4.8 Kết quả thực nghiệm HIL của TC5 sau khi chạy lại CLK_SWEEP

Phân tích dạng sóng. Hình 4.8 là kết quả LIVE sau khi chạy lại `CLK_SWEEP`; GUI hiển thị sample rate làm tròn 176 MS/s và cửa sổ 31,41 ms. CH0–CH7 lần lượt được gán CLK 1, 2, 3, 4, 5, 8, 10 và 12 kHz. Các tín hiệu này có thời gian HIGH và LOW xấp xỉ nhau, vì vậy duty cycle nằm trong ngưỡng 44–56% và được phân loại là CLK. Mật độ cạnh tăng dần theo tần số: CH0 còn quan sát rõ từng chu kỳ 1 ms, trong khi CH6 và CH7 tạo các vạch dày hơn do có 10–12 nghìn chu kỳ mỗi giây.

CH8–CH11 được nhận dạng là PWM với các thông số tương ứng 1 kHz–25%, 2 kHz–75%, 3 kHz–10% và 5 kHz–90%. Trên waveform, CH8 chỉ duy trì mức cao trong một phần tư chu kỳ, CH9 duy trì mức cao trong ba phần tư chu kỳ, CH10 tạo các xung cao rất hẹp và CH11 chỉ có khoảng mức thấp ngắn. CH12–CH15 tiếp tục được gán CLK 8, 10, 15 và 20 kHz. Ở cửa sổ dài 31,41 ms, CH14 và CH15 có số chu kỳ rất lớn trên bề rộng hữu hạn của màn hình nên các sườn bị nén thành dải màu đặc; nhãn 15 kHz và 20 kHz ở bảng CHANNELS xác nhận đây là hiệu ứng mật độ điểm ảnh, không phải tín hiệu bị giữ ở một mức logic.

Kết quả. PRISM phân loại đúng 12 kênh CLK và bốn kênh PWM, đồng thời các tần số và duty cycle hiển thị khớp toàn bộ cấu hình đã thiết kế. Khung DECODE báo 0 bus và 0 event, phù hợp vì TC5 không chứa UART, I2C, SPI hoặc CAN. Cờ `FIFO OVF`

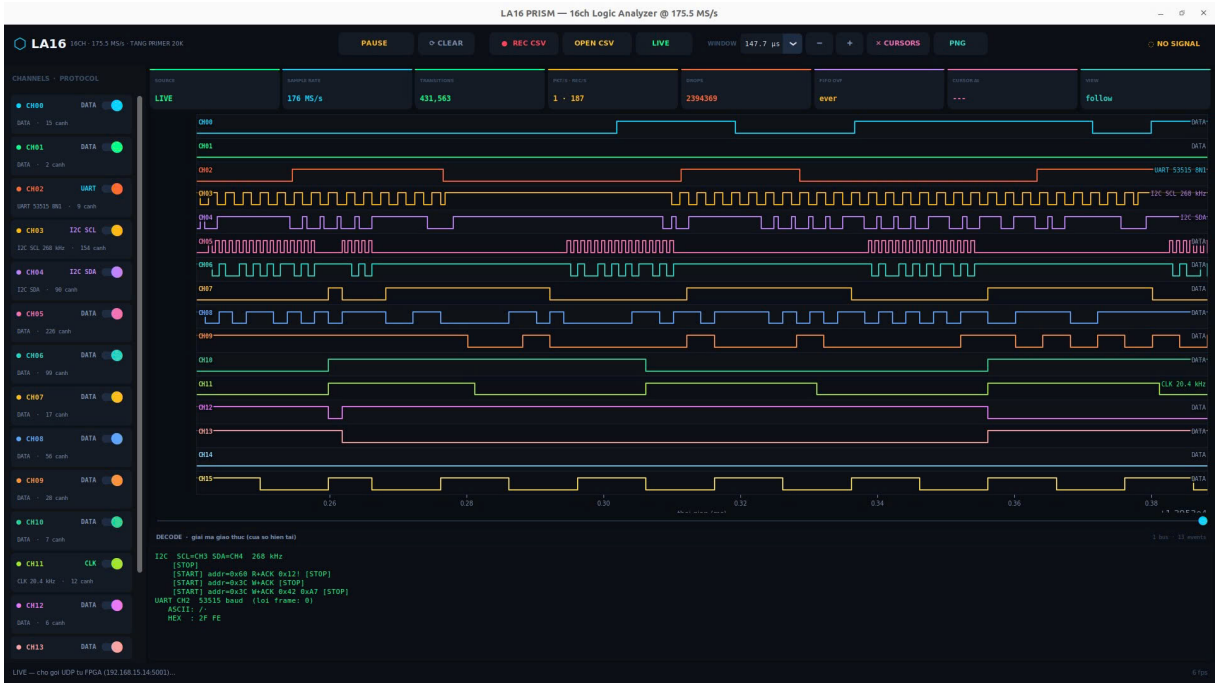
ever và giá trị drops trên thanh trạng thái là thông tin tích lũy của phiên chạy; chúng không làm thay đổi kết luận của cửa sổ hiện tại vì waveform vẫn tuần hoàn, đầy đủ và được nhận dạng đúng trên cả 16 kênh. Kết quả SIL độc lập đạt 36/36 tiêu chí, thống nhất với ảnh HIL.

Đánh giá. TC5 đạt đầy đủ ở cả hai mức SIL và HIL. Kết quả chứng minh bộ đo chu kỳ hoạt động ổn định trên dải 1–20 kHz, ngưỡng duty cycle phân biệt đúng CLK với PWM và auto-detect không gán nhầm các tín hiệu tuần hoàn thành giao thức dữ liệu. Việc sử dụng cửa sổ dài hơn chu kỳ của kênh chậm nhất cũng xác nhận giải pháp tăng *Window size* là cần thiết để tạo đủ thống kê cho bộ phân loại.

4.4.7 TC6 – MIX_ALL

Thiết kế và vai trò. TC6 đặt toàn bộ năm nhóm giao thức lên 16 kênh: UART trên CH0–CH2, I2C trên CH3/CH4, SPI trên CH5–CH7, CAN trên CH8/CH9 và CLK/PWM trên CH10–CH15. Mục tiêu là đánh giá auto-detect và tải xử lý khi các đặc trưng tín hiệu xuất hiện đồng thời.

```
UartCase U[] = {{0,115200}, {1,9600}, {2,57600}};
paint_i2c(3, 4, 400000, 0x3C, i2c_data, 2, t, true);
paint_spi(5, 6, 7, 1000000, spi_data, 2, 0, t, 2.0);
paint_can(8, 500000, 0x305, can0, 2, t);
paint_can(9, 250000, 0x201, can1, 2, t);
paint_clk(14, 1000, .25, 0, -1);
```



Biểu đồ 4.9 Kết quả hiển thị TC6 khi nhiều giao thức cùng hoạt động

Phân tích dạng sóng. Hình 4.9 thể hiện đúng tính chất hỗn hợp: CH0–CH2 có dạng UART bất đồng bộ; CH3 tạo clock burst và CH4 là dữ liệu I2C; CH5–CH7 có burst SPI; CH8/CH9 mang khung CAN; CH10–CH15 có các xung tuần hoàn hoặc PWM. Tuy nhiên với cửa sổ 147,7 μs, PRISM chỉ xác nhận CH2 là UART khoảng 53515 bps, ghép CH3/CH4 thành I2C khoảng 268 kHz và nhận CH11 là CLK 20,4 kHz. Các nhóm SPI và CAN vẫn mang nhãn DATA do cửa sổ chưa chứa đủ burst hoặc một frame CRC hợp lệ.

Kết quả. Khung DECODE đọc được một số giao dịch I2C, trong đó có địa chỉ 0x3C và payload `42 A7`, đồng thời giải mã được dữ liệu UART trên CH2. Baudrate 53515 bps và I2C 268 kHz còn sai lệch so với 57600 bps và 400 kHz trong cấu hình. SPI CH5–CH7, CAN CH8/CH9 và phần lớn CLK/PWM chưa được tự động gán nhãn. SIL với dữ liệu liên tục đạt 10/10 tiêu chí tích hợp.

Đánh giá. TC6 chứng minh waveform của nhiều giao thức có thể được thu đồng thời, nhưng auto-detect live chỉ đạt một phần. Kết quả cho thấy độ dài và tính liên tục của cửa sổ là tham số quan trọng: decoder CAN cần đủ một frame cùng CRC, còn SPI cần quan sát trọn quan hệ SCK–MOSI–CS. Cần giảm drops và tăng cửa sổ trước khi kết luận khả năng nhận dạng đồng thời toàn bộ 16 kênh.

4.4.8 TC7 – EDGE_CASES

Thiết kế và vai trò. TC7 chủ động đưa hệ thống đến các trường hợp biên: xung 150 ns, UART 250000 bps, UART 8E1, SPI mode 2, CAN có nhiều bit stuffing và I2C NAK. Kịch bản này phân biệt giới hạn của tầng lấy mẫu với giới hạn cấu hình của decoder.

```
set_lvl(0, s, s + 3, 0);                          // pulse 150 ns
paint_uart(1, 250000, u0, 2, t, 1, -1, 3.0);     // nonstandard baud
paint_uart(2, 19200, u1, 2, t, 1, 0, 3.0);       // UART 8E1
paint_spi(3, 4, 5, 2000000, sp, 2, 2, t, 2.0);   // SPI mode 2
paint_can(6, 500000, 0x000, can_d, 4, t);         // stuffing stress
paint_i2c(7, 8, 100000, 0x55, i2c_d, 1, t, false); // NAK
```



Biểu đồ 4.10 Kết quả hiển thị TC7 với các trường hợp biên của decoder

Phân tích dạng sóng. Trên CH0 của Hình 4.10, các xung 150 ns bị nén thành những vạch hẹp ở thang hiển thị 147,7 μs nhưng vẫn xuất hiện liên tục. Mỗi xung tương ứng khoảng $150/5{,}698 = 26{,}3$ mẫu FPGA, vì vậy việc nhìn thấy đầy đủ các vạch chứng minh tầng lấy mẫu không bỏ xung. CH1 có cấu trúc UART idle-high và được nhận đúng 250000 bps. CH2 mang UART 8E1 nhưng chỉ được gán DATA do decoder giả định 8N1. Clock của SPI mode 2 trên CH3 bị detector UART diễn giải thành khoảng 3,98 Mbps; các kênh CH4/CH5 vẫn giữ waveform MOSI/CS. CAN trên CH6 chưa vượt điều kiện CRC để được xác nhận. CH7/CH8 được ghép thành I2C nhưng chỉ đo khoảng 68,1 kHz.

Kết quả. Hai yêu cầu quan trọng được xác nhận trực tiếp là bắt được xung 150 ns và nhận UART phi chuẩn 250000 bps. Các cấu hình cố ý nằm ngoài phạm vi hỗ trợ tạo ra biểu hiện dự kiến: UART 8E1 không được giải mã như 8N1 và SPI mode 2 bị nhận sai giao thức. Tuy nhiên I2C trong ảnh trả địa chỉ 0x54 với ACK, chưa khớp địa chỉ 0x55 và NAK đã thiết kế; CAN stuffing cũng chưa được xác nhận trong khung DECODE. SIL đạt 7/7 tiêu chí bắt buộc khi đầu vào không mất transition.

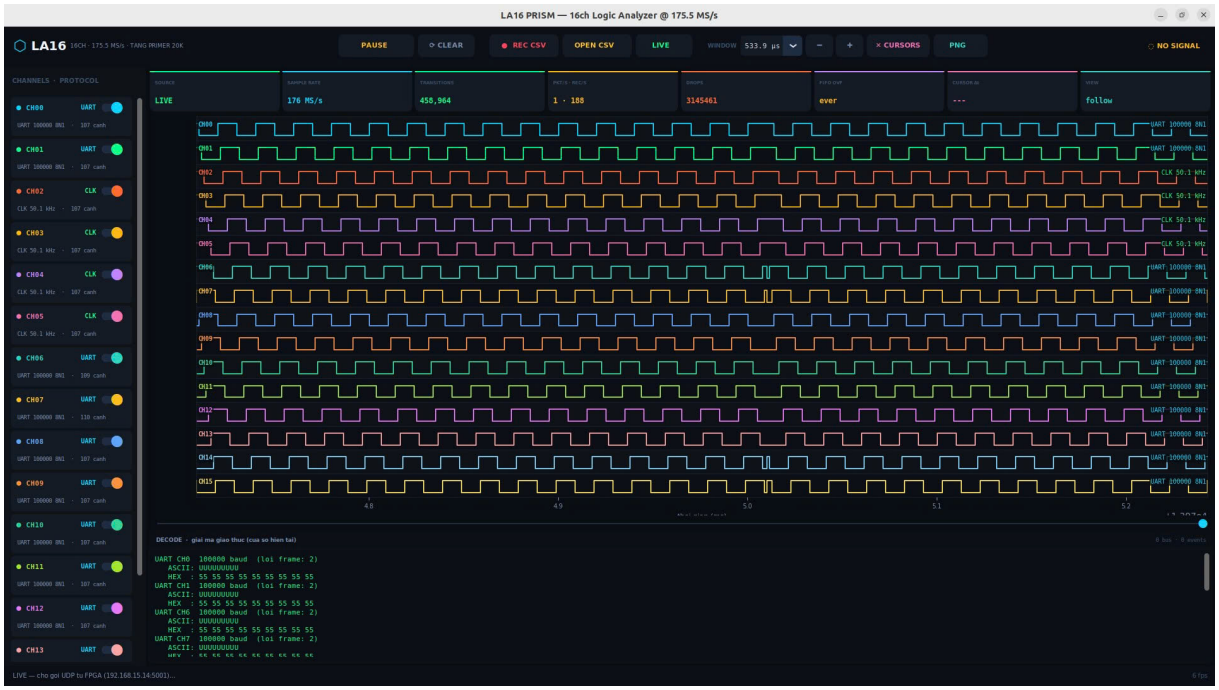
Đánh giá. TC7 đạt mục tiêu chứng minh độ phân giải phần cứng và đồng thời bộc lộ giới hạn phần mềm. Kết quả củng cố hướng mở rộng selector CPOL/CPHA, parity/stop bit và điều kiện ghép I2C. Phần CAN và I2C NAK trên ảnh chỉ đạt một phần, nên không được dùng làm bằng chứng rằng toàn bộ trường hợp biên đã giải mã đúng.

4.4.9 TC8 – BW_STRESS

Thiết kế và vai trò. TC8 phát 16 xung vuông 50 kHz với pha lệch đều $1/16$ chu kỳ. Các cạnh không trùng nhau nên RLE không thể gộp nhiều kênh vào cùng một record;

đây là tải bất lợi nhất đối với FIFO và Ethernet.

```
prepare(8000000, 6e6);
double per_ns = 1e9 / 50000.0;
for (int ch = 0; ch < 16; ch++) {
    idle_ch(ch, 0);
    paint_clk(ch, 50000, 0.5, per_ns * ch / 16.0, -1);
}
```



Biểu đồ 4.11 Kết quả hiển thị TC8 với 16 xung vuông 50 kHz lệch pha

Phân tích dạng sóng. Hình 4.11 cho thấy cả 16 kênh đều có chu kỳ gần 20 μs, tương ứng 50,1 kHz, và các sườn được dịch lần lượt theo pha. Như vậy nguồn I2S–DMA đã tạo đúng tải vật lý. PRISM chỉ gán CH2–CH5 là CLK; 12 kênh còn lại bị nhận thành UART 100000 bps và khung DECODE xuất hiện lặp byte 55. Sự nhập nhằng này được biểu diễn trực tiếp bởi

$$T_{\text{half}} = \frac{1}{2 \times 50\ \text{kHz}} = 10\ \mu\text{s} = \frac{1}{100000\ \text{bps}} = T_{\text{bit,UART}}. \tag{4.12}$$

Như vậy mỗi nửa chu kỳ CLK trùng hoàn toàn với một bit UART 100000 bps. Nếu chọn một sườn xuống làm start bit, chuỗi mức cao–thấp xen kẽ tiếp theo có thể được lấy mẫu thành 01010101_2, tức byte 0x55. Khi drops liên tục làm mất một số cạnh và phá vỡ tiêu chí ổn định chu kỳ của bộ phân loại CLOCK, detector UART vẫn có thể khớp các khoảng 10 μs còn lại và đưa ra nhãn UART giả.

Kết quả. Dạng sóng vật lý và tần số của 16 kênh phù hợp thiết kế, nhưng kết quả

auto-detect không đạt kỳ vọng 16/16 CLK: chỉ 4 kênh mang nhãn CLK và 12 kênh mang nhãn UART. Tổng tốc độ cạnh lý tưởng vẫn là

$$R_{edge} = 16 \times 2 \times 50\ \text{kHz} = 1{,}6 \times 10^6\ \text{cạnh/s}, \tag{4.13}$$

vượt ngân sách thực dụng khoảng 1,5 triệu transition/s của đường truyền. Ảnh có `FIFO OVF ever` và hơn ba triệu drops tích lũy, chứng minh phiên chạy từng quá tải, nhưng không có `RECENT!` nên không thể khẳng định overflow vừa phát sinh đúng trong cửa sổ đang chụp. SIL xác nhận cấu hình 16 clock và dự báo đúng điều kiện tải vượt ngưỡng.

Đánh giá. TC8 đạt mục tiêu tạo tải và xác định biên thông lượng, nhưng chưa đạt tiêu chí phân loại toàn bộ kênh là CLK. Việc nhầm UART 100000 bps là trường hợp mơ hồ về mặt toán học, không phải mất khả năng lấy mẫu; decoder cần ưu tiên tính tuần hoàn dài hạn hoặc phát hiện đồng thời nhiều kênh cùng tần số trước khi thử UART. Khi overflow xảy ra, bộ đếm `delta` vẫn chạy nên record hợp lệ tiếp theo tiếp tục bảo toàn time base, dù một số transition trung gian bị thiếu.

4.5 Giải pháp mô phỏng Software-in-the-Loop

Trong V-Model, SIL cung cấp lớp kiểm thử đơn vị/chức năng đối xứng với thiết kế chi tiết của detector và decoder. Kiểm thử hoàn toàn trên phần cứng làm vòng lặp phát triển kéo dài, trong khi mô hình SIL tái tạo dữ liệu của firmware ngay trên máy tính. Tệp `tc_lib.py` tạo mảng 16 bit, các painter ghi giao thức vào từng khoảng mẫu, rồi `to_transitions()` chuyển mảng thành chuỗi thời gian–trạng thái giống dữ liệu RLE.

Thời điểm transition được lượng tử hóa theo 175,5 MHz; các transition trùng chu kỳ được hợp nhất như khi FPGA chỉ chốt một trạng thái ở mỗi sườn `clk_sample`. Tệp `tc_scenarios.py` chứa tham số TC1–TC8. Tệp `run_testcases.py` gọi trực tiếp `detect.analyze()` của PRISM, nên SIL kiểm tra đúng mã giải mã sản phẩm.

Kết quả đạt 8/8 test case tự động: TC1 đạt 48/48, TC2 đạt 13/13, TC3 đạt 7/7, TC4 đạt 25/25, TC5 đạt 36/36, TC6 đạt 10/10, TC7 đạt 7/7 và TC8 nhận đúng 16 clock. Script còn xuất được `golden_TC*.csv` để mở trực tiếp bằng PRISM. SIL nhờ đó rút ngắn vòng phát triển, hỗ trợ kiểm thử hồi quy và cung cấp chuẩn tham chiếu lặp lại được.

Kết chương

Chương này đã chuẩn hóa kiểm thử LA16 theo V-Model, trong đó SIL xác minh thuật toán, HIL xác minh chuỗi tích hợp vật lý và TC8 đánh giá biên hệ thống. Bộ phát I2S0–DMA tạo được 16 kênh đồng thời và loại bỏ jitter tương đối do phần mềm. Thiết kế dùng 4078/20736 tài nguyên Logic, 13/46 BSRAM và đóng timing tại 175,5 MHz

với độ phân giải 5,70 ns. Kết quả SIL đạt 8/8 kịch bản tự động TC1–TC8, xác nhận thuật toán khi dữ liệu đầu vào đầy đủ. Các ảnh thực nghiệm cho thấy TC0 và TC5 đạt đầy đủ; TC1, TC2, TC4, TC6 và TC7 đạt một phần; TC3 bộc lộ lỗi ghép SPI thành I2C khi thiếu transition; TC8 tạo đúng waveform nhưng gặp giới hạn băng thông và sự mơ hồ giữa CLK 50 kHz với UART 100000 bps. Những sai lệch này xác định rõ các hướng cần cải thiện về cửa sổ quan sát, kiểm soát mất gói và thứ tự ưu tiên của auto-detect trong Chương 5.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Tổng quan chương

Từ kiến trúc, quá trình hiện thực và kết quả kiểm thử ở Chương 4, chương này tổng hợp các kết quả chính của LA16 Logic Analyzer, phân tích hạn chế và đề xuất hướng nâng cấp. Hệ thống được đánh giá theo yêu cầu bài tập lớn ET4361: tối thiểu 8 kênh, tần số lấy mẫu 25 MHz và quan sát dạng sóng trên máy tính [1]. Nội dung đồng thời phân biệt tốc độ lấy mẫu trong FPGA với khả năng truyền dữ liệu liên tục qua Ethernet.

5.1 Kết luận

Đề tài đã hoàn thành và vượt các mục tiêu cơ bản của một máy phân tích logic nhúng hiệu năng cao. LA16 lấy mẫu đồng thời 16 kênh tại 175,5 MHz, tương ứng chu kỳ 5,70 ns. Các đầu vào được chốt theo cùng miền xung nhịp do rPLL tạo ra [44], nhờ đó duy trì nhất quán quan hệ thời gian giữa các kênh. Với phép đo bất đồng bộ, sai số xác định thứ tự sườn xung không quá một chu kỳ mẫu, tức $\pm 5{,}70$ ns. Kết quả đáp ứng yêu cầu timing và tạo đủ số mẫu cho các tín hiệu UART, I^2C, SPI thông dụng.

Phương pháp nén RLE phần cứng cũng được chứng minh là đúng đắn. FPGA chỉ tạo bản ghi gồm độ lệch thời gian và trạng thái 16 bit mới khi có kênh chuyển mức, qua đó giảm mạnh dữ liệu của các giao thức có khoảng nghỉ mà vẫn giữ độ phân giải 5,70 ns cho mỗi cạnh. UDP được chọn để giảm độ trễ và độ phức tạp phần cứng [41]; số thứ tự gói giúp phát hiện mất gói. Khi FIFO đầy, cờ tràn được gửi về máy tính nhưng bộ đếm thời gian vẫn tiếp tục. Vì vậy, hệ thống có thể mất cạnh trong khoảng quá tải song các bản ghi sau đó vẫn đúng trên trục thời gian. Cơ chế này phản ánh rõ trạng thái quá tải, thay vì tạo backlog do truyền lại hoặc backpressure như TCP [6].

Bo mạch mở rộng analog bổ sung mạng suy giảm bù tần số, ESD/steering diode, comparator TLV3801/TLV3601, ngưỡng MCP4728 và khối nguồn được bảo vệ bởi TPS259250, LT3045 và TLVH431. I2C Master trên FPGA cùng TCA9535 cho phép cấu hình ngưỡng và trình tự điện áp kéo lên, qua đó mở rộng hệ thống từ datapath số nối trực tiếp thành một nguyên mẫu có front-end bảo vệ và điều khiển mức điện áp.

Phần mềm PRISM hoàn thiện chuỗi thu nhận, hiển thị và phân tích. Giao diện hỗ trợ 16 kênh, zoom, tạm dừng, con trỏ đo thời gian, lưu CSV, xuất ảnh và giải mã UART, I^2C, SPI, CAN Classic 2.0A, clock/PWM; bộ giải mã I^2C tuân theo START, STOP và ACK của UM10204 [43]. Với chi phí hợp lý và thao tác trực quan, hệ thống có khả năng ứng dụng trong thực hành Vi xử lý, FPGA và Hệ thống nhúng tại Trường Điện – Điện tử.

Xét theo tổ chức thực hiện, kết quả của đề tài được hình thành từ bốn gói công việc:

(i) Nguyễn Đức Phúc chủ trì RTL FPGA, I2C Master và timing closure; (ii) Bùi Đức Công Vinh phụ trách báo cáo, Host backend, heuristic, decoder và SIL; (iii) Trần Quang Khải phụ trách bo mạch analog, nguồn bảo vệ và driver I2S0–DMA; (iv) Nguyễn Minh Đức phụ trách GUI PRISM, điều khiển ngưỡng/PWM, đo HIL và phân tích timing. Việc định nghĩa rõ giao diện giữa các gói giúp kết quả kiểm thử có thể truy vết từ yêu cầu, thiết kế, mã nguồn đến bằng chứng thực nghiệm.

5.2 Hạn chế của hệ thống

Bo front-end đã mở rộng miền mức cao số có thể tiếp nhận từ 0,9 V đến 5 V bằng bộ phân áp bù tần số, comparator và mạng bảo vệ ESD/quá áp. Tuy nhiên, cấu hình HIL nối trực tiếp PMOD vẫn chỉ an toàn trong miền LVCMOS 0–3,3 V. Hệ thống không đo trực tiếp RS-232 khoảng ± 12 V, tín hiệu có mức âm hoặc tín hiệu analog liên tục; các trường hợp này cần tầng cách ly hay chuyển đổi chuyên dụng. Khả năng chuyển mạch 175 MHz của comparator cũng không đồng nghĩa hệ lấy mẫu 175,5 MS/s có thể tái dựng lossless một clock 175 MHz liên tục.

Giới hạn thứ hai là Fast Ethernet 100 Mbps sử dụng PHY RTL8201F và RMII 50 MHz [7, 40]. Trong khi đó, luồng thô 16 bit tại 175,5 MS/s đạt 2,808 Gbps. RLE phù hợp tín hiệu burst nhưng giảm hiệu quả khi nhiều kênh chuyển mức liên tục. Ở TC8, FIFO có thể tràn nên hệ thống không bảo đảm truyền đầy đủ mọi cạnh trên 16 kênh. Đây là giới hạn truyền thời gian thực: FPGA vẫn lấy mẫu ở 175,5 MHz nhưng Fast Ethernet không thể tải toàn bộ dữ liệu phát sinh.

5.3 Hướng phát triển và nâng cấp

Ưu tiên đầu tiên là chế độ *Triggered Capture*. FPGA duy trì bộ đệm vòng trong BSRAM hoặc DDR3 để lưu dữ liệu trước và sau trigger theo sườn, mẫu bit hoặc giao thức. BSRAM hỗ trợ bộ nhớ hai cổng và các miền clock độc lập [11], phù hợp cửa sổ ngắn; DDR3 trên Tang Primer 20K đáp ứng cửa sổ dài hơn. Sau khi dừng thu, dữ liệu được truyền chậm về máy tính, nhờ đó bảo toàn toàn bộ mẫu tại tần số cực đại mà không phụ thuộc băng thông tức thời.

Hướng thứ hai là nâng cấp sang Gigabit Ethernet hoặc FTDI FT601. Gigabit Ethernet tăng đáng kể tốc độ nhưng vẫn cần nén hoặc trigger do luồng thô đạt 2,808 Gbps. FT601 cung cấp cầu nối USB 3.0 SuperSpeed và FIFO song song 32 bit [12], qua đó giảm mạnh nút thắt truyền tải.

Cuối cùng, có thể bổ sung *Glitch Filter* lập trình được cho từng kênh và hiệu chuẩn skew của front-end. Chuyển mức chỉ được công nhận khi trạng thái mới ổn định đủ số chu kỳ mẫu; xung ngắn hơn ngưỡng sẽ bị loại bỏ. Bộ lọc giúp giảm nhiễu do probe dài, xuyên nhiễu hoặc sườn chậm, còn bảng hiệu chuẩn bù sai lệch cố định của comparator và đường mạch. Các ngưỡng phải được chọn hợp lý để không loại bỏ xung hợp lệ.

Kết chương

Báo cáo đã trình bày quá trình xây dựng LA16 từ yêu cầu, front-end bảo vệ và dịch mức, thiết kế FPGA–Ethernet, nén RLE, phần mềm PRISM đến kiểm thử. Nguyên mẫu đạt 16 kênh, 175,5 MS/s và độ phân giải 5,70 ns, hoàn thành tốt mục tiêu học phần. Các giới hạn còn lại về tốc độ lấy mẫu, băng thông truyền và miền tín hiệu âm đã được nhận diện cùng lộ trình nâng cấp bằng capture theo trigger, giao tiếp tốc độ cao, hiệu chuẩn skew và lọc glitch. Đây là nền tảng để LA16 tiếp tục phát triển thành công cụ đo lường và giảng dạy hệ thống nhúng hoàn chỉnh hơn.

TÀI LIỆU THAM KHẢO

[1] Bộ môn Kỹ thuật máy tính, "Bài tập lớn Hệ thống nhúng và thiết kế giao tiếp nhúng 2025.2," Đại học Bách khoa Hà Nội, Hà Nội, Việt Nam, 2025.

[2] Sipeed, "Tang Primer 20K Documentation," Sipeed Co., Ltd., 2023. [Online]. Available: Sipeed Wiki.

[3] Đ. V. Hùng, "Hệ thống nhúng – Phần I," bài giảng học phần Hệ thống nhúng và thiết kế giao tiếp nhúng, Đại học Bách khoa Hà Nội, Hà Nội, Việt Nam.

[4] Đ. V. Hùng, "Thiết kế giao tiếp nhúng – Phần II," bài giảng học phần Hệ thống nhúng và thiết kế giao tiếp nhúng, Đại học Bách khoa Hà Nội, Hà Nội, Việt Nam.

[5] Intel Corporation, "Intel Quartus Prime Pro Edition User Guide: Timing Analyzer – Managing Metastability," Intel Corporation, 2022.

[6] W. Eddy, "Transmission Control Protocol (TCP)," Internet Engineering Task Force, RFC 9293, Aug. 2022.

[7] Realtek Semiconductor Corp., "RTL8201F Single-Chip/Single-Port 10/100 Mbps Ethernet PHYceiver Data Sheet," Realtek Semiconductor Corp., Hsinchu, Taiwan.

[8] Texas Instruments, "AN-1405: RMII Mode for the DP83848 Ethernet Transceiver," Texas Instruments, Dallas, TX, USA, Application Report SNLA076.

[9] Nhóm phát triển LA16, "LA16 – Kiến trúc hệ thống," tài liệu thiết kế nội bộ, Đại học Bách khoa Hà Nội.

[10] Nhóm phát triển LA16, "LA16 – Giao thức UDP Streaming," tài liệu thiết kế nội bộ, Đại học Bách khoa Hà Nội.

[11] Gowin Semiconductor, "Gowin BSRAM User Guide," Gowin Semiconductor Corp., Guangzhou, China.

[12] Future Technology Devices International Ltd., "FT600/FT601 USB 3.0 to FIFO Bridge Datasheet," Future Technology Devices International Ltd., Glasgow, UK.

[40] Texas Instruments, "DP83826 Deterministic, Low-Latency, Low-Power 10/100 Mbps Ethernet PHY Datasheet," Texas Instruments, Dallas, TX, USA, Doc. SNLS641A, 2020.

[41] J. Postel, "User Datagram Protocol," Internet Engineering Task Force, RFC 768, Aug. 1980.

[42] Robert Bosch GmbH, “CAN Specification, Version 2.0,” Robert Bosch GmbH, Stuttgart, Germany, 1991.

[43] NXP Semiconductors, “I2C-bus Specification and User Manual,” NXP Semiconductors, Eindhoven, Netherlands, User Manual UM10204, Rev. 7, 2021.

[44] Gowin Semiconductor, “GW2A Series FPGA Products Data Sheet,” Gowin Semiconductor Corp., Guangzhou, China, Doc. DS102, 2024.

[45] Espressif Systems, “Universal Asynchronous Receiver/Transmitter (UART),” *ESP-IDF Programming Guide for ESP32*. [Online]. Available: Espressif Documentation. Accessed: Jul. 19, 2026.

[46] NXP Semiconductors, “Synchronous Serial Peripheral Interface,” in *M68HC11 Reference Manual*, Rev. 6.1, Sec. 8.

[47] Texas Instruments, “ISO1044 Isolated CAN FD Transceiver in Small Package,” Datasheet SLLSFB0A, Rev. A, Jul. 2020.

[48] Espressif Systems, *ESP32 Series Datasheet*, Version 5.2. [Online]. Available: Espressif Systems. Accessed: Jul. 19, 2026.

[49] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.7. [Online]. Available: Espressif Systems. Accessed: Jul. 19, 2026.

[50] Nhóm phát triển LA16, *Bộ phát kiểm thử 16 kênh I2S0 LCD-Parallel DMA trên ESP32*, mã nguồn và tài liệu thiết kế nội bộ, Đại học Bách khoa Hà Nội, 2026.

[51] Nhóm phát triển LA16, *Sơ đồ nối dây ESP32–Tang Primer 20K*, tài liệu thiết kế nội bộ, Đại học Bách khoa Hà Nội, 2026.

[52] Texas Instruments, *TLV3801, TLV3802, TLV3811(C) 225-ps High-Speed Comparators with LVDS Outputs*, Datasheet, Rev. C, Dec. 2023.

[53] Analog Devices, *ADCMP604/ADCMP605 Rail-to-Rail, Very Fast, 2.5 V to 5.5 V, Single-Supply LVDS Comparators*, Datasheet, Rev. C.

[54] Texas Instruments, *TLV3604, TLV3605, TLV3607 800-ps High-Speed Comparators with LVDS Outputs*, Datasheet.

[55] Texas Instruments, *TLV360x 325-MHz High-Speed Comparators with 2.5-ns Propagation Delay*, Datasheet, Rev. E, Apr. 2023.

[56] Analog Devices, *LTC6752: 280-MHz, 2.9-ns Comparator Family with Rail-to-Rail Inputs and CMOS Outputs*, Datasheet, Apr. 2017.

[57] Analog Devices, *ADCMP600/ADCMP601/ADCMP602 Rail-to-Rail, Very Fast, Single-Supply TTL/CMOS Comparators*, Datasheet, Rev. A.

[58] Microchip Technology, *MCP4728 12-Bit, Quad Digital-to-Analog Converter with EEPROM Memory*, Datasheet DS22187E.

[59] Analog Devices, *LTC2635 Quad 12-/10-/8-Bit I2C Voltage-Output DACs with 10-ppm/°C Reference*, Datasheet, Rev. D, May 2017.

[60] Texas Instruments, *DACx3204 12-Bit, 10-Bit, and 8-Bit Quad Voltage and Current Output Smart DACs*, Datasheet, Rev. A, Dec. 2021.

[61] Texas Instruments, *TLV906xS 10-MHz, RRIO, CMOS Operational Amplifiers for Cost-Sensitive Systems*, Datasheet, Rev. N, Jul. 2026.

[62] Texas Instruments, *OPAx333 1.8-V, microPower, CMOS Operational Amplifiers, Zero-Drift Series*, Datasheet, Rev. E, Dec. 2015.

[63] Analog Devices, *AD8605/AD8606/AD8608 Low-Noise, CMOS, Rail-to-Rail Input/Output Precision Amplifiers*, Datasheet, Rev. O.

[64] Texas Instruments, *LSF0204x 4-Bit Bidirectional Multi-Voltage Level Translator for Open-Drain and Push-Pull Applications*, Datasheet, Rev. H, Apr. 2021.

[65] Texas Instruments, *TXB0104 4-Bit Bidirectional Voltage-Level Translator with Automatic Direction Sensing*, Datasheet, Rev. K, Mar. 2025.

[66] Texas Instruments, *TXS0104E 4-Bit Bidirectional Voltage-Level Translator for Open-Drain and Push-Pull Applications*, Datasheet, Rev. K, Oct. 2023.

[67] Texas Instruments, *SN74LVC2T45 Dual-Bit Dual-Supply Bus Transceiver with Configurable Voltage Translation*, Datasheet.

[68] S. Rao and A. Ozer, *Biasing Requirements for TXS, TXB, and LSF Auto-Bidirectional Translators*, Texas Instruments Application Report SCEA060, Oct. 2017.

[69] R. Land, *Voltage-Level Translation with the LSF Family*, Texas Instruments Application Report SLVA675B, Mar. 2015.

[70] Texas Instruments, *TPDxE05U06 1-, 4-, and 6-Channel ESD Protection Device for Super-Speed Interfaces*, Datasheet, Rev. O.

[71] Infineon Technologies, *BAT15-04W Silicon RF Schottky Diode*, Datasheet.

[72] Texas Instruments, *TPS7A8300 2-A, 6-μV_{RMS} RF LDO Voltage Regulator*, Datasheet, Rev. F, Oct. 2015.

[73] Texas Instruments, *TCA9535 Low-Voltage 16-Bit I2C/SMBus I/O Expander*, Datasheet, Rev. E, May 2022.

[74] ROHM Semiconductor, *DTC114E NPN Digital Transistor with Built-In Bias Resistors*, Datasheet.

[75] STMicroelectronics, *SMAJ Transil Series: 400-W TVS Diodes*, Datasheet.

[76] Texas Instruments, *TPS25925x/TPS25926x Simple 5-V/12-V eFuse Protection Switches*, Datasheet, Rev. B.

[77] Analog Devices, *LT3045 20-V, 500-mA, Ultralow-Noise, Ultrahigh-PSRR Linear Regulator*, Datasheet, Rev. D.

[78] Texas Instruments, *TLVH431/TLVH432 Low-Voltage Adjustable Precision Shunt Regulators*, Datasheet.

PHỤ LỤC

PHỤ LỤC A: CẤU TRÚC THƯ MỤC NGUỒN DỰ ÁN LA16

Tổng quan

Dự án LA16 được tổ chức theo hướng phân tách chức năng, trong đó tài liệu kỹ thuật, thiết kế phần cứng FPGA, phần mềm chạy trên máy chủ và chương trình sinh tín hiệu kiểm thử được đặt trong các thư mục độc lập. Cách tổ chức này phản ánh trực tiếp kiến trúc của hệ thống: ESP32 tạo các dạng sóng số, FPGA thực hiện lấy mẫu và nén dữ liệu, còn máy tính tiếp nhận các gói UDP để lưu trữ, phân tích và hiển thị kết quả. Việc phân chia mã nguồn theo từng nền tảng cũng giúp mỗi phân hệ có thể được phát triển, biên dịch và kiểm thử riêng trước khi tích hợp thành hệ thống hoàn chỉnh.

Cấu trúc logic của các thư mục và tệp nguồn chính trong dự án được trình bày như sau. Cây thư mục chỉ liệt kê các thành phần trực tiếp phục vụ thiết kế và thuyết minh hệ thống; các tệp trung gian do công cụ biên dịch sinh ra không được đưa vào danh sách.

```
LogicAnalyzer/
|-- docs/                       # Tai lieu ky thuat
|   |-- ARCHITECTURE.md         # Kien truc he thong va duong du lieu
|   |-- PROTOCOL.md             # Goi UDP va ban ghi RLE
|   `-- WIRING.md               # Dau noi vat ly giua ESP32 va FPGA
|-- hardware/                   # Thiet ke bo mach mo rong
|   `-- analog_frontend/        # Schematic, PCB 4 lop va mo phong SPICE
|-- fpga/                       # Ma nguon phan cung FPGA (Gowin)
|   |-- constraints/
|   |   |-- la16.cst            # Rang buoc chan vat ly PMOD
|   |   `-- la16.sdc            # Rang buoc xung nhip va thoi gian
|   |-- ip/
|   |   `-- pll_175m5.v         # IP rPLL tao clock lay mau
|   `-- rtl/
|       |-- la16_top.sv         # Module thuc the dinh
|       |-- sampler_rle.sv      # Lay mau va nen RLE theo pipeline
|       |-- async_fifo.sv       # FIFO bat dong bo dung con tro Gray
|       |-- packetizer.sv       # Dong goi va dieu khien truyen UDP
|       `-- i2c_master_core.sv  # I2C Master dieu khien DAC/GPIO expander
|-- host/                       # Phan mem may chu bang Python
|   |-- la16/
|   |   |-- __init__.py
|   |   |-- packet.py           # Phan tich cu phap goi UDP
|   |   |-- store.py            # Bo dem vong va truc thoi gian nguyen
|   |   `-- detect.py           # Nhan dien va giai ma giao thuc
|   |-- la_capture.py           # Chuong trinh thu thap dang dong lenh
|   |-- la_gui.py               # Giao dien do hoa PRISM
|   `-- requirements.txt        # Danh sach thu vien Python phu thuoc
`-- esp32/                      # Thiet bi sinh tin hieu kiem thu
    `-- testcase/
        |-- host/
        |   `-- run_testcases.py # Kiem thu hoi quy SIL
```

```
‘-- la16_testgen/       # Bo phat I2S LCD-parallel dung DMA
    |-- i2s16.cpp       # Driver DMA I2S0 muc thanh ghi
    |-- la16_paint.cpp  # Thu vien tao chin kich ban dang song
    ‘-- platformio.ini  # Cau hinh bien dich PlatformIO
```

Nhóm tài liệu kỹ thuật

Thư mục `docs/` đóng vai trò là lớp đặc tả chung của toàn bộ dự án. Các tài liệu trong thư mục này tạo ra một giao diện thống nhất giữa phần cứng, phần mềm và quá trình lắp đặt hệ thống. Nhờ đó, thay đổi tại một phân hệ có thể được đối chiếu với các yêu cầu liên quan trước khi cập nhật sang các phân hệ còn lại.

Tệp `ARCHITECTURE.md` mô tả kiến trúc tổng thể và đường đi của dữ liệu từ 16 đầu vào số đến máy tính. Theo đặc tả này, tín hiệu đầu vào được đồng bộ và lấy mẫu trong miền xung nhịp tốc độ cao, sau đó được biểu diễn bằng các bản ghi RLE, chuyển qua bộ đệm bất đồng bộ, ghép thành gói và truyền tới máy chủ. Tài liệu cũng giải thích sự phân chia miền xung nhịp, cơ chế chuyển miền clock và vai trò của từng khối RTL. Vì vậy, đây là tài liệu liên kết trực tiếp với các tệp trong `fpga/rtl/` và là cơ sở để kiểm tra tính nhất quán của kiến trúc phần cứng.

Tệp `PROTOCOL.md` đặc tả giao diện dữ liệu giữa FPGA và phần mềm máy chủ. Một gói UDP gồm phần tiêu đề nhận dạng, phiên bản giao thức, các cờ trạng thái, số lượng bản ghi, số thứ tự gói và tần số lấy mẫu. Mỗi bản ghi RLE chứa `delta` 32 bit, biểu diễn số chu kỳ lấy mẫu kể từ lần chuyển trạng thái trước, cùng `value` 16 bit, biểu diễn mức logic mới của 16 kênh. Đặc tả này được sử dụng đồng thời bởi `packetizer.sv` ở phía FPGA và `packet.py` ở phía máy chủ; do đó hai thành phần có thể trao đổi dữ liệu mà không phụ thuộc vào chi tiết cài đặt của nhau.

Tệp `WIRING.md` mô tả ánh xạ vật lý giữa các chân GPIO của ESP32 và 16 đầu vào của FPGA trên các đầu nối PMOD. Bên cạnh thứ tự kênh, tài liệu còn quy định mức điện áp logic và yêu cầu nối chung đất giữa hai bo mạch. Đây là cầu nối giữa tên tín hiệu trong mã nguồn, ràng buộc chân tại `la16.cst` và hệ thống dây nối thực tế. Việc duy trì bảng ánh xạ tập trung giúp hạn chế hiện tượng đảo kênh hoặc sai thứ tự bit khi đối chiếu dạng sóng trên giao diện máy chủ.

Nhóm thiết kế bo mạch analog và nguồn bảo vệ

Thư mục logic `hardware/analog_frontend/` đại diện cho bộ hồ sơ thiết kế bo mạch mở rộng, gồm schematic, layout PCB bốn lớp, tính toán linh kiện và mô phỏng SPICE. Bo mạch đặt giữa đầu probe và FPGA để thực hiện bảo vệ ESD, suy giảm bù tần số, kẹp quá áp, so sánh ngưỡng và dịch mức. Các khối chính gồm TPD1E05U06, BAT15-04W, TLV3801/TLV3601, MCP4728, TLV9064 và mạng kết cuối LVDS. Phần thiết kế này do Trần Quang Khải chủ trì và được liên kết với mô tả front-end trong Chương 3.

Hồ sơ nguồn của bo mạch còn bao gồm TPS259250 tại đầu vào USB-C, LT3045 cho rail analog 3,3 V nhiễu thấp và TLVH431 làm phần tử kẹp dòng dội ngược. Mạch TCA9535, TPS7A8300, LSF0204 và DTC114E tạo miền điện áp PWM điều khiển được. Các trạng thái an toàn và trình tự chuyển điện áp là giao diện chung giữa thiết kế analog và I2C Master trên FPGA; vì vậy thay đổi schematic phải được phản ánh đồng thời trong thanh ghi điều khiển và widget PRISM.

Nhóm mã nguồn phần cứng FPGA

Thư mục `fpga/` chứa toàn bộ thành phần cần thiết để tổng hợp thiết kế cho FPGA Gowin trên bo mạch Tang Primer 20K. Nội dung thư mục được chia thành ba nhóm: các ràng buộc triển khai trong `constraints/`, khối tạo xung nhịp trong `ip/` và mã RTL mô tả đường dữ liệu trong `rtl/`. Sự phân chia này tách biệt mô tả chức năng khỏi các tham số phụ thuộc bo mạch, nhờ đó thuận lợi hơn khi kiểm tra thời gian hoặc chuyển thiết kế sang cấu hình phần cứng khác.

Trong thư mục `constraints/`, tệp `la16.cst` ánh xạ các cổng logic của module đỉnh tới chân vật lý của FPGA. Đối với LA16, nhóm ràng buộc quan trọng nhất là 16 đầu vào lấy mẫu trên các đầu nối PMOD, bên cạnh các chân xung nhịp, reset, Ethernet và đèn báo trạng thái. Tệp này bảo đảm chỉ số bit của bus đầu vào trong `la16_top.sv` tương ứng với số kênh được quy định trong `WIRING.md`.

Tệp `la16.sdc` khai báo các ràng buộc thời gian cho quá trình tổng hợp và định tuyến. Các khai báo clock cho phép công cụ Gowin đánh giá đường truyền trong từng miền xung nhịp, trong khi các ràng buộc chuyển miền clock được đặt phù hợp với kiến trúc FIFO bất đồng bộ và các mạch đồng bộ nhiều tầng. Như vậy, `la16.cst` trả lời câu hỏi "tín hiệu được nối tới chân nào", còn `la16.sdc` quy định "tín hiệu phải đáp ứng yêu cầu thời gian nào".

Thư mục `ip/` chứa tệp `pll_175m5.v`. Đây là khối rPLL được cấu hình để biến đổi xung nhịp tham chiếu 27 MHz của bo mạch thành xung nhịp lấy mẫu 175,5 MHz. Việc đặt khối PLL trong một thư mục riêng giúp cô lập thành phần phụ thuộc công nghệ Gowin khỏi phần RTL mô tả chức năng. Tín hiệu báo khóa PLL cũng được module đỉnh sử dụng để chỉ cho phép đường lấy mẫu hoạt động sau khi xung nhịp đã ổn định.

Thư mục `rtl/` hiện thực đường dữ liệu chính của máy phân tích logic. Tệp `la16_top.sv` là module thực thể đỉnh, có nhiệm vụ khai báo các cổng vật lý và kết nối những khối chức năng còn lại. Module này nhận bus 16 kênh, phân phối các tín hiệu clock và reset, đồng thời nối chuỗi xử lý từ bộ lấy mẫu đến giao diện truyền dữ liệu. Vì vậy, `la16_top.sv` không tập trung vào một thuật toán riêng lẻ mà đóng vai trò điều phối cấu trúc của toàn bộ thiết kế FPGA.

Tệp `sampler_rle.sv` thực hiện đồng bộ tín hiệu đầu vào, lấy mẫu đồng thời 16

kênh và phát hiện sự thay đổi của từ dữ liệu 16 bit. Thay vì truyền mọi mẫu thô, module chỉ sinh bản ghi khi trạng thái đầu vào thay đổi. Khoảng thời gian giữa hai lần thay đổi được đếm theo số nguyên chu kỳ lấy mẫu và ghép với giá trị logic mới để tạo bản ghi {delta, value}. Phương pháp RLE theo chuyển mức giữ nguyên độ phân giải thời gian của cạnh tín hiệu, đồng thời giảm đáng kể lượng dữ liệu đối với UART, I2C, SPI và các tín hiệu số có thời gian ổn định dài.

Tệp async_fifo.sv tạo bộ đệm FIFO bất đồng bộ giữa miền lấy mẫu tốc độ cao và miền xử lý gói có tần số thấp hơn. Con trỏ đọc và con trỏ ghi được mã hóa Gray trước khi đồng bộ sang miền clock đối diện, nhờ đó tại mỗi bước chỉ có một bit con trỏ thay đổi và giảm nguy cơ đọc trạng thái trung gian không hợp lệ. FIFO vừa thực hiện chuyển miền clock an toàn, vừa hấp thụ các đợt dữ liệu phát sinh nhanh trong khoảng thời gian ngắn trước khi phía truyền mạng xử lý.

Tệp packetizer.sv đọc tuần tự các bản ghi từ FIFO, tạo phần tiêu đề theo PROTOCOL.md và ghép các bản ghi thành tải dữ liệu UDP. Khối này quản lý số lượng bản ghi trong gói, số thứ tự gói, cờ báo tràn và thông tin tần số lấy mẫu. Gói có thể được phát khi đã tích lũy đủ bản ghi hoặc khi hết thời gian chờ; trong trường hợp không có cạnh mới, gói heartbeat được sử dụng để máy chủ tiếp tục nhận biết trạng thái hoạt động của thiết bị. Như vậy, chuỗi xử lý trong FPGA có thể khái quát thành:

Tệp i2c_master_core.sv thuộc phần mở rộng điều khiển của thiết kế FPGA. Module tạo giao dịch I2C để ghi mã ngưỡng vào DAC MCP4728 và cấu hình TCA9535, qua đó điều khiển các chân ANY-OUT và enable của TPS7A8300, đồng thời đóng cắt transistor xả DTC114E. Khối này hoạt động ngoài đường timing lấy mẫu chính nhưng được tích hợp trong la16_top.sv; trạng thái lỗi hoặc timeout I2C phải đưa mạch về cấu hình nguồn an toàn.

la16_top.sv → sampler_rle.sv → async_fifo.sv → packetizer.sv → UDP,
i2c_master_core.sv → MCP4728 và TCA9535.

Nhóm phần mềm máy chủ

Thư mục host/ chứa phần mềm Python chạy trên máy tính, đảm nhiệm tiếp nhận dữ liệu UDP, tái dựng trục thời gian, lưu kết quả, nhận diện giao thức và hiển thị dạng sóng. Các chức năng lõi được đóng gói trong thư mục host/la16/, còn hai tệp thực thi ở cấp trên cung cấp chế độ làm việc dòng lệnh và chế độ giao diện đồ họa.

Tệp __init__.py đánh dấu la16/ là một package Python. Cách đóng gói này cho phép các chương trình thu thập và giao diện cùng nhập lại những module xử lý chung, tránh sao chép thuật toán phân tích gói, lưu trữ hoặc nhận diện giao thức giữa nhiều chương trình.

Tệp packet.py hiện thực bộ phân tích cú pháp cho giao thức được mô tả trong

PROTOCOL.md. Module kiểm tra trường nhận dạng và phiên bản, đọc các trường tiêu đề, xác định số lượng bản ghi, sau đó chuyển phần tải nhị phân thành các cặp delta–value. Số thứ tự gói cho phép phía máy chủ nhận biết gói bị mất, còn các cờ trạng thái giúp chuyển thông tin tràn FIFO từ phần cứng tới chương trình giám sát.

Tệp store.py quản lý bộ đệm vòng chứa các chuyển mức đã nhận. Thời điểm của mỗi chuyển mức được lưu dưới dạng tổng số chu kỳ nguyên, chỉ được đổi sang đơn vị thời gian thực khi cần hiển thị hoặc xuất dữ liệu. Cách biểu diễn này tránh sai số tích lũy do liên tục cộng các số dấu phẩy động. Bộ đệm vòng cho phép hệ thống thu liên tục trong khi vẫn giới hạn bộ nhớ và cung cấp nhanh một cửa sổ dữ liệu gần nhất cho giao diện hoặc thuật toán giải mã.

Tệp detect.py chứa các thuật toán phân tích đặc trưng chuyển mức nhằm phân loại tín hiệu và giải mã các giao thức số được hỗ trợ. Module sử dụng dữ liệu do store.py cung cấp để ước lượng chu kỳ, tần số, độ rộng xung và mối quan hệ giữa các kênh; từ đó nhận diện các tín hiệu UART, I2C, SPI hoặc clock/PWM và tạo thông tin giải mã tương ứng. Việc tách bộ nhận diện khỏi giao diện giúp thuật toán có thể được kiểm thử độc lập bằng các dạng sóng chuẩn do bộ sinh tín hiệu cung cấp.

Tệp la_capture.py là chương trình thu thập dạng dòng lệnh, phù hợp với chế độ chạy không có giao diện đồ họa. Chương trình mở cổng UDP, sử dụng packet.py để giải mã gói nhận được, cập nhật kho dữ liệu và có thể ghi kết quả ra tệp để phục vụ phân tích sau. Chế độ này cũng hữu ích khi cần kiểm tra nhanh kết nối mạng, tốc độ gói hoặc tình trạng mất gói trước khi khởi động toàn bộ giao diện.

Tệp la_gui.py xây dựng ứng dụng đồ họa PRISM cho người sử dụng. Giao diện lấy ảnh chụp dữ liệu từ bộ đệm, dựng dạng sóng 16 kênh, hỗ trợ quan sát và đo khoảng thời gian, đồng thời hiển thị kết quả nhận diện và giải mã. Phần thu dữ liệu được duy trì tách khỏi quá trình vẽ, nhờ đó thao tác tạm dừng hoặc thay đổi vùng quan sát không làm thay đổi trục thời gian đã được FPGA ghi nhận.

Tệp requirements.txt liệt kê các thư viện Python phụ thuộc của phần mềm máy chủ. Tệp này giúp thiết lập môi trường chạy có thể tái lập trên máy tính khác và bảo đảm chương trình dòng lệnh, giao diện cũng như các module xử lý sử dụng cùng một tập thư viện tương thích.

Nhóm sinh tín hiệu kiểm thử trên ESP32

Thư mục esp32/ chứa mã nguồn dành cho thiết bị tạo tín hiệu đầu vào chuẩn. Trong cấu trúc được thuyết minh, chương trình tại testcase/la16_testgen/ sử dụng ngoại vi I2S0 của ESP32 ở chế độ LCD-parallel 16 bit kết hợp với DMA. Mỗi từ 16 bit trong bộ đệm tương ứng với trạng thái tức thời của 16 kênh LA16. Vì các từ dữ liệu được phần cứng DMA phát theo cùng một clock, các kênh có quan hệ thời gian xác định và

không phụ thuộc vào độ trễ của thao tác điều khiển GPIO bằng phần mềm.

Tệp `i2s16.cpp` là lớp điều khiển mức thấp cho ngoại vi I2S0. Tệp thực hiện cấu hình chế độ LCD-parallel, ánh xạ các đường dữ liệu tới GPIO, thiết lập bộ chia clock và quản lý các mô tả DMA. Thành phần này cung cấp cơ chế phát lặp một bộ đệm mẫu đã chuẩn bị trước, qua đó giải phóng CPU khỏi nhiệm vụ thay đổi đồng thời 16 chân GPIO ở tốc độ cao.

Tệp `la16_paint.cpp` cung cấp các hàm "vẽ" dạng sóng vào bộ đệm mẫu 16 bit. Các hàm tạo mức logic, cạnh và khung giao thức được kết hợp thành chín kịch bản từ hiệu chuẩn đến kiểm thử UART, I2C, SPI, CAN, clock/PWM, hoạt động hỗn hợp, trường hợp biên và thử tải băng thông. Do toàn bộ dạng sóng được tính trước khi DMA phát, cùng một kịch bản có thể được dùng làm đầu vào chuẩn để đối chiếu kết quả lấy mẫu và giải mã trên máy chủ.

Tệp `run_testcases.py` chạy mô hình Software-in-the-Loop trên máy tính. Script sử dụng cùng tham số TC0–TC8, lượng tử hóa transition theo clock 175,5 MHz và gọi trực tiếp `detect.analyze()` để kiểm tra heuristic và decoder. Kết quả pass/fail cùng các tệp golden cung cấp chuẩn hồi quy trước khi chạy lại phép đo HIL bằng ESP32 và PRISM.

Tệp `platformio.ini` mô tả môi trường biên dịch PlatformIO, bao gồm nền tảng ESP32, bo mạch đích, framework và các tham số nạp chương trình. Việc lưu cấu hình cùng mã nguồn giúp cố định môi trường công cụ và giảm khác biệt giữa các máy phát triển khi xây dựng firmware kiểm thử.

Mối liên hệ giữa các thư mục

Các nhóm thư mục không hoạt động độc lập hoàn toàn mà được liên kết thông qua các giao diện đã xác định. Ở phía kiểm thử, `la16_paint.cpp` tạo bộ mẫu và `i2s16.cpp` chuyển bộ mẫu thành 16 tín hiệu điện trên các chân ESP32. Thứ tự dây nối được quy định bởi `WIRING.md` và phải thống nhất với ánh xạ chân trong `la16.cst`. Khi sử dụng bo mở rộng, tín hiệu còn đi qua front-end bảo vệ/comparator trước khi tới input buffer FPGA. Tại FPGA, `sampler_rle.sv` chuyển các tín hiệu này thành bản ghi RLE; `async_fifo.sv` chuyển bản ghi sang miền clock truyền; sau đó `packetizer.sv` tạo dữ liệu theo định dạng trong `PROTOCOL.md`. Song song với datapath, module I2C Master cấu hình MCP4728 và TCA9535 theo yêu cầu từ hệ thống điều khiển.

Ở phía máy chủ, `packet.py` thực hiện phép biến đổi ngược từ gói UDP về các bản ghi chuyển mức. Các bản ghi được `store.py` tích lũy thành trục thời gian, sau đó được `detect.py` phân tích và được `la_capture.py` hoặc `la_gui.py` cung cấp cho người sử dụng. Toàn bộ chuỗi xử lý có thể biểu diễn ngắn gọn như sau:

ESP32 sinh tín hiệu $\rightarrow$ FPGA lấy mẫu và nén RLE $\rightarrow$ đóng gói UDP $\rightarrow$ máy chủ lưu

trữ $\rightarrow$ nhận diện, giải mã và hiển thị.

Như vậy, cấu trúc thư mục nguồn của LA16 không chỉ nhằm phân loại tệp theo ngôn ngữ lập trình hoặc công cụ phát triển, mà còn thể hiện ranh giới chức năng và luồng dữ liệu của hệ thống. Cách tổ chức này hỗ trợ truy vết yêu cầu từ tài liệu đến mã nguồn, cho phép kiểm thử từng phân hệ và tạo thuận lợi cho việc bảo trì, mở rộng số thuật toán giải mã hoặc thay đổi nền tảng phần cứng trong tương lai.